



UNIVERSITATEA TEHNICĂ
DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
DEPARTAMENTUL CALCULATOARE

SUPORT CLINIC PENTRU DECIZII ÎN DEPARTAMENTUL DE URGENȚĂ

LUCRARE DE LICENȚĂ

Absolvent: **Ember Darius-Robert**

Coordonator **As. drd. ing. Roxana Ramona Szomiu**
științific:

2026

Cuprins

Capitolul 1	Introducere	1
Capitolul 2	Obiectivele proiectului	4
2.1	Obiectivul principal	4
2.2	Obiective specifice	4
2.3	Cerințe și constrângeri	5
2.4	Validarea obiectivelor	6
Capitolul 3	Studiu bibliografic	7
3.1	Învățare automată pentru triaj și predicția rezultatelor clinice	7
3.1.1	Modele clasice și benchmark-uri publice	7
3.1.2	Studii de sinteză și abordări emergente bazate pe NLP și LLM	8
3.2	Modele lingvistice mari și sisteme multi-agent în medicină	9
3.3	Sisteme multi-agent pentru triaj și fluxul clinic în UPU	9
3.4	Benchmark-uri și evaluarea raționamentului clinic al LLM-urilor	10
3.5	Poziționarea soluției propuse	11
Capitolul 4	Analiză și fundamentare teoretică	12
4.1	Formularea problemei de decizie clinică	12
4.2	Structura logică a sistemului multi-agent	13
4.3	Principiul hibrid: limbaj natural prin LLM, predicție prin învățare automată	14
4.4	Extragerea structurată a informației din limbaj natural	15
4.5	Modelul de predicție în cascadă	16
4.6	Fundamentele algoritmilor de învățare automată	17
4.6.1	Arbori de decizie cu gradient boosting	17
4.6.2	Reprezentarea textului prin TF-IDF	18
4.6.3	Dezechilibrul claselor și ponderarea atenuată	18
4.6.4	Predicția ordinală a acuității	19
4.6.5	Calibrarea probabilităților	19
4.6.6	Rezolvarea codului ICD exact prin căutare	20
4.7	Modelul caracteristicilor și absența scurgerilor de informație	20
Capitolul 5	Proiectare de detaliu și implementare	22
5.1	Arhitectura generală a aplicației	22
5.2	Orchestrarea agenților și fluxul de execuție	24
5.3	Agentul de parsare a limbajului natural	26
5.4	Agentul de triaj	27
5.5	Agentul doctor	30
5.5.1	Evaluarea inițială	30
5.5.2	Rafinarea dispoziției	32
5.5.3	Reevaluarea și recuperarea codului ICD exact	33
5.6	Agentul asistent	35
5.7	Pipeline-ul de antrenare și modelul de date	37
5.8	Interfața web și runtime-ul conversațional	39

Capitolul 6 Testare și validare	43
6.1 Metodologia de evaluare	43
6.2 Evaluarea agentului de triaj	44
6.2.1 Predicția acuității (ESI 1–5)	44
6.2.2 Predicția dispoziției (internare / externare)	45
6.2.3 Evoluția pe versiuni a triajului	46
6.3 Evaluarea agentului doctor	46
6.3.1 Diagnosticul (13 categorii)	47
6.3.2 Departamentul (11 servicii)	48
6.3.3 Rafinarea dispoziției de către doctor	48
6.3.4 Robustetea dispoziției pe subgrupuri clinice	49
6.3.5 Recuperarea codului ICD exact	50
6.3.6 Evoluția pe versiuni a doctorului	52
6.4 Validarea cap la cap pe cazuri în limbaj natural	53
6.5 Compararea backendurilor de limbaj	54
6.6 Auditul configurației prin optimizarea hiperparametrilor	55
6.7 Discuție și limitări	56
Capitolul 7 Manual de instalare și utilizare	58
7.1 Cerințe software și hardware	58
7.2 Instalarea aplicației	59
7.2.1 Obținerea codului și a dependențelor	59
7.2.2 Configurarea backendului de limbaj	59
7.2.3 Pregătirea interfeței web	59
7.2.4 Reantrenarea modelelor (opțional)	60
7.3 Utilizarea din linia de comandă	60
7.4 Utilizarea interfeței web	61
7.4.1 Pornirea interfeței	61
7.4.2 Parcurgerea unei consultații	62
Capitolul 8 Concluzii	65
Bibliografie	67
Anexa A Secțiuni relevante din cod	71
A.1 Algoritmul de ordonare a codurilor ICD	71
A.2 Construirea antecedentelor fără scurgeri de informație	71
A.3 Reprezentarea în timp a semnelor vitale	72
A.4 Mecanismul de comunicare cu utilizatorul	73
Anexa B Lucrări publicate (dacă există)	74

Capitolul 1. Introducere

Unitatea de Primiri Urgențe (UPU) este primul loc din spital unde sunt preluați și îngrijiți pacienții care au nevoie de asistență medicală de urgență. De multe ori, este și cea mai aglomerată. Fiecare pacient care sosește trece printr-un proces de triaj în care un asistent îi evaluează gravitatea și îi stabilește prioritatea. Scopul este ca resursele medicale limitate să ajungă întâi la cazurile cu cel mai mare risc. În practica internațională, această evaluare folosește standarde de acuitate precum scala *Emergency Severity Index* (ESI). Această scală încadrează pacientul pe cinci niveluri posibile, de la intervenția imediată până la cazul neurgent [1]. Triajul nu este însă o sarcină simplă. Deoarece se face rapid, sub presiunea timpului și a volumului de pacienți, pe baza unei descrieri exprimate de pacient în limbaj comun, decizia depinde mult de experiența persoanei care o ia.

Această dependență de factorul uman introduce o variabilitate implicită. Există două erori posibile care pot fi făcute, însă efectele lor sunt foarte diferite. Sub-triajul înseamnă că starea reală de gravitate a unui pacient este subestimată. Din acest motiv, persoana poate primi ajutor medical mai târziu decât ar trebui, iar acest lucru este asociat cu un risc de mortalitate ridicat. Supra-triajul înseamnă că pacienții sunt considerați mai gravi bolnavi decât sunt cu adevărat în realitate. Asta duce la folosirea inutilă a resurselor medicale, fără vreun beneficiu real pentru pacient. Sintezele recente despre utilizarea inteligenței artificiale în triaj arată că ambele tipuri de eroare apar încă destul de des și că utilizarea unor sisteme automate de sprijin poate reduce atât numărul erorilor de triaj, cât și timpul necesar pentru evaluarea pacienților [2, 3]. Aglomerarea din UPU și creșterea volumului de pacienți, probleme care s-au accentuat în timpul pandemiei COVID-19, au transformat acest tip de sprijin într-un domeniu important de cercetare.

Triajul este însă numai primul pas dintr-un șir de decizii pe care le presupune trecerea unui pacient prin UPU. După ce este stabilită gravitatea, urmează decizia de internare sau externare. Pentru pacienții internați urmează orientarea către un diagnostic, direcționarea către un departament și atribuirea unui cod de diagnostic. Fiecare dintre aceste decizii a fost studiată de obicei separat, ca o sarcină independentă. În realitate ele sunt legate între ele, pentru că rezultatul unei etape limitează opțiunile etapei următoare. Un pacient care a fost evaluat ca fiind neurgent are șanse slabe de a fi internat, iar diagnosticul ghidează direct departamentul în care va ajunge. Un sistem de sprijin clinic care se oprește la nivelul acuității acoperă doar o mică parte din parcursul real al pacientului. Din acest motiv, lucrarea de față își propune să modeleze lanțul complet de decizii, nu doar o parte.

În jurul acestei probleme s-au format două direcții tehnice diferite. Luate separat, fiecare are un punct slab clar. Prima folosește modele de învățare automată antrenate pe datele electronice ale departamentului de urgență. Aceste modele oferă rezultate care pot fi reproduse și verificate, iar pe baze de date publice precum MIMIC-IV-ED, cu aproximativ 425.000 de prezentări reale, ating performanțe solide [4, 5]. Dezavantajul lor este că rezolvă de obicei o singură sarcină tabulară, spitalizarea sau îngrijirea critică, și nu pot procesa direct descrierea în limbaj natural a pacientului. A doua direcție folosește modele lingvistice mari (*Large Language Models*, LLM) și sisteme cu mai mulți agenți. Acestea înțeleg limbajul natural și pot simula un flux clinic întreg. Problema lor este

că sunt costisitoare, greu de auditat și încă predispuse la halucinații, adică pot produce afirmații plauzibile, dar false. Evaluările recente arată că aceste sisteme obțin creșteri modeste de acuratețe pe sarcini clinice și au un cost de calcul ridicat, deși au acces la instrumente puternice. [6, 7].

Această lucrare pornește de la ideea că cele două abordări nu ar trebui să fie alese separat, ci folosite împreună, astfel încât fiecare să compenseze punctele slabe ale celeilalte. Sistemul propus este o arhitectură cu mai mulți agenți, construită pe framework-ul CrewAI și simulează parcursul unui pacient prin UPU. Pacientul își descrie starea în limbaj natural, iar un lanț de agenți specializați duce cazul prin toate etapele: triaj, evaluare medicală inițială, colectarea de date suplimentare de către un asistent și o reevaluare finală. Decizia de proiectare principală este limita strictă pe care o impune rolului LLM-ului. LLM-ul este folosit *exclusiv* pentru a înțelege textul natural al pacientului și a-l transforma într-o reprezentare structurată. Toate predicțiile clinice, precum acuitatea ESI, decizia de internare sau externare, diagnosticul, departamentul și codul de diagnostic ICD, sunt realizate de modele supervizate de tip XGBoost, antrenate pe prezentările reale din MIMIC-IV-ED. Astfel, LLM-ul păstrează strict responsabilitatea de a procesa limbajul natural, iar decizia clinică rămâne responsabilitatea învățării automate, care este verificabilă și reproductibilă.

Cele cinci predicții nu sunt independente, ci înlanțuite într-o cascadă care urmează ordinea naturală a fluxului clinic. Gravitatea evaluată la triaj influențează decizia de internare, care delimitează pacienții pentru care diagnosticul și departamentul au sens, iar diagnosticul orientează alegerea departamentului. O a doua particularitate a sistemului este că evaluarea medicului se face în două momente, înainte și după ce asistentul colectează semnele vitale, medicația și antecedentele. Această separare permite măsurarea directă a valorii clinice aduse de datele suplimentare. Comparatia aceasta lipsește din majoritatea lucrărilor din domeniu. Întrebarea de cercetare urmărită explicit este astfel cât de mult se îmbunătățesc predicțiile atunci când la datele inițiale disponibile la sosire se adaugă semnele vitale, medicația și antecedentele. Răspunsul la această întrebare se obține comparând cele două evaluări produse pe același caz.

Fundamentul empiric al sistemului este setul de date MIMIC-IV-ED, care conține aproximativ 425.000 de prezentări reale la urgență. Aceste prezentări conțin simptomele semnalate de pacient, semnele vitale de la triaj, datele demografice, dispoziția finală și codurile de diagnostic asociate fiecărei prezentări [4]. Etichetele necesare pentru toate cele cinci sarcini de predicție provin direct din aceste înregistrări, ceea ce permite formularea fiecărei sarcini ca o problemă de clasificare bine definită și antrenarea modelelor pe cazuri reale, nu pe date sintetice. Tot din aceste înregistrări se construiesc și antecedentele medicale ale unui pacient cunoscut. Acestea sunt adunate din prezentările sale anterioare. Volumul mare și diversitatea acestei baze de date permit modelarea întregului lanț de decizii cu modele antrenate separat pentru fiecare etapă.

Domeniul lucrării este delimitat cu precizie. Sistemul acoperă exclusiv parcursul din cadrul UPU, de la sosirea pacientului până la decizia de dispoziție și direcționarea către un departament, dar nu intră în zona terapiei intensive sau a tratamentului propriu-zis. Rolul său este de a *sprîji* decizia clinică, nu de a o înlocui. Predicțiile sunt orientative și rămân sub controlul personalului medical calificat. Datele folosite pentru antrenare și evaluare provin din MIMIC-IV-ED, o bază de date în limba engleză. Toate rezultatele raportate sunt obținute pe seturi de test reținute, separate de cele de antrenare, astfel încât performanța măsurată reflectă comportamentul pe cazuri nevăzute.

Relevanța practică a unei astfel de soluții constă în faptul că acoperă întregul flux

clinic, dar menține în același timp controlul în mâinile personalului medical. Un asistent de triaj suprasolicitat poate beneficia de o a doua opinie generată automat pentru fiecare pacient, fără influența oboselii sau a variațiilor de judecată care apar în mod normal într-o tură foarte aglomerată. Pentru că fiecare predicție poate fi urmărită până la datele care au produs-o, sistemul rămâne un instrument de sprijin ușor de verificat de către personalul medical, și nu o cutie neagră care ia decizii în locul lui. Această proprietate este și motivul pentru care predicția clinică este lăsată în seama unor modele auditabile, iar LLM-ul este folosit doar ca suport, în zonele secundare ale procesului, acolo unde aduce un avantaj real fără a prelua decizia.

Restul lucrării este organizat după cum urmează. Capitolul 2 formulează tema ca o problemă de cercetare și enumeră obiectivele concrete. Capitolul 3 analizează literatura din cele trei direcții relevante și poziționează soluția propusă. Capitolul 4 stabilește fundamentul teoretic, de la formularea problemelor de decizie până la algoritmi de învățare automată folosiți. Capitolele următoare descriu implementarea efectivă, testarea și validarea pe date reale, manualul de utilizare și concluziile.

Capitolul 2. Obiectivele proiectului

Tema lucrării este formulată ca o problemă de cercetare. Pornind de la descrierea în limbaj natural a unui pacient care ajunge la Unitatea de Primiri Urgențe, sistemul trebuie să producă întregul lanț de decizii clinice din UPU. Fiecare dintre aceste decizii trebuie să fie explicabilă și reproductibilă. Provocarea nu este dată de o singură predicție izolată, ci de acoperirea fluxului complet, de la gravitate până la codul de diagnostic, păstrând în același timp trasabilitatea fiecărei decizii. Această cerință dublă, înțelegerea textului liber al pacientului pe de o parte și generarea unei predicții clinice care poate fi explicată și verificată pe de altă parte, delimitează problema pe care lucrarea o rezolvă și stă la baza obiectivelor de mai jos.

Pe lângă partea de proiectare, lucrarea are și o componentă de cercetare. Întrebarea pe care o urmărește este cât de mult se îmbunătățesc predicțiile clinice atunci când la datele sumare disponibile la triaj se adaugă semnele vitale, medicația și antecedentele colectate ulterior. Acest răspuns nu poate fi dat de un model izolat, ci are nevoie de un sistem care produce o predicție înainte și una după colectarea acestor date, pe același pacient și în aceleași condiții. Evaluarea dublă este motivul pentru care fluxul este împărțit pe mai multe etape și pentru care responsabilitățile sunt separate între agenți distincți, fiecare cu o intrare și o ieșire bine definite.

2.1. Obiectivul principal

Obiectivul principal este proiectarea și implementarea unui sistem cu mai mulți agenți pentru sprijinirea deciziei în UPU, în care un model lingvistic mare este folosit numai pentru a transforma descrierea naturală a pacientului într-o reprezentare structurată, iar toate predicțiile clinice sunt realizate de modele de învățare automată supervizată, antrenate pe date reale. Sistemul trebuie să parcurgă secvențial etapele fluxului clinic, adică triaj, evaluare medicală inițială, colectarea de date suplimentare și reevaluare. De asemenea, trebuie să permită compararea predicțiilor făcute înainte și după colectarea acestor date, pentru a măsura impactul clinic al acestora.

2.2. Obiective specifice

Obiectivul principal este împărțit în mai multe obiective specifice. Fiecare dintre acestea se referă la o parte concretă a sistemului:

- **O1. Extragerea structurată a informației din limbaj natural.** Realizarea unui agent de parsare bazat pe LLM care convertește descrierea naturală a pacientului într-o schemă fixă de câmpuri (simptome principale, scor al durerii, vârstă, sex, mod de sosire, antecedente, medicație). Atunci când un câmp relevant lipsește, agentul poate adresa întrebări specifice pentru a completa informația.
- **O2. Predicția de triaj.** Antrenarea a două modele XGBoost care estimează acuitatea ESI pe cinci niveluri, tratată ca problemă ordinală, și probabilitatea de internare. Acesta pornește de la simptome, date demografice, semne vitale (când există, adică în cazul în care pacientul a fost adus cu ambulanța sau cu elicopterul) și antecedente.

- **03. Predicția în cascadă.** Înlănțuirea modelelor astfel încât ieșirea fiecăruia să aibă rol de intrare pentru etapa următoare. Este urmată ordinea acuitate, dispoziție, diagnostic, departament, pentru a pune în evidență dependențele reale dintre deciziile clinice înlănțuite.
- **04. Evaluarea medicală înainte și după colectarea datelor.** Implementarea unui agent doctor care formulează o evaluare inițială pe baza datelor de la sosire și o reia după ce un agent asistent colectează interactiv semne vitale, medicație și antecedente. Acest lucru determină direct valoarea adusă de aceste informații suplimentare.
- **05. Rafinarea dispoziției de internare/externare și recuperarea codului ICD (International Classification of Diseases, un sistem standardizat de clasificare și codificare a bolilor, afecțiunilor și cauzelor de deces, utilizat la nivel mondial pentru diagnostic) exact.** Adăugarea unui model binar calibrat de internare sau externare (dispoziție), mai precis decât concluzia de triaj, și a unei etape de ordonare care recuperează codul ICD exact în interiorul categoriei de diagnostic prezise.
- **06. Implementarea unui sistem de parsare modular (cu LLM comutabil, limitat la parsare).** Posibilitatea de a comuta între un model lingvistic comercial (Gemini 2.5 Flash) și unul specializat medical, auto-găzduit (MedGemma). Rolul LLM-ului este păstrat strict la înțelegerea limbajului natural și nu este lăsat să intervină în predicția clinică.
- **07. Obținerea unui runtime interactiv.** Reprezentarea sistemului prin intermediul unei interfețe web conversaționale care rulează același flux de agenți ca varianta din linia de comandă, oferind un mediu interactiv pentru utilizarea și demonstrarea sistemului.
- **08. Evaluare reproductibilă.** Validarea sistemului prin intermediul unui set de benchmark-uri rulate pe date de test reținute din MIMIC-IV-ED, astfel încât performanțele obținute să poată fi reproduse și comparate în mod obiectiv.

2.3. Cerințe și constrângeri

Pe lângă funcțiile pe care le oferă, soluția ține cont și de câteva constrângeri care apar din specificul clinic al problemei. Predicțiile trebuie să fie:

- *explicabile*;
- *verificabile*;
- *reproductibile*.

Acest lucru motivează folosirea modelelor bazate pe arbori în locul unei decizii generate de LLM. Caracteristicile folosite la antrenare trebuie să fie *lipsite de scurgeri de informație (data leakage)*, adică să conțină numai date care ar fi fost disponibile în momentul deciziei reale. Domeniul este limitat la traseul pacientului din UPU, fără a include secția de terapie intensivă, iar rolul sistemului rămâne consultativ. Antrenarea și inferența trebuie să poată fi rulate pe hardware obișnuit, fără infrastructură specializată. Pașii de preprocesare a textului trebuie să fie identici între antrenare și inferență, pentru ca modelul să primească în producție exact tipul de date pentru care a fost antrenat.

Structura generală a fluxului propus, de la descrierea pacientului până la rezultatele produse de cei patru agenți, este prezentată în Figura 2.1. Detalierea teoretică a fiecărei etape se face în Capitolul 4, iar descrierea implementării în capitolul de proiectare de detaliu.

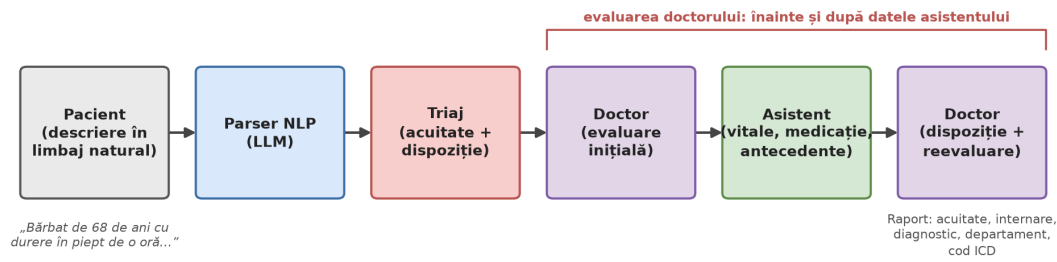


Figura 2.1: Privire de ansamblu asupra fluxului propus și a obiectivelor, din perspectiva parcursului unui pacient prin sistem.

2.4. Validarea obiectivelor

Fiecare obiectiv este însoțit de o metodă clară de verificare, astfel încât îndeplinirea lui să poată fi demonstrată prin rezultate concrete, nu doar afirmată. Modelele de triaj și cele ale doctorului sunt evaluate pe seturi de test reținute din MIMIC-IV-ED. Aceste date sunt separate de datele de antrenare, folosind metrice adaptate fiecărei sarcini. Pentru acuitate se folosesc acuratețea și acordul ordinal, pentru dispoziție se folosesc aria sub curba ROC și eroarea de calibrare, iar pentru diagnostic și departament se folosește acuratețea în primele poziții. Valoarea datelor colectate de asistent, care fac parte din obiectivul O4, se măsoară prin diferența dintre predicția inițială și cea de după colectare și sunt calculate pe aceiași pacienți. Întregul lanț, de la descrierea în limbaj natural până la predicția finală, este verificat printr-un set de cazuri de test exprimate în limbaj natural. Aceste cazuri sunt construite din prezentări tabulare reale și permit compararea fluxului complet cu varianta ce primește direct datele structurate. În acest mod, costul introdus de înțelegerea limbajului natural este izolat. Rezultatele acestor evaluări sunt prezentate în capitolul de testare și validare.

Capitolul 3. Studiu bibliografic

Triajul în Unitatea de Primiri Urgențe (UPU) este procesul prin care pacienții sunt evaluați și prioritizați în funcție de cât de gravă este starea lor, astfel încât resursele medicale limitate să poată fi alocate cazurilor cu cel mai mare risc. În practica internațională, acest proces este standardizat prin scale de acuitate precum *Emergency Severity Index* (ESI), cu cinci niveluri care ghidează decizia asistentului de triaj [1]. Deși este util, triajul efectuat de personalul uman poate fi influențat de factori precum prezența variațiilor între evaluatori, volumul crescut de pacienți și erori de sub-triaj (subevaluarea gravității) sau supra-triaj (supraevaluarea acesteia). Ambele erori aduc consecințe grave asupra siguranței pacientului și a utilizării resurselor [2, 3]. Aglomerarea în cadrul UPU și creșterea volumului de pacienți, accentuate de pandemia COVID-19, au transformat sprijinirea deciziei clinice prin inteligență artificială într-o direcție de cercetare activă [5, 8].

Lucrarea de față se află la mijlocul a trei direcții de cercetare care sunt analizate în acest capitol:

- (1) modelele de *învățare automată* (*machine learning*, ML) pentru triaj și predicția rezultatelor clinice în UPU;
- (2) modelele lingvistice mari (*Large Language Models*, LLM) și sistemele *multi-agent* aplicate în medicină;
- (3) benchmark-urile dedicate evaluării raționamentului clinic al LLM-urilor.

3.1. Învățare automată pentru triaj și predicția rezultatelor clinice

3.1.1. Modele clasice și benchmark-uri publice

O metodă clasică de cercetare aplică modele de Machine Learning asupra datelor de triaj pentru a prezice rezultate clinice. Raita et al. [9] dezvoltă pe datele *National Hospital Ambulatory Medical Care Survey* (NHAMCS) patru modele: regresie Lasso, *random forest*, *gradient boosted decision tree* și Deep Neuronal Network, pentru a prezice îngrijirea critică (internare la secția de terapie intensivă sau deces în spital) și spitalizarea, folosind ca model de referință o regresie logistică bazată pe nivelul ESI. Pe cele aproximativ 135.000 de intrări analizate, toate modelele de ML reușesc să depășească semnificativ referința ESI: pentru îngrijirea critică, aria de sub curba ROC (AUC)¹ atinge 0,86 față de 0,74 a referinței, iar pentru spitalizare 0,82 față de 0,69, cu mai puține cazuri de sub-triaj în nivelurile ESI 3-5.

Yao et al. [10] propun un sistem de *deep learning* care transformă datele structurate din fișa electronică în text, procesându-le folosind o arhitectură convoluțională și recurentă cu mecanism de atenție, obținând o acuratețe de 0,83 și un AUROC de 0,87–0,88 pentru predicția spitalizării pe seturi de date de peste 118.000 și, respectiv, 745.000 de pacienți.

Vântu et al. [11], de la Universitatea Transilvania din Brașov, compară regresia logistică, *random forest* și o rețea neuronală secvențială pentru clasificarea pacienților pe

¹AUC sau AUROC (*Area Under the Receiver Operating Characteristic Curve*) reprezintă aria de sub curba ROC și măsoară capacitatea unui model de clasificare de a distinge între clase. O valoare de 0,5 indică o performanță echivalentă unei alegeri aleatorii, iar o valoare de 1,0 indică o clasificare perfectă.

coduri de urgență ESI. Acesta evidențiază dificultatea cauzată de dezechilibrul claselor și beneficiile pe care le aduc tehnicile de echilibrare a datelor.

Un obstacol des întâlnit al acestor studii este că modelele sunt dezvoltate specializat, pentru o singură sarcină și pe un singur set de date. Acest lucru le face greu de comparat și generalizat [5]. Pentru a aborda această problemă, Xie et al. [5] introduc un *benchmark* public bazat pe MIMIC-IV-ED cu peste 400.000 de intrări în UPU între 2011 și 2019, cu trei sarcini de predicție standardizate: spitalizare, rezultate critice și reprezentare în UPU în 72 de ore. Pe lângă pipeline-ul reproductibil de preprocesare, autorii implementează și compară diferite modele de referință (regresie logistică, *random forest*, rețele neuronale și scoruri de avertizare timpurie). Acest benchmark, alături de bazele de date publice de tip MIMIC-IV-ED [4] și eICU, care reunește peste 200.000 de internări în terapie intensivă din mai multe centre [12], reprezintă o infrastructura de date din care se inspiră și această lucrare.

3.1.2. Studii de sinteză și abordări emergente bazate pe NLP și LLM

Două sinteze recente confirmă maturizarea domeniului. Tyler et al. [2] face o analiză de tip *scoping review* asupra utilizării inteligenței artificiale în triajul din UPU, iar Abdalhalim et al. [13] face o analiză sistematică conformă PRISMA. Din 119 înregistrări identificate, șase studii îndeplinesc criteriile de includere și raportează că sistemele de IA pot reduce timpul de triaj și rata de mis-triaj (cu 0,3–8,9%) și pot îmbunătăți acuratețea documentării. Totuși, aceste sisteme suferă de anumite riscuri, precum sub-triaj, variabilitate și faptul că sunt validate pe un singur set de date dintr-o anumită țară/regiune. Concluzia des întâlnită a acestor sinteze este necesitatea unor *algoritmi explicabili* și a validării în cadrul unor populații și condiții diverse. Acest lucru susține direct opțiunea noastră pentru modele de predicție care sunt transparente și reproductibile.

Recent, există tendințe de integrare a procesării limbajului natural (NLP) și a LLM-urilor în fluxul de triaj, întrucât simptomul principal (*chief complaint*) și notele clinice sunt exprimate în limbaj natural de către pacient. Lansiaux et al. [14] compară trei arhitecturi de IA pentru predicția nivelului de triaj conform scalei franceze FRENCH: un model NLP, un model LLM și o arhitectură predictivă cu *joint embedding* (JEPA). Modelul cel mai performant, URGENTIAPARSE, combină un model lingvistic (FlauBERT) cu un clasificator **XGBoost**, și obține un scor F1 de 0,900 și un AUC-ROC de 0,879, astfel depășind atât celelalte arhitecturi, cât și triajul efectuat de asistenți reali. Acest rezultat este relevant pentru lucrarea de față, deoarece confirmă exact combinația pe care am ales-o: un model lingvistic pentru înțelegerea textului natural al pacientului, urmat de un clasificator XGBoost pentru decizia propriu-zisă. Oliveira et al. [8] ilustrează, pe cazul triajului pacienților COVID-19, robustețea și limitele acestor modele în condiții de deplasare a distribuției datelor (*domain shift*).

Spre deosebire de aceste exemple discutate, care se concentrează în principal pe o singură etapă (predicția nivelului de acuitate), sistemul nostru extinde fluxul mai departe de triaj. Fluxul se extinde către dispoziție, diagnostic, alegerea departamentului și determinarea codului ICD. De asemenea, acesta separă clar rezultatele obținute înainte și după ce sunt colectate datele de către asistentul medical. Astfel, putem vedea clar valoarea acestor informații suplimentare (precum PMH) și cât de mult îmbunătățesc rezultatele predicțiilor făcute.

3.2. Modele lingvistice mari și sisteme multi-agent în medicină

În ultimul timp, sistemele multi-agent bazate pe LLM s-au răspândit rapid. Guo et al. [15] oferă o sinteză structurată a acestui domeniu, organizând cercetarea în jurul domeniilor și mediilor simulate, al modului în care agenții sunt specializați și comunică, și al mecanismelor prin care își dobândesc capacitățile. Yan et al. [16] propun o perspectivă *centrată pe comunicare*. Acesta face o distincție între comunicarea la nivel de sistem (arhitectură, obiective, protocoale) și cea internă (strategii, paradigme, conținut), și identifică drept provocări majore eficiența comunicării, vulnerabilitățile de securitate, scalabilitatea și lipsa benchmark-urilor. Pe baza acestora, noi putem să poziționăm sistemul propus ca un pipeline *secvențial* deterministic în care fluxul de informație urmează ordinea naturală a fluxului clinic, în locul unor arhitecturi de tip dezbatere sau negociere între diferiți agenți.

Yang et al. [17] propun LLM-MedQA. Acesta este un sistem multi-agent care îmbunătățește răspunsul la întrebări din domeniul medical prin *generarea de cazuri similare*, obținând o creștere de aproximativ 7% a acurateței și a scorului F1 pe setul MedQA [18], fără antrenare suplimentară. Ideea de a genera cazuri pentru a sprijini raționamentul prezintă o paralelă directă cu *agentul de generare de cazuri* din sistemul nostru, care este folosit însă într-un scop diferit: construirea, în mod offline, a unor cazuri de test convertite în limbaj natural, extrase din date tabulare reale, pentru a evalua pipeline-ul într-un mod corect, pe baza unor cazuri exprimate în limbaj natural. Spre deosebire de aceste lucrări, care sunt orientate către răspunsul la întrebări de tip examen, sistemul propus operează asupra fluxului clinic real, pe baza datelor electronice din cadrul UPU.

3.3. Sisteme multi-agent pentru triaj și fluxul clinic în UPU

Cele mai apropiate lucrări de sistemul propus combină arhitectura multi-agent cu fluxul de lucru din UPU. Chen et al. [7] introduc MAP (*Multi-Agent Inpatient Pathways*) și benchmark-ul IPDS, derivat din dataset-ul MIMIC-IV, cu 51.274 de cazuri repartizate pe nouă departamente de triaj, 17 categorii majore de boli și 16 opțiuni de tratament standardizate. MAP folosește patru agenți LLM: un agent de triaj care gestionează internarea, un agent de diagnostic care ia deciziile principale la nivel de departament, un agent de tratament și un agent “șef” care coordonează ceilalți trei agenți. Acesta semnalează o creștere de 25,10% a acurateței diagnosticului în comparație cu modelul de referință HuatuoGPT2-13B, depășind trei clinicieni certificați cu 10–12% la conformitatea clinică. Lu et al. [19] propun TriageAgent, un sistem multi-agent în care agenții LLM joacă roluri în discuții pe mai multe runde, cu mecanisme de auto-încredere și oprire timpurie, și folosesc manualul ESI printr-o abordare de tip *retrieval-augmented generation* (RAG). De asemenea, autorii publică și primul benchmark public de triaj clinic cu niveluri ESI și performanță umană de referință.

Două lucrări folosesc aceeași infrastructură ca sistemul nostru, dar cu obiective diferite. Ruiz Mejia și Rawat [20] prezintă MedScrubCrew, un sistem multi-agent construit pe framework-ul *CrewAI*, la fel ca și sistemul nostru. Acesta combină agenți LLM cu grafuri de cunoștințe și cu algoritmul de cuplare stabilă Gale-Shapley pentru automatizarea programării și a clasificării de triaj. Accentul este însă pe optimizarea logistică (alocarea pacient-furnizor), nu pe predicția clinică propriu-zisă. Srirag et al. [21] propun TriageSim, un sistem care generează conversații artificiale de triaj pe baza datelor structurate din MIMIC-IV-ED. Acesta folosește o arhitectură multi-agent formată dintr-un agent regizor

de dialog, un agent asistent medical și un agent pacient, și produce aproximativ 800 de transcrieri și înregistrări audio. TriageSim prezintă o paralelă remarcabilă cu doi dintre agenții noștri, agentul asistent (*nurse*), care colectează interactiv datele de la pacient (precum semne vitale, antecedente anterioare, etc) și agentul de generare de cazuri. Scopul său, însă, se oprește la *generarea* datelor conversaționale, fără a antrena și valida un flux predictiv complet pe baza acestora.

Diferența esențială față de aceste lucrări și principala contribuție de proiectare a sistemului propus constă în *rolul atribuit LLM-ului*. În MAP și TriageAgent, LLM-ul este *predictorul* care produce diagnosticul și decizia de triaj prin raționament generativ. În sistemul nostru, LLM-ul (Gemini 2.5 Flash, respectiv MedGemma, comutabile) este folosit *exclusiv* pentru parsarea limbajului natural, în timp ce fiecare predicție clinică este produsă de un model XGBoost antrenat pe aproximativ 425.000 de prezentări reale, ceea ce oferă predicții verificabile și reproductibile. De asemenea, în locul unui agent orchestrator bazat pe LLM, coordonarea se realizează printr-o cascadă deterministă, iar evaluarea medicului este efectuată în mod explicit *înainte* și *după* colectarea semnelor vitale și a medicației de către agentul asistent. Astfel, este evidențiată valoarea reală a datelor colectate de către agentul asistent, o comparație critică care nu este prezentă în celelalte lucrări.

3.4. Benchmark-uri și evaluarea raționamentului clinic al LLM-urilor

Mai multe lucrări recente evaluează critic limitele LLM-urilor în sarcini clinice și motivează indirect alegerea noastră de a păstra predicția în zona ML. Liu et al. [6] evaluează sistematic sisteme agentice (OpenManus, bazat pe Llama-4, și Manus) pe trei familii de benchmark-uri (AgentClinic, MedAgentsBench și *Humanity's Last Exam*) și realizează că agenții obțin doar creșteri modeste de acuratețe, deși au acces la instrumente puternice: 60,3% pe AgentClinic MedQA, 28,0% pe subsetul MIMIC-IV și 30,3% pe MedAgentsBench, cu un cost de peste 10 ori mai mare în consum de *token*-uri și halucinații care rămân prezente deși există filtre interne care ar trebui să prevină acest lucru. Gaber et al. [3] evaluează modele din familia Claude și un flux asistat de RAG (cu o bază de 30 de milioane de rezumate PubMed) pe 2.000 de cazuri din MIMIC-IV-ED, pentru predicția nivelului ESI, a specialității de referință și a diagnosticului, arătând că LLM-urile pot atenua atât rata de sub-triaj, cât și cea de supra-triaj.

La nivel de evaluare a raționamentului, Mehandru et al. [22] introduc ER-Reason, un benchmark care acoperă întregul flux de la urgență (triaj, selecția tratamentului, dispoziție și diagnostic final) pe baza a 25.174 de note clinice de la 3.437 de pacienți. Studiul evidențiază diferența dintre performanța pe evaluări de tip examen și cea pe cazuri clinice reale. Wu et al. [23] propun MedCaseReasoning, cu 14.489 de cazuri însoțite de raționament clinic. Se constată că cel mai bun model open-source, DeepSeek-R1, atinge doar 48% acuratețe diagnostică. Totuși, antrenarea suplimentară pe exemple ce conțin raționamentul clinic complet produce îmbunătățiri relative de aproximativ 29%. Hou et al. [24] propun ClinicalMC. Acesta este un benchmark de decizie clinică *multi-curs* (de la internare la externare) evaluat printr-un sistem cu agenți de tipul pacient, examinator și medic. Plafoanele de performanță, costul computațional mare și prezența persistentă a halucinațiilor justifică decizia noastră de a restrânge LLM-ul la sarcina de parsare a limbajului natural și de a lăsa predicția clinică în mâna unor modele supervizate, care pot fi validate pe seturi de test reținute.

3.5. Poziționarea soluției propuse

Sinteza literaturii existente evidențiază două extreme. Pe de o parte, avem modelele clasice de ML pentru triaj. Acestea sunt robuste și auditabile, dar limitate la o singură sarcină și incapabile să proceseze direct limbajul natural. Pe de altă parte, există sistemele multi-agent bazate pe LLM. Acestea sunt flexibile și conversaționale, dar costisitoare, greu de auditat și cu un plafon de acuratețe încă insuficient pentru predicția clinică. Sistemul propus în această lucrare reprezintă o variantă de mijloc: folosește un LLM *doar* pentru înțelegerea limbajului natural și un pipeline cascadat de modele XGBoost pentru toate predicțiile. Sistemul reușește astfel să combine transparența și fiabilitatea ML cu flexibilitatea interfeței în limbaj natural. Tabelul 3.1 rezumă această poziționare în raport cu lucrările cele mai apropiate.

Tabela 3.1: Poziționarea soluției propuse față de lucrările reprezentative.

Sistem	Paradigmă	Predictor	Set de date	Sarcini acoperite
MAP [7]	Multi-agent LLM	LLM	MIMIC-IV (51K)	Triaj, diagnostic, tratament
TriageAgent [19]	Multi-agent LLM	LLM	Benchmark ESI propriu	Acuitate ESI
MedScrubCrew [20]	Multi-agent LLM (CrewAI)	LLM + graf	Profile pacient/furnizor	Programare, triaj
Raita et al. [9]	ML tabular	ML	NHAMCS (135K)	Îngrijire critică, spitalizare
Xie et al. [5]	ML tabular	ML	MIMIC-IV-ED (400K)	Spitalizare, critic, reprezentare
Soluția propusă	Multi-agent hibrid (CrewAI)	ML (XGBoost)	MIMIC-IV-ED (425K)	Acuitate, dispoziție, diagnostic, departament, ICD

Spre deosebire de aceste lucrări, care prezic de regulă *un singur* rezultat tabular (spitalizare sau îngrijire critică) cu un model izolat, sistemul propus realizează o *predicție în cascadă* care simulează întreg fluxul unui pacient venit la departamentul de urgență: acuitatea ESI alimentează decizia de dispoziție (internare / externare), care alimentează la rândul ei predicția diagnosticului, a departamentului și a codului ICD exact. În plus, modelele noastre integrează antecedente medicale anterioare vizitei curente (*Past Medical History*, PMH) extrase din prezentări anterioare și serii longitudinale de semne vitale. În același timp, se păstrează predicția în zona ML verificabilă, fără a se baza pe un LLM care poate halucina.

Față de toate aceste lucrări, contribuțiile unice ale sistemului propus sunt: (1) separarea clară a rolurilor printr-un LLM pentru NLP, ML pentru predicție, care păstrează auditabilitatea deciziilor; (2) un pipeline predictiv cascadat care acoperă integral fluxul UPU, de la acuitate până la codul ICD exact; (3) integrarea antecedentelor medicale și a semnelor vitale longitudinale în modelele de predicție; și (4) o comparație explicită a predicției medicale înainte și după colectarea datelor de către agentul asistent, care pune clar în evidență valoarea clinică a acestor date suplimentare.

Capitolul 4. Analiză și fundamentare teoretică

Acest capitol stabilește fundamentul teoretic al sistemului propus, fără a intra în detalii de implementare. Sunt formulate mai întâi problemele de decizie clinică pe care sistemul le rezolvă. Apoi, este prezentată structura logică a arhitecturii multi-agent, principiul hibrid care separă înțelegerea limbajului natural de predicția clinică, modelul de predicție în cascadă și fundamentele algoritmilor de învățare automată folosiți. Descrierea concretă a modulelor, a claselor și a fișierelor se face în capitolul următor.

4.1. Formularea problemei de decizie clinică

Parcursul unui pacient prin Unitatea de Primiri Urgențe (UPU) nu este o singură decizie, ci un șir de decizii înlanțuite. Primele două decizii se aplică oricărui pacient: stabilirea gravității la sosire (codul ESI) și decizia de internare sau externare. Pentru pacienții a căror decizie este internarea, lanțul continuă cu determinarea unui diagnostic, direcționarea către un departament și atribuirea unui cod de diagnostic exact. Pentru pacienții externați se redactează în schimb un rezumat de externare. Sistemul propus modelează acest lanț în întregime. Acest sistem se diferențiază clar de majoritatea lucrărilor analizate în capitolul anterior, care prezic un singur rezultat tabular cu un model izolat [5]. Fiecare parte a lanțului este tratată ca o problemă de *învățare automată supervizată*. Se pleacă de la o descriere a prezentării pacientului, codificată într-un vector de caracteristici x , se estimează o variabilă-țintă y , pe baza unui model antrenat dintr-un set de prezentări istorice etichetate $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$.

Setul de date fundamental al sistemului nostru este MIMIC-IV-ED, o bază de date publică ce conține aproximativ 425.000 de prezentări reale în UPU. Aceste intrări conțin simptomele semnalate de pacienți, semnele vitale de la triaj, datele demografice, dispoziția finală și codurile de diagnostic codificate, asociate fiecărei prezentări [4]. Etichetele necesare pentru antrenarea celor cinci modele provin direct din aceste înregistrări, permițând formularea fiecărei sarcini ca o problemă de clasificare bine definită:

- **Acuitatea**, predicția nivelului de gravitate pe scala *Emergency Severity Index* (ESI), cu cinci niveluri ordonate, de la 1 (stare critică, intervenție imediată) la 5 (caz neurgent). Este o problemă de clasificare *ordinală*, nu nominală.
- **Dispoziția**, predicția binară a prezentării pacientului: internare sau externare.
- **Diagnosticul**, clasificarea într-una dintre categoriile majore de diagnostic, obținute prin gruparea codurilor ICD în clase suficient de populate pentru antrenare.
- **Departamentul**, direcționarea pacientului care va fi internat către serviciul medical sau chirurgical corespunzător.
- **Codul ICD exact**, determinarea codului de diagnostic exact în interiorul categoriei prezise, formulată ca o problemă de ordonare (*ranking*) a codurilor candidate.

Dintre aceste cinci sarcini, acuitatea ocupă o poziție unică prin structura ei ordinală. Scala ESI grupează pacienții după resursele anticipate și după riscul vital, iar cele cinci niveluri au o ordine importantă [1]. O predicție de nivel 2 pentru un pacient de nivel 1 este o eroare mai gravă decât o predicție de nivel 2 pentru un pacient de nivel 3. Deși ambele greșesc cu un singur nivel, predicția unei clase mai puțin grave pentru un caz mai grav poate duce la probleme serioase. Mai mult, cele două tipuri de eroare

nu sunt simetrice din punct de vedere clinic. *Sub-triajul*, adică subevaluarea gravității, duce la întârzierea îngrijirii unui pacient grav și se asociază cu risc de mortalitate. *Supra-triajul* consumă resurse limitate fără un beneficiu echivalent. Această asimetrie de cost este o constrângere de proiectare care va influența atât funcția obiectiv a modelului de acuitate, cât și alegerea pragului modelului de dispoziție. Aceste aspecte sunt tratate în secțiunea 4.6.

Sarcinile de diagnostic și de departament se confruntă cu o problemă diferită, cea a dezechilibrului accentuat al claselor. Există mii de coduri ICD diferite, iar multe din ele apar de prea puține ori pentru a permite antrenarea unui clasificator fiabil. Soluția adoptată este gruparea codurilor în categorii de diagnostic mai largi. Astfel, fiecare clasă poate să dispună de suficiente exemple. Predicția diagnosticului se aplică numai pacienților internați. Această decizie este bazată pe faptul că, în lumea reală, rolul UPU nu este neapărat de a oferi un diagnostic clar, ci de a se asigura că pacientul este tratat corespunzător, iar în cazul externării poate pleca acasă în siguranță. De asemenea, departamentul este prezis tot doar pentru pacienții internați. În schimb, pacienții externati vor primi un rezumat de externare. Codul ICD exact rămâne totuși recuperabil în interiorul categoriei prezise, printr-o etapă suplimentară de ordonare descrisă în secțiunea 4.6.6.

Cele cinci variabile-țintă nu sunt independente. Acuitatea influențează probabilitatea de internare, dispoziția delimitează populația pentru care diagnosticul are sens, iar diagnosticul influențează alegerea departamentului. Această dependență este evidențiată explicit prin modelul de predicție în cascadă prezentat în secțiunea 4.5.

4.2. Structura logică a sistemului multi-agent

Lanțul de decizii descris mai sus este împărțit între mai mulți agenți specializați. Fiecare agent este responsabil de o funcționalitate specifică și comunică prin intermediul unor interfețe de intrare și ieșire bine definite. Această împărțire urmează principiul separării responsabilităților: în loc de un singur model monolitic care să rezolve toate sarcinile, fiecare etapă a fluxului clinic este atribuită unui agent specific. Astfel, fiecare agent primește o intrare structurată, produce o ieșire structurată și o transmite mai departe. Sistemul cuprinde patru agenți operaționali.

Agentul de **parsare a limbajului natural** preia descrierea în limbaj natural a pacientului și o transformă într-o reprezentare structurată: simptome (chief complaints), scorul durerii, vârsta, sexul, modul de sosire, antecedente medicale și internări anterioare. Agentul de **triaj** primește această reprezentare și estimează acuitatea ESI și probabilitatea de internare. Agentul **doctor** produce diagnosticul inițial, departamentul și o decizie de dispoziție rafinată. Agentul **asistent** colectează interactiv de la pacient datele care nu sunt disponibile la sosire, precum semnele vitale. Fiecare câmp solicitat de asistent poate lipsi, iar în acest caz modelul tratează valoarea ca necunoscută, fără a forța o estimare. Datele colectate de către asistent pot fi apoi folosite de către doctor pentru a face predicțiile rafinate.

Coordonarea celor patru agenți se realizează printr-un proces *secvențial determinist*. Fluxul de informație urmează ordinea naturală a parcursului clinic. Această alegere de proiectare se diferențiază intenționat de arhitecturile multi-agent bazate pe dezbateri sau negociere între agenți, des întâlnite în sistemele construite în jurul LLM-urilor [16, 15]. Într-un pipeline secvențial, ordinea de execuție este fixă și fiecare ieșire poate fi urmărită până la intrarea care a produs-o. Aceasta este o proprietate dorită pentru un sistem de sprijin al deciziei clinice, unde contează mai mult trasabilitatea decât flexibilitatea conversațională.

O particularitate care trebuie menționată este că agentul doctor intervine de *trei* ori, ceea ce ridică numărul total de sarcini la șase. Doctorul formulează o primă evaluare imediat după triaj, pe baza datelor disponibile la sosire. După ce asistentul colectează semnele vitale și medicația, doctorul reia evaluarea de două ori: o dată pentru a rafina decizia de internare sau externare printr-un model calibrat dedicat, și încă o dată pentru a reevalua diagnosticul și departamentul conform noilor date. Această dublare a evaluării doctorului nu este o redundanță, ci principiul central de proiectare al lucrării. Se separă explicit predicția făcută *înainte* de colectarea datelor de cea făcută *după*, pentru a permite sistemului să măsoare direct valoarea clinica adusă de informațiile suplimentare adunate de asistent. Această comparație lipsește din lucrările analizate în capitolul anterior. Reevaluarea finală nu se mai bazează pe verdictul de triaj, ci pe decizia de dispoziție rafinată, astfel încât un pacient pe care triajul l-ar fi externat, dar pe care modelul de dispoziție îl reclasifică drept caz de internare, declanșează din nou analiza completă de diagnostic și departament.

Structura logică a întregului flux, cu cei patru agenți și cele șase sarcini, este prezentată în Figura 4.1.

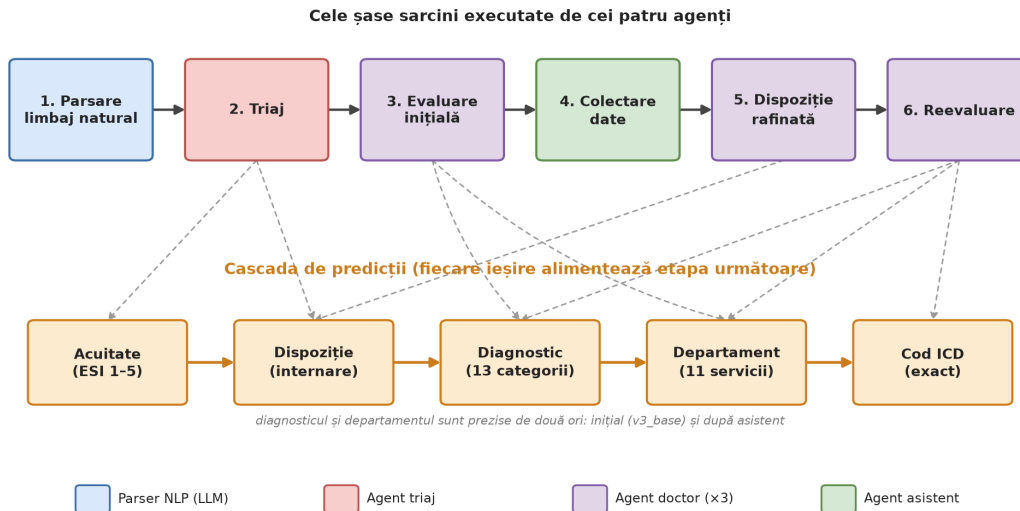


Figura 4.1: Structura logică a sistemului multi-agent: patru agenți și șase sarcini executate secvențial, cu evaluarea doctorului realizată înainte și după colectarea datelor de către asistent.

4.3. Principiul hibrid: limbaj natural prin LLM, predicție prin învățare automată

Cea mai importantă decizie de proiectare a sistemului este rolul atribuit LLM-ului. Pacientul își descrie starea în limbaj natural, iar simptomele și antecedentele sunt exprimate în text liber, nu în câmpuri tabulare. Această barieră lingvistică trebuie depășită de orice sistem care pornește de la descrierea pacientului. Întrebarea principală nu este *dacă* se folosește un LLM, ci *până unde* i se permite să intervină în lanțul de decizii.

Sistemul propus impune o graniță strictă. LLM-ul este folosit *exclusiv* pentru parsarea limbajului natural, adică pentru a converti descrierea liberă a pacientului într-o reprezentare structurată. Toate cele cinci predicții clinice sunt produse de modele

supervizate antrenate pe date reale. Această separare se bazează pe trei proprietăți pe care un model supervizat le oferă, iar un LLM generativ nu le garantează.

Prima este *reproductibilitatea*. Un model supervizat este o funcție deterministă a intrării: aceeași prezentare produce aceeași predicție, cu aceleași probabilități, la fiecare execuție. Un LLM generativ poate produce răspunsuri diferite pentru aceeași intrare, iar comportamentul lui se poate schimba între versiuni ale modelului, ceea ce face dificilă validarea pe seturi de test reținute. A doua este *auditabilitatea*. La modelele bazate pe arbori putem înțelege ușor de ce s-a luat o decizie, deoarece vedem ce informații au contat. De exemplu, un diagnostic poate fi explicat prin simptomele și semnele vitale. Raționamentul intern al unui LLM nu este accesibil în același mod. A treia este *fiabilitatea în fața halucinațiilor*. Lucrările de evaluare analizate în capitoul anterior arată că sistemele agentice bazate pe LLM ating încă plafoane de acuratețe modeste pe sarcini clinice. De asemenea, suferă de halucinații chiar și cu filtre interne și au un cost computațional mult mai mare [6, 24]. Un model care prezice o categorie dintr-un set fix de clase nu poate inventa un diagnostic inexistent, spre deosebire de un model generativ.

Această separare pune în valoare punctele forte ale fiecărei componentă fără a moșteni slăbiciunile celeilalte. LLM-ul aduce singura capacitate pe care modelele tabulare nu o au, înțelegerea limbajului natural deschis. În schimb, este ținut departe de decizia clinică, unde transparența și stabilitatea sunt esențiale. Modelele supervizate produc predicții verificabile, dar nu pot procesa direct text liber, aceasta sarcină fiind preluată de LLM. Alegerea este susținută indirect și de literatură, unde cel mai performant sistem din studiul comparativ al lui Lansiaux et al. [14] combină un model lingvistic pentru înțelegerea textului cu un clasificator de tip *gradient boosting* pentru decizia propriu-zisă. Acesta imită exact tiparul adoptat aici. Spre deosebire de sistemele în care LLM-ul este *predictorul* [7, 19], sistemul propus îl restrânge la rolul de interpret al textului și lasă predicția în zona verificabilă a învățării automate.

4.4. Extragerea structurată a informației din limbaj natural

Delimitarea menționată în secțiunea precedentă cere ca ieșirea LLM-ului să fie o reprezentare structurată, nu o decizie clinică. Prin urmare, sarcina agentului de parsare este una de *extragere structurată a informației*: transformarea unei descrieri din limbaj natural deschis într-un set fix de câmpuri structurate. Intrarea este nestructurată, poate fi incompletă și folosește vocabular nespecializat, iar ieșirea trebuie să fie o structură pe care modelele de predicție o pot folosi direct. Acesta este tipul de problemă în care un LLM aduce un avantaj real față de metodele bazate pe reguli, deoarece tolerează parafraza, sinonimele și expresiile uzuale.

Rezultatul extragerii este o *schemă* predefinită, formată din câmpuri precum simptome (chief complaints), scorul durerii, vârsta, sexul și modul de sosire, la care se adaugă semnele vitale atunci când pacientul este adus de un echipaj medical, precum ambulanță sau elicopter. Schema funcționează ca o legătură între partea de limbaj și cea de învățare automată: modelul nu poate produce orice rezultat, ci doar completează spațiile dintr-o structură deja definită. Tot ce nu se încadrează în schemă este ignorat. Câmpurile sunt verificate ca tip și normalizate la același vocabular pe care îl folosesc modelele, prin expandarea abrevierilor și aceeași normalizare a textului descrisă în secțiunea 4.7. În felul acesta, influența LLM-ului asupra restului sistemului este limitată la conținutul unei structuri bine definite și ușor de verificat, ceea ce păstrează posibilitatea de a urmări și înțelege deciziile.

De multe ori, pacientul omite câmpuri necesare, precum vârsta sau modul de

sosire, atunci când face o descriere inițială. Atunci când avem informații incomplete, există două strategii: estimarea valorii lipsă sau solicitarea ei. Pentru câmpurile ușor de obținut prin întrebare, sistemul adoptă a doua cale. Parsarea este proiectată ca o *bucă interactivă* în loc de o extragere într-un singur pas. Atunci când un câmp obligatoriu nu poate fi completat din text, agentul formulează o întrebare specifică către pacient și completează câmpul lipsă din reprezentarea structurată. Această decizie este justificată deoarece este de preferat să reducem absența datelor, printr-o simplă întrebare, decât imputarea ulterioară care introduce incertitudine în vectorul de caracteristici și nu adaugă o informație reală.

Există o limită implicită a acestei abordări. Pentru că pacientul descrie simptomele în limbaj comun, simptomul principal extras este o parafrază a terminologiei clinice scurte pe care modelele au învățat-o din date. Diferența dintre „durere de stomac” și forma clinică abreviată corespunzătoare nu poate fi eliminată complet la parsare, ci doar atenuată de stratul de normalizare a textului, prin expandarea abrevierilor și prin reprezentarea pe grupuri de cuvinte discutată în secțiunea 4.6.2. Această limitare provine din faptul că se folosește limbajul natural ca intrare și este o caracteristică a problemei în sine, nu a unei implementări anume.

4.5. Modelul de predicție în cascadă

Cele cinci sarcini de predicție nu sunt rezolvate independent. Ieșirea fiecărui model devine intrare pentru următorul. Se formează o cascadă care urmează ordinea naturală a fluxului clinic. Gravitatea evaluată la triaj influențează decizia de internare, care delimitează populația pentru care diagnosticul are sens, iar diagnosticul orientează alegerea departamentului. Această dependență între obiective, observată deja în secțiunea 4.1, devine un principiu de proiectare.

Formal, fie x vectorul de caracteristici extras din prezentarea pacientului. Primul model estimează distribuția de probabilitate a acuității, $P(a \mid x)$. Al doilea model nu primește doar x , ci vectorul extins cu ieșirea primului model, estimând $P(d \mid x, P(a \mid x))$, unde d este dispoziția. Lanțul continuă la fel pentru diagnostic și departament, fiecare model fiind influențat de predicțiile celor dinaintea lui. Predicția acuității acționează ca o informație suplimentară, care comprimă într-o formă învățată o parte din datele din x , relevante pentru sarcinile următoare.

Modul în care rezultatul unui model este transmis mai departe permite două tipuri de cascadă. În *cascada dură* (*hard cascade*), modelul inițial trimite mai departe doar clasa pe care o consideră cea mai probabilă. Cu alte cuvinte, se păstrează doar decizia finală. Această abordare este simplă, dar ignoră cât de sigur este modelul în acea decizie. De exemplu, un pacient pentru care modelul oscilează între nivelurile ESI 2 și 3 va fi tratat la fel ca unul pentru care ESI 2 este aproape sigur. În *cascada moale* (*soft cascade*), în schimb, modelul nu trimite doar o singură etichetă, ci întreaga distribuție de probabilități. Astfel, modelul următor primește un vector complet de tip softmax, care include atât acuitatea, cât și probabilitatea de internare. Acest lucru îi permite să distingă între cazurile clare și cele incerte și să ia decizii mai nuanțate pentru situațiile care se află la limita dintre 2 clase.

Sistemul propus folosește ambele variante, în funcție de etapă. Predicțiile de diagnostic și departament moștenesc eticheta prezisă la nivelul anterior printr-o cascadă dură. Rafinarea deciziei de dispoziție folosește o cascadă moale, primind vectorul complet de probabilități ale acuității împreună cu probabilitatea de internare estimată la triaj. Motivul acestei alegeri este că decizia de internare sau externare este cea mai sensi-

bilă la incertitudinea de triaj, fiind și decizia cu cel mai direct impact asupra pacientului. În concluzie, este și cea care beneficiază cel mai mult de păstrarea distribuției complete.

Cascada conține și o relație de *conditionare*, nu doar de transmitere de caracteristici. Decizia de dispoziție delimitează pacienții asupra cărora se aplică predicția de diagnostic și departament. Acestea au sens doar pentru pacienții internați. Reevaluarea finală a doctorului se declanșează în funcție de decizia de dispoziție rafinată, nu de verdictul inițial de triaj. În acest mod, o reclasificare a dispoziției propagă corect efectul către restul lanțului. Avantajul teoretic al acestei structuri este că fiecare model primește un rezumat al deciziilor anterioare, pe lângă datele brute. Se evită astfel redundanța și nevoia să se reconstruiască de la zero informația deja extrasă de modelele din etapele anterioare.

4.6. Fundamentele algoritmilor de învățare automată

Toate cele cinci predicții se bazează pe aceeași familie de algoritmi. Mai exact, sunt folosiți arbori de decizie antrenati prin *gradient boosting*, care primesc ca intrare o reprezentare numerică a textului prin TF-IDF, împreună cu variabilele structurate ale cazului. Această secțiune prezintă fundamentele teoretice ale acestor metode și ale ajustărilor impuse de specificul clinic al problemei, precum dezechilibrul claselor, structura ordinală a acuității, necesitatea unor probabilități calibrate și recuperarea codului ICD exact.

4.6.1. Arbori de decizie cu gradient boosting

Un model bazat pe *gradient boosting* este un ansamblu aditiv de arbori de regresie. Predicția se obține ca sumă a contribuțiilor a K arbori.

$$\hat{y} = F(x) = \sum_{k=1}^K f_k(x),$$

Fiecare f_k este un arbore antrenat secvențial. Ideea de bază a metodei, formulată de Friedman [25], este construirea ansamblului pas cu pas. La fiecare iterație se adaugă un arbore care aproximează gradientul negativ al funcției de pierdere evaluat pe predicția curentă, ceea ce echivalează cu o coborâre pe gradient în spațiul funcțiilor. Arborii ulteriori corectează astfel erorile arborilor anteriori.

Implementarea utilizată este XGBoost [26], un algoritm bazat pe arbori de decizie care urmărește să obțină predicții cât mai precise. Acesta evită în același timp ca modelul să învețe excesiv detaliile din datele de antrenare (supraînvățarea). Pentru acest lucru, algoritmul optimizează o funcție obiectiv care combină eroarea de predicție cu un termen de regularizare:

$$\mathcal{L} = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k),$$

Primul termen măsoară cât de bine se potrivesc predicțiile cu valorile reale, iar al doilea penalizează arborii prea complecși. Penalizarea depinde de numărul de frunze și de valorile asociate acestora, încurajând construirea unor modele mai simple și mai robuste. Pentru a stabili cele mai bune împărțiri în arbori, XGBoost folosește o aproximare de ordinul al doilea a funcției de pierdere, ceea ce îi permite să învețe eficient și să obțină performanțe ridicate.

Alegerea acestei familii de modele în locul rețelelor neuronale are mai multe motive

care sunt legate de natura problemei. Caracteristicile de intrare sunt de tipuri diferite și includ atât valori structurate (vârstă, semne vitale, scor de durere), cât și o reprezentare a textului sub forma unui vector text mare și rar. Arborii tratează nativ acest amestec. Aceștia surprind interacțiuni neliniare între caracteristici fără a fi necesară ingineria lor explicită și nu este necesară normalizarea valorilor. La fel de important, contribuția fiecărei caracteristici la predicție poate fi măsurată, ceea ce ajută la îndeplinirea cerinței de auditabilitate din secțiunea 4.3. Pentru sarcinile cu mai multe clase, modelul produce o distribuție de probabilitate peste clase printr-o funcție softmax. Această distribuție alimentează direct cascada moale descrisă în secțiunea 4.5.

4.6.2. Reprezentarea textului prin TF-IDF

Simptomele principale provin dintr-un text liber scurt, iar un model bazat pe arbori are nevoie de o reprezentare numerică a acestuia. Metoda folosită este TF-IDF (*term frequency – inverse document frequency*). Această metodă atribuie fiecărui termen t dintr-o descriere d o pondere care depinde de două lucruri: cât de des apare acel termen în document și cât de comun este el în întregul set de texte. Termenii care apar des într-un document, dar sunt rari în restul corpusului, vor avea o importanță mai mare, iar cuvintele foarte frecvente în multe documente vor avea o importanță mai mică.

$$w_{t,d} = \text{tf}_{t,d} \cdot \log \frac{N}{n_t},$$

unde N este numărul total de descrieri și n_t numărul celor care conțin termenul t . Factorul de *inverse document frequency*, introdus de Spärck Jones [27], scade ponderea cuvintelor frecvente și fără putere de discriminare și o crește pe cea a termenilor rari, dar specifici, fiind cei mai utili pentru distincția între cazuri.

Vectorizarea nu se limitează la cuvinte izolate. Se folosesc grupuri de unu, două și trei cuvinte consecutive (unigrame, bigrame și trigrame), cu rolul de a surprinde expresii clinice în care sensul rezultă din combinație, nu din cuvintele luate separat, precum „chest pain” sau „shortness of breath”. Rezultă un vector rar de dimensiune mare, care este combinat cu caracteristicile structurate înainte de a fi transmis modelului. În plus, fiecărui cuvânt i se asociază un *prior de severitate*, adică nivelul mediu de acuitate al cazurilor în care acel cuvânt apare în datele de antrenare. Acest lucru oferă modelului un semnal suplimentar despre gravitatea sugerată de vocabularul folosit.

4.6.3. Dezechilibrul claselor și ponderarea atenuată

Distribuția claselor este foarte dezechilibrată în toate sarcinile. La triaj, nivelul ESI 3 domină, iar cazurile critice de nivel 1 sunt rare. La diagnostic, există câteva categorii frecvente care acoperă majoritatea prezentărilor. Un model antrenat fără corecție pe astfel de date tinde să favorizeze clasele majoritare și să rateze clasele rare. Pentru sarcinile clinice această tendință este periculoasă, deoarece tocmai clasele rare, cazul critic sau diagnosticul neobișnuit, sunt cele în care o eroare poate să coste cel mai mult.

Corecția se face prin ponderarea exemplelor la antrenare în funcție de frecvența clasei lor. Ponderarea cu inversul direct al frecvenței supracorectează, deoarece acordă claselor rare o influență atât de mare încât distruge acuratețea pe clasele frecvente. Soluția adoptată este o ponderare atenuată, proporțională cu rădăcina pătrată a inver-

sului frecvenței clasei,

$$w_c \propto \sqrt{\frac{N}{N_c}},$$

unde N_c este numărul de exemple din clasa c . Rădăcina pătrată reprezintă un compromis între distribuția naturală și cea perfect echilibrată. Această metodă ridică recall-ul pe clasele rare fără a sacrifica excesiv performanța pe cele dominante.

4.6.4. Predicția ordinală a acuității

Acuitatea ESI este o variabilă ordinală, iar acest lucru schimbă atât evaluarea, cât și antrenarea modelului. Un clasificator obișnuit tratează clasele ca nominale și consideră orice eroare la fel de gravă, ceea ce ignoră faptul că o predicție de nivel 4 pentru un pacient de nivel 1 este mult mai gravă decât una de nivel 2. Pentru o evaluare corectă se folosește coeficientul kappa ponderat quadratic al lui Cohen [28], care penalizează dezacordurile proporțional cu pătratul distanței dintre nivelul prezis și cel real, prin ponderile

$$w_{ij} = \frac{(i - j)^2}{(k - 1)^2},$$

unde i și j sunt nivelurile real și prezis, iar k este numărul de niveluri. Erorile mari sunt astfel penalizate mult mai puternic decât cele de un singur nivel.

Structura ordinală influențează și antrenarea, prin două modificări aplicate doar modelului de acuitate. Prima modificare crește importanța (ponderile) claselor extreme peste ponderarea standard a claselor. Astfel, actualizările de gradient sunt împinse mai agresiv către nivelul critic 1 și către nivelul neurgent 5, care sunt și cele mai rare. A doua modificare înlocuiește criteriul de oprire timpurie a antrenării. În loc să selecteze iterația cu cea mai mică pierdere logaritmică, alege iterația cu cel mai mare kappa ponderat quadratic, adică cel mai bun acord pe ordinea claselor. Ambele modificări sunt motivate de asimetria de cost discutată în secțiunea 4.1: este mai important să reducem cazurile de sub-triaj decât să maximizăm ușor acuratețea globală.

4.6.5. Calibrarea probabilităților

Decizia de internare sau externare se face comparând probabilitatea de internare estimată cu un prag. Pentru ca pragul să aibă o semnificație clinică clară, această probabilitate trebuie să fie *calibrată*. Dintre cazurile pentru care modelul atribuie o probabilitate de internare de 0,8, aproximativ 80% ar trebui să fie într-adevăr internate. Modelele bazate pe arbori cu *boosting* nu produc în mod natural probabilități calibrate, ci tind să le distorsioneze către o formă sigmoidă, împingând valorile dinspre extremele 0 și 1 [29].

Corecția se aplică printr-o etapă separată de calibrare, după antrenarea modelului. Se folosește regresia izotonică, adică o transformare monotonă neparametrică care mapează scorurile brute ale modelului în probabilități calibrate, estimată pe un set de date reținut special pentru acest scop. Calitatea calibrării se măsoară prin eroarea de calibrare așteptată (*Expected Calibration Error*, ECE), care compară încrederea medie a modelului cu frecvența reală observată, pe intervale de probabilitate. O probabilitate de internare bine calibrată permite alegerea pragului de operare în funcție de compromisul de cost dintre sub-triaj și supra-triaj, în loc să fie fixat implicit la 0,5.

4.6.6. Rezolvarea codului ICD exact prin căutare

Predicția de diagnostic se oprește la nivelul categoriei. Codurile ICD individuale sunt prea numeroase și prea rare pentru a antrena un clasificator separat pentru fiecare. Recuperarea codului exact în interiorul categoriei care a fost prezisă anterior este stabilită ca fiind o problemă de *ordonare (ranking)*, nu de clasificare. Codurile candidate din categoria respectivă sunt ordonate după un scor care combină mai multe surse de evidență,

$$\text{scor}(c) = w_{\text{text}} \cdot s_{\text{text}}(c) + w_{\text{vit}} \cdot s_{\text{vit}}(c) + w_{\text{pmh}} \cdot s_{\text{pmh}}(c) + w_{\text{prev}} \cdot s_{\text{prev}}(c),$$

unde s_{text} este similaritatea cosinus dintre vectorul TF-IDF al descrierii și un prototip de cod construit din descrierile cazurilor care au fost deja asociate cu acel cod. s_{vit} măsoară cât de asemănător este profilul fiziologic al pacientului cu profilul fiziologic caracteristic unui anumit cod, prin compararea cu cel mai apropiat prototip. Acest profil nu include doar valorile semnelor vitale măsurate la un moment dat, ci și alte informații relevante, luate împreună: indicatorii de anormalitate, evoluția în timp a fiecărui semn vital și ritmul cardiac. Astfel, comparația surprinde atât starea curentă a pacientului, cât și dinamica acesteia. s_{pmh} compară antecedentele pacientului cu prototipuri construite din pacienții cu istoric și este *condiționat (gated)*. Acesta intervine doar pentru pacienții care au un istoric medical, iar pentru ceilalți ponderea lui se redistribuie pe celelalte trei surse. s_{prev} reflectă cât de frecvent apare codul în categoria respectivă. Ponderile sunt stabilite dinainte, prin căutare pe un set de validare.

Această abordare este o căutare după cel mai apropiat prototip și este preferată în locul unui clasificator tocmai pentru că funcționează și pentru codurile cu foarte puține exemple. În astfel de situații, nu s-ar putea antrena în mod fiabil un model separat pentru fiecare cod. Etapa are un rol *consultativ*: ea sugerează codul ICD cel mai probabil, dar nu modifică predicția de categorie sau de departament produsă de modelele din cascadă.

4.7. Modelul caracteristicilor și absența scurgerilor de informație

Vectorul de caracteristici folosit de modele include cinci categorii de informație: datele structurate de la triaj (vârstă, sex, mod de sosire, scor de durere), reprezentarea text a simptomelor principale, semnele vitale, antecedentele medicale anterioare (PMH) și medicație curentă. Nu toate sunt însă disponibile în același moment. Din acest motiv, evaluarea doctorului este împărțită în două etape, în funcție de datele care devin disponibile progresiv.

Semnele vitale afișează cel mai bine această dependență de moment. Pentru un pacient adus cu ambulanța sau elicopterul, ele sunt măsurate de echipajul medical înainte de sosire și sunt disponibile încă de la triaj. Pentru un pacient care se prezintă pe cont propriu, ele nu există în acel moment și sunt colectate ulterior de asistent. Modelele nu pot presupune valori pe care nu le au, așa că o valoare lipsă este marcată explicit ca atare printr-un indicator de absență, în loc să fie înlocuită cu o valoare implicită care ar induce modelul în eroare. Aceasta este și diferența de informație dintre evaluarea inițială și reevaluarea doctorului. Prima evaluare lucrează fără semne vitale pentru pacienții care vin pe cont propriu, iar a doua evaluare le are pe toate.

Antecedentele medicale ridică o problemă mai delicată, cea a scurgerilor de informație (*data leakage*). PMH se construiește din prezentările anterioare ale aceluiași pacient, dar trebuie incluse numai informațiile care ar fi fost cunoscute la momentul deciziei. O caracteristică care provine dintr-o prezentare viitoare, sau chiar din concluzia prezentării

curente, ar produce la antrenare o acuratețe mărită artificial și ar fi imposibil de reprodus în funcționarea reală. Pentru a evita acest lucru, antecedentele sunt filtrate strict temporal. Sunt folosite doar prezentările a căror dată de internare este înainte de momentul sosirii în cazul curent. Modelul este astfel *lipsit de scurgeri prin construcție*, iar performanța măsurată pe setul de test arată onest ceea ce sistemul poate face cu adevărat în practică.

Atât antecedentele, cât și medicația au o particularitate legată de sursa lor. Pentru un pacient cunoscut, ambele pot fi preluate din înregistrările prezentărilor sale anterioare. Acestea pot fi regăsite pe baza unui cod unic de pacient. Pentru un pacient aflat la prima prezentare sau neidentificat, aceste înregistrări nu există. Informația în acest caz este obținută prin auto-raportare în cadrul conversației cu asistentul. Indiferent de metodă, reprezentarea finală trebuie să fie aceeași. În absența unui istoric, câmpurile provenite din prezentări anterioare sunt completate cu zero, acesta fiind exact tiparul pe care modelul l-a întâlnit la antrenare pentru pacienții fără istoric. Filtrarea pe bază de timp discutată mai sus se aplică identic ambelor surse, astfel încât nici medicația, nici antecedentele preluate din înregistrări nu pot introduce informație ulterioară momentului deciziei.

O ultimă cerință de corectitudine esențială este similaritatea între antrenare și inferență. Pașii de transformare a intrării, normalizarea textului, expandarea abrevierilor, vectorizarea TF-IDF și construirea vectorului de caracteristici, trebuie să fie identici în cele două situații. Orice diferență între modul în care o caracteristică este calculată la antrenare și la inferență înseamnă că modelul primește date cu o distribuție diferită de cea pe care a învățat-o. Acest lucru îi degradează predicțiile fără un semnal vizibil de eroare. Din acest motiv, aceeași logică de preprocesare și aceiași parametri ai vectorizatorului sunt reutilizați la inferență. Este garantat astfel că modelul vede în producție exact tipul de intrare pentru care a fost antrenat.

Capitolul 5. Proiectare de detaliu și implementare

Capitolul de față descrie cum a fost construit efectiv sistemul a cărui fundamentare teoretică a fost prezentată în Capitolul 4. Dacă acolo au fost formulate principiile, adică structura logică a celor patru agenți, principiul hibrid care separă înțelegerea limbajului de predicția clinică, modelul de predicție în cascadă și algoritmi de învățare automată, aici se arată unde se găsesc aceste principii în cod: pachetul Python, fișierele de configurare ale agenților, instrumentele (*tools*) care încapsulează modelele, pipeline-ul de antrenare, modelul de date și interfața web. Descrierea urmează ordinea fluxului clinic și conține câte o secțiune pentru fiecare agent, astfel încât un cititor care vrea să întrețină aplicația să poată regăsi rapid componenta responsabilă de o anumită funcționalitate.

5.1. Arhitectura generală a aplicației

Sistemul este o aplicație Python realizată pe baza framework-ului de orchestrare multi-agent CrewAI [30] (versiunea 1.9.3). Predicțiile clinice sunt produse de modele XGBoost [26], care sunt antrenate și salvate cu ajutorul bibliotecilor scikit-learn [31] (vectorizare TF-IDF, calibrare, metrice) și *joblib* (serializarea artefactelor), iar prelucrarea datelor se face cu *pandas*. Potrivirea aproximativă a textului folosește biblioteca *thefuzz*. Interfața web opțională este construită cu FastAPI pe partea de server și cu React, Vite și Tailwind pe partea de client. Această alegere de tehnologii respectă constrângerea evidențiată în secțiunea 2.3, aceea că antrenarea și inferența rulează pe hardware obișnuit, fără infrastructură specializată.

Codul sursă este grupat în pachetul `src/proiect_licenta`. Punctul de intrare este `main.py`, care colectează descrierea pacientului și pornește echipajul de agenți. În varianta de antrenare, execuția este redirecționată către pipeline-urile din subpachetul `training`. Definiția echipajului se află în `crew.py`. Restul modulelor de la nivelul pachetului sunt componente împărțite de mai mulți agenți: `paths.py` centralizează toate căile către date și artefacte, `preprocessing.py` normalizează textul simptomelor, `llm_config.py` comută între backend-urile LLM (Gemini Flash și MedGemma), `interaction.py` gestionează dialogul interactiv, iar `pmh_features.py`, `pmh_vocab.py`, `icd_resolution.py` și `vital_trajectory.py` construiesc caracteristicile derivate (antecedente, cod ICD exact, semne vitale longitudinale). Instrumentele la care agenții au acces se află în subpachetul `tools`, definițiile lor de configurare în `config`, iar pipeline-urile de antrenare în `training`. Organizarea pe disc a întregului pachet și a directoarelor de artefacte și date este prezentată în Figura 5.1.

Două module merită evidențiate separat, pentru că pe ele se bazează corectitudinea întregului sistem. Primul este `paths.py`, care exportă fiecare cale către un fișier de date sau un director de artefacte ca o constantă (`TRIAGE_V3_DIR`, `DOCTOR_V3_DIR`, `TRIAGE_CSV` și celelalte). Toate instrumentele, pipeline-urile de antrenare și benchmark-urile importă aceste constante, în loc să construiască manual căile. Astfel, mutarea unui director se face într-un singur loc. Al doilea este `preprocessing.py`, care deține funcția `normalize_complaint_text` și dicționarul `ABBREVIATIONS` de expandare a abrevierilor medicale. Atât antrenarea, cât și inferența apelează exact aceeași funcție de normalizare pentru a garanta că vectorizatorul TF-IDF primește în producție textul în forma în care

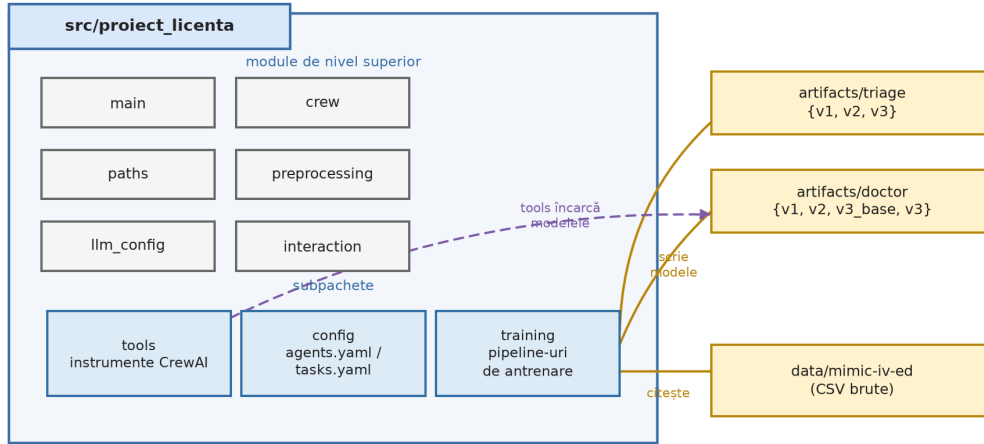


Figura 5.1: Structura pe pachete a aplicației și legăturile către artefactele antrenate și datele MIMIC-IV-ED.

a fost antrenat. Această identitate între antrenare și inferență a fost discutată teoretic în secțiunea 4.7 și este asigurată în practică prin reutilizarea aceluiași modul de preprocesare, nu prin replicarea logicii.

Artefactele antrenate sunt organizate și păstrate pe versiuni prin intermediul directoroarelor, nu prin nume de fișiere. Fiecare tier de model are propriul director sub `artifacts/triage/` sau `artifacts/doctor/`, iar fișierele dinăuntru păstrează aceleași nume (`acuity_model.joblib`, `disposition_model.joblib`, `diagnosis_model.joblib`, `department_model.joblib`, `tfidf_vectorizer.joblib` și metadatele asociate). Modelele folosite în runtime sunt cele din tier-ul v3, adică ultima versiune. Versiunile v1 și v2 rămân pe disc ca puncte de referință pentru comparațiile din lucrare. Repartizarea celor patru agenți pe instrumente și sarcini este rezumată în Tabelul 5.1.

Tabela 5.1: Cei patru agenți operaționali, instrumentele și sarcinile lor.

Agent	Instrumente	Sarcini (<i>tasks</i>)
nlp_parser	AskPatientTool, ConfirmIntakeTool	parse_symptoms_task
triage_agent	TriagePredictionTool, PatientHistoryLookupTool	triage_assessment_task
doctor_agent	DoctorPredictionToolV3Base, DoctorDispositionTool, DoctorPredictionToolV3, PatientHistoryLookupTool	doctor_assessment_task, doctor_disposition_task, doctor_reassessment_task
nurse_agent	NurseDataCollectionTool	nurse_data_collection_task

5.2. Orchestrarea agenților și fluxul de execuție

Echipajul de agenți este definit în `crew.py`, prin clasa `ProiectLicenta` decorată cu `@CrewBase`. Această clasă declară cei patru agenți, fiecare printr-o metodă marcată cu `@agent`, precum și cele șase sarcini, fiecare printr-o metodă marcată cu `@task`, și metoda `@crew` care reunește toți agenții și sarcinile și assemblează echipajul. Un agent este construit din configurația lui din fișierul YAML, din lista de instrumente pe care le poate folosi și din modelul de limbaj returnat de `get_llm()`. O sarcină este construită doar din configurația ei, pentru că legăturile dintre sarcini sunt declarate tot în YAML. Structura este ilustrată în Listingul 5.1.

```
@CrewBase
class ProiectLicenta():
    """Multi-Agent Medical Decision Support System."""
    agents: List[BaseAgent]
    tasks: List[Task]

    @agent
    def triage_agent(self) -> Agent:
        return Agent(
            config=self.agents_config['triage_agent'],
            tools=[TriagePredictionTool(), PatientHistoryLookupTool()],
            llm=get_llm(),
            verbose=True,
        )

    @task
    def triage_assessment_task(self) -> Task:
        return Task(config=self.tasks_config['triage_assessment_task'])

    @crew
    def crew(self) -> Crew:
        return Crew(
            agents=self.agents,
            tasks=self.tasks,
            process=Process.sequential,
            verbose=True,
        )
```

Listing 5.1: Scheletul clasei `ProiectLicenta` din `crew.py`.

Prompturile agenților și definițiile sarcinilor sunt ținute separat de cod, în `config/agents.yaml` și `config/tasks.yaml`. Pentru fiecare agent, fișierul `agents.yaml` stabilește rolul (*role*), obiectivul (*goal*) și povestea de fundal (*backstory*) care îi dau comportamentul. Pentru fiecare sarcină, `tasks.yaml` descrie ce trebuie făcut (*description*), forma rezultatului așteptat (*expected_output*), agentul responsabil și contextul, adică lista sarcinilor anterioare ale căror ieșiri îi sunt furnizate ca intrare. Separarea prompturilor de cod permite modificarea comportamentului unui agent fără a modifica logica programului.

Coordonarea celor patru agenți se face prin `Process.sequential`. Acest lucru înseamnă că sarcinile rulează una după alta, în ordinea declarată. Transmiterea informației între sarcini folosește două mecanisme. Primul este câmpul de context din `tasks.yaml`, prin care o sarcină primește textul produs de sarcinile dinaintea ei. Al doilea este o convenție de format: fiecare sarcină de predicție își încheie raportul cu un bloc delimitat de marcajul `---STRUCTURED_DATA---`. În acesta, valorile structurate (nivelul ESI, decizia de dispoziție, vârsta, sexul) sunt scrise într-o formă ușor de extras de sarcina următoare.

Astfel, agentul nu trebuie să reconstruiască aceste valori din textul liber al raportului anterior.

Cele șase sarcini se execută în următoarea ordine: parsarea simptomelor (`parse_symptoms_task`), evaluarea de triaj (`triage_assessment_task`), evaluarea inițială a doctorului (`doctor_assessment_task`), colectarea datelor de către asistent (`nurse_data_collection_task`), rafinarea dispoziției (`doctor_disposition_task`) și reevaluarea finală (`doctor_reassessment_task`). Agentul doctor apare în trei dintre acestea, ceea ce explică de ce patru agenți produc șase sarcini. O particularitate importantă a fluxului este modul în care se decide când are loc reevaluarea finală. Aceasta nu se mai declanșează pe baza verdictului de triaj, ci pe baza deciziei de dispoziție rafinată produsă de `doctor_disposition_task`. Dacă un pacient este evaluat ca fiind externat la triaj și apoi modelul de dispoziție îl reclasifică drept caz de internare, se declanșează din nou analiza completă de diagnostic și departament. Această înlanțuire este implementarea concretă a cascadei descrise în secțiunea 4.5 și este reprezentată în Figura 5.2.

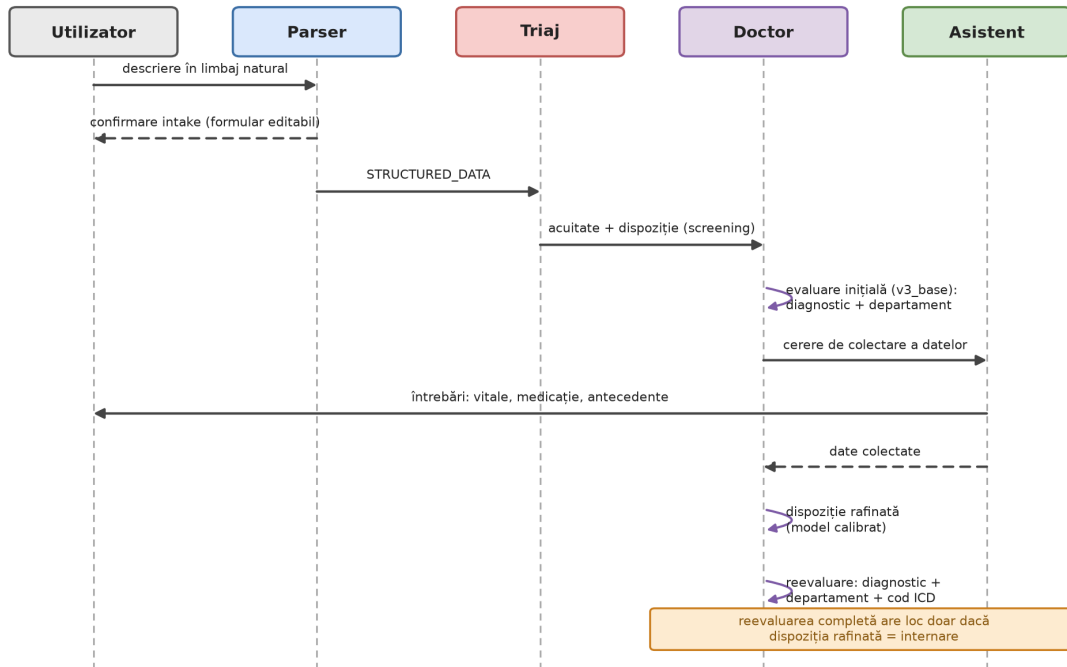


Figura 5.2: Fluxul secvențial de execuție al celor șase sarcini și punctul în care reevaluarea finală este condiționată de dispoziția rafinată.

Punctul de intrare `main.py` expune funcția `run`, care afișează un mesaj de întâmpinare, citește descrierea pacientului de la tastatură și apelează `ProiectLicenta().crew().kickoff(input_message': ...})`, după care afișează rezultatul final. Tot `main.py` expune și funcțiile de antrenare (`train_triage_v3`, `train_doctor_v3`, `train_nurse_v3`, `train_doctor_disposition`, `train_icd_resolver` și celelalte). Fiecare din acestea trimite execuția mai departe către pipeline-ul corespunzător din subpachetul `training`. Aceste funcții sunt înregistrate ca subcomenzi în `pyproject.toml`, astfel încât antrenarea unui model se reduce la o singură comandă.

Rolul modelului de limbaj în întregul echipaj este stabilit într-un singur loc, modulul `llm_config.py`. Funcția `get_llm()` este apelată la construirea fiecărui agent și retur-

nează backend-ul de limbaj în funcție de variabila de mediu `LLM_BACKEND`. Pentru valoarea implicită `flash`, funcția întoarce `None` și lasă agentul pe configurația standard CrewAI (Gemini 2.5 Flash), deci calea implicită rămâne neschimbată. Pentru valoarea `medgemma`, funcția întoarce un obiect LLM care indică un model MedGemma auto-găzduit în spațiile unui endpoint compatibil OpenAI. Funcția auxiliară `get_parse_llm()` returnează întotdeauna un model explicit și este folosită pentru a compara cei doi parseri în același regim. Comutarea aceasta pune în practică obiectivul O6 din secțiunea 2.2: backend-ul LLM poate fi schimbat în orice moment, dar rolul lui rămâne limitat la înțelegerea limbajului natural.

Ultima componentă a stratului de orchestrare este `interaction.py`. Aceasta permite agenților să pună întrebări pacientului fie în terminal, fie în interfața web, fără ca instrumentele să știe pe ce cale comunică. Modulul definește protocolul `InteractiveChannel`, cu metoda `ask`, și funcția-fabrică `make_ask`, care întoarce o funcție de întrebare legată de canalul activ. Dacă niciun canal nu este configurat, întrebarea este afișată cu `print` și răspunsul este citit cu `input`, păstrând comportamentul din linia de comandă. Dacă rulează interfața web, canalul de sesiune este legat la firul de execuție al echipajului printr-o variabilă de context (`ContextVar`), iar întrebarea este trimisă în browser. Acest mecanism este reutilizat de toate instrumentele interactive descrise în secțiunile următoare și face legătura cu interfața web prezentată în secțiunea 5.8.

5.3. Agentul de parsare a limbajului natural

Agentul `nlp_parser` are rolul de specialist de înțelegere a pacientului. Acesta realizează obiectivul O1 din secțiunea 2.2, transformarea descrierii naturale a pacientului într-o reprezentare structurată. Sarcina lui este `parse_symptoms_task`, care primește textul scris de pacient și rezultă un obiect JSON cu un set fix de câmpuri: identificatorul pacientului (`subject_id`), simptomele principale (`chief_complaints`), scorul durerii (`pain_score`), vârsta, sexul, modul de sosire (`arrival_transport`), antecedentele relatate (`prior_history`), numărul de internări anterioare (`n_prior_admissions`) și, doar pentru pacienții aduși cu ambulanța sau elicopterul, semnele vitale măsurate de echipaj. Schema aceasta este separarea concretă discutată teoretic în secțiunea 4.4. Modelul de limbaj completează spațiile dintr-o structură deja definită și tot ce nu se încadrează în ea este ignorat.

Următoarea descriere a fost transformată de agent în obiectul structurat din Listingul 5.2 pentru a ilustra forma rezultatului: „Am 68 de ani, m-a adus salvarea, mă doare puternic în piept de o oră și respir greu, iau pastile pentru tensiune și pentru diabet”. Simptomele sunt extrase și exprimate în termenii clinici scurți pe care îi folosesc modelele. Scorul durerii este dedus din intensitatea relatată, modul de sosire este recunoscut, iar pentru că pacientul a fost adus cu ambulanța sunt incluse și semnele vitale măsurate de echipaj. Identificatorul `-1` marchează un pacient neidentificat, la prima prezentare. Câmpurile pe care pacientul nu le menționează pot fi obținute prin întrebări de completare înainte ca obiectul să fie finalizat.

```
{
  "subject_id": -1,
  "chief_complaints": "chest pain, shortness of breath",
  "pain_score": 8,
  "age": 68,
  "gender": "male",
  "arrival_transport": "ambulance",
  "prior_history": "hypertension, diabetes",
```

```
"n_prior_admissions": 2,
"vitals": { "heartrate": 112, "sbp": 158, "o2sat": 92 }
}
```

Listing 5.2: Exemplu ilustrativ de obiect de intrare produs de agentul de parsare.

Agentul dispune de două instrumente. Primul este **AskPatientTool**, care conține o schemă de intrare cu un singur câmp (**question**) și permite agentului să pună o întrebare de completare atunci când un câmp obligatoriu lipsește din descrierea inițială. Aceasta este realizarea buclei interactive de parsare din secțiunea 4.4. În loc să imputeze o valoare lipsă, agentul o cere în mod direct. Întrebarea ajunge la pacient prin funcția **make_ask** din **interaction.py**, deci în terminal este afișată cu **input**, iar în interfața web este trimisă în browser, fără ca instrumentul să fie conștient de calea folosită. Al doilea instrument este **ConfirmIntakeTool**, care este apelat ca pas final cu obiectul JSON construit de agent. La terminal, el returnează JSON-ul neschimbat, deci nu modifică fluxul din linia de comandă. În contrast, interfața web afișează câmpurile extrase într-un formular care poate fi editat și permite pacientului sau clinicianului să corecteze orice eroare de extragere înainte ca triajul să pornească. Acest pas este un punct de control plasat intenționat înaintea primei predicții clinice.

Indiferent de calea de extragere, simptomele principale trec apoi prin stratul de normalizare partajat din **preprocessing.py**, descris teoretic în secțiunea 4.7. Funcția **normalize_complaint_text** transformă textul la litere mici, înlocuiește semnele de punctuație cu spații, expandează abrevierile din dicționarul **ABBREVIATIONS** (de exemplu **sob** devine “shortness of breath”, **cp** devine “chest pain”, **abd** devine “abdominal”) și elimină cuvintele de un singur caracter. A doua transformare este **clinicalize_complaint**, care rescrie parafraze de limbaj comun în termeni clinici. Modelele au învățat acești termeni clinici din date, prin dicționarul **LAY_TO_CLINICAL** (de exemplu “stomach pain” devine “abdominal pain”, iar “brain bleed” devine “intracranial hemorrhage”). Această a doua transformare se aplică *numai* ieșirii în limbaj natural a parserului. Niciodată nu se va aplica această transformare datelor tabulare și niciodată la antrenare, tocmai pentru a nu modifica vocabularul pe care a fost antrenat vectorizatorul TF-IDF. Aceasta e metoda practică prin care se reduce diferența dintre limbajul pacientului și terminologia clinică scurtă, limitarea în cauză fiind recunoscută în secțiunea 4.4.

5.4. Agentul de triaj

Agentul **triage_agent** realizează obiectivul O2: estimarea acuității ESI și a probabilității de internare. Sarcina lui, **triage_assessment_task**, citește reprezentarea structurată produsă de parser și folosește instrumentul **TriagePredictionTool**, care integrează cele două modele XGBoost ale triajului. Schema de intrare a instrumentului este **TriageInput** și cuprinde paisprezece câmpuri: simptomele, scorul durerii, vârsta, sexul, modul de sosire, cele șase semne vitale, antecedentele relatate, numărul de internări anterioare și un câmp opțional pentru blocul de antecedente preluat din fișa electronică. Valorile necunoscute au convenții explicite (de exemplu un semn vital necunoscut este transmis ca -1), astfel încât modelul de limbaj să poată semnala absența unei valori fără a o inventa.

Schema de intrare **TriageInput** este redată în Listingul 5.3 în formă prescurtată. Fiecare câmp are o valoare implicită care codifică absența informației (un semn vital lipsă este -1 , antecedentele lipsă sunt șirul vid). În acest mod, instrumentul poate fi apelat și atunci când unele date lipsesc.

```
class TriageInput(BaseModel):
```

```

chief_complaints: str
pain_score: int # 0-10; -1 daca lipseste
age: int = 50
gender: str = "unknown"
arrival_transport: str = "unknown"
temperature: float = -1.0
heartrate: float = -1.0
resprate: float = -1.0
o2sat: float = -1.0
sbp: float = -1.0
dbp: float = -1.0
prior_history: str = ""
n_prior_admissions: int = -1
pmh_lookup_json: str = ""
    
```

 Listing 5.3: Schema de intrare **TriageInput** (prescurtat).

Metoda `_run` a instrumentului construiește vectorul de caracteristici respectând exact aceeași ordine în care a fost antrenat modelul, prin concatenarea a patru grupuri de informații: douăzeci și trei de caracteristici structurate moștenite din triajul v1 (durere, demografice, mod de sosire și interacțiunile dintre ele), douăzeci și opt de caracteristici legate de semnele vitale (valorile brute, indicatorii de absență, indicatorii de anormalitate precum febră sau tahicardie, și interacțiunile lor cu vârsta și modul de sosire), nouăsprezece caracteristici de antecedente și două mii de caracteristici TF-IDF. Rezultă din acestea un vector de aproximativ 2070 de coloane. Modelele și artefactele asociate (vectorizatorul TF-IDF, harta de severitate, medianele semnelor vitale și metadatele) sunt încărcate o singură dată din directorul `TRIAGE_V3_DIR` și păstrate într-o memorie locală ca să nu fie recitite la fiecare apel. Cascada descrisă în secțiunea 4.5 apare aici în forma ei cea mai directă. Modelul de acuitate prezice mai întâi nivelul ESI, iar acest nivel este adăugat ca o caracteristică suplimentară (`predicted_acuity`) în vectorul transmis modelului de dispoziție. Rezultatul instrumentului este un obiect JSON cu nivelul ESI prezis, probabilitățile asociate fiecăruia dintre cele cinci niveluri, predicția de dispoziție și gradul de încredere al acesteia.

Cele patru blocuri și dimensiunile lor sunt rezumate în Tabelul 5.2. Un exemplu de rezultat produs de acest instrument este dat în Listingul 5.4, într-o formă prescurtată la câmpurile de predicție.

Tabela 5.2: Cele patru blocuri ale vectorului de caracteristici al triajului v3.

Bloc	Coloane	Exemple
Structurate (v1)	23	durere, vârstă, sex, mod de sosire, priori de severitate, interacțiuni
Semne vitale (v2)	28	valori brute, indicatori de absență și de anormalitate, interacțiuni
Antecedente (PMH, v3)	19	13 indicatoare de grup de diagnostic + 6 numerice de istoric
Text TF-IDF	2000	unigrame, bigrame și trigrame ale simptomelor
Total	2070	

```

{
  "acuity_prediction": {
    "predicted_esi_level": 2,
    
```

```

    "confidence": "61.4%",
    "probability_breakdown": {
        "ESI 1": "4.2%", "ESI 2": "61.4%", "ESI 3": "30.1%",
        "ESI 4": "3.1%", "ESI 5": "1.2%"
    }
},
"disposition_prediction": { "prediction": "ADMITTED", "confidence": "78.0%" }
}

```

Listing 5.4: Exemplu ilustrativ de rezultat al instrumentului de triaj (prescurtat).

În etapa de inferență, blocul de nouăsprezece caracteristici de antecedente este construit prin aceeași logică pe care o folosesc instrumentele doctorului. În acest fel, modul de reprezentare a antecedentelor rămâne consistent în întregul sistem. Lista celor treisprezece grupuri de diagnostic și vocabularul de cuvinte-cheie care le activează se află în modulul `pmh_vocab.py`, prin funcția `flags_from_text`. Aceasta caută potriviri la nivel de cuvânt și ține cont de negații precum “no history of” sau “denies”, astfel încât o afecțiune menționată ca fiind absentă să nu fie interpretată din greșeală ca fiind prezentă. Ordinea coloanelor blocului este fixată de constanta `PMH_FEATURE_COLS` din `pmh_features.py`. Ele sunt ordonate astfel: cele treisprezece indicatoare binare de grup, urmate de șase valori numerice cu privire la istoricul de vizite (numărul de internări și de prezentări anterioare, numărul de zile care au trecut de la ultima vizită, indicatorul de simptom identic cu o prezentare anterioară și indicatorul de absență a istoricului). Atunci când pacientul nu raportează antecedente (și pacientul nu declară un cod unic de identificare valid, pe baza căruia s-ar căuta istoricul său din fișele electronice), aceste câmpuri sunt completate cu valorile de substituție, corespunzătoare unui pacient aflat la prima prezentare (`no_history` egal cu 1). Acesta este exact tiparul pe care modelul l-a întâlnit pe aproximativ 39% dintre rândurile de antrenare.

Pentru pacienții care revin și își dau codul de identificare, `triage_agent` dispune și de instrumentul `PatientHistoryLookupTool`. Acesta are acces la un index de prezentări anterioare construit în prealabil și îi returnează agentului toate antecedentele pacientului, precum și detaliile importante pe care pacientul nu le poate raporta la pat, precum numărul de zile de la ultima internare sau dacă la prezentarea curentă raportează un simptom identic cu unul din trecut. Interogarea este filtrată din punct de vedere al timpului pentru a se asigura că numai prezentările anterioare momentului curent sunt folosite. Acest lucru garantează că este respectată constrângerea de absență a scurgerilor de informație din secțiunea 4.7. Dacă antecedentele medicale au fost auto-raportate de către pacient, dar este disponibil și acest bloc real de antecedente pe baza codului unic de verificare, sunt luate în considerare doar cele din blocul real pentru predicție. În schimb, antecedentele pe care pacientul le-a menționat, dar nu apar în fișa lui electronică, vor fi semnalate printr-o funcție de reconciliere, fără a modifica predicția.

Modelele triajului sunt produse de pipeline-ul `train_triage_v3.py`. Antrenarea folosește exact aceeași funcție `build_features` ca inferența, ceea ce garantează identitatea dintre cele două situații. Dezechilibrul claselor este corectat prin ponderarea atenuată proporțională cu rădăcina pătrată a inversului frecvenței clasei, care a fost discutată în secțiunea 4.6.3. Specificul ordinal al acuității, tratat teoretic în secțiunea 4.6.4, este implementat prin două ajustări aplicate doar modelului de acuitate. Prima este un multiplicator de pondere pentru clasele extreme, `ESI_EXTREME_BOOST`, care crește importanța nivelului critic 1 și a nivelului neurgent 5 (valorile 1,5 pentru ESI 1, 1,3 pentru ESI 2 și 2,0 pentru ESI 5). A doua este înlocuirea criteriului de oprire timpurie cu o metrică proprie,

neg_quadratic_kappa. Această metrică selectează iterăția cu cel mai bun acord ordinal, măsurat prin kappa ponderat quadratic, în locul celei cu cea mai mică pierdere logaritmică. Aceste decizii din implementare reflectă în cod diferența de cost dintre sub-triaj și supra-triaj, așa cum este definită în secțiunea 4.1.

Structura instrumentelor pe care le folosesc agenții urmează un tipar comun, ilustrat în Figura 5.3. Fiecare instrument este o clasă care extinde **BaseTool** din CrewAI, expune un nume, o descriere și o schemă de argumente de tip Pydantic, și implementează metoda **_run** care produce rezultatul în format JSON. Schema de argumente este vizibilă modelului de limbaj și îi spune ce parametri poate furniza. Câmpurile interne ale instrumentului, precum canalul interactiv, sunt ascunse de aceasta. Acest tipar uniform este motivul pentru care toate predicțiile clinice pot fi adăugate în echipaj fără a schimba logica de orchestrare.

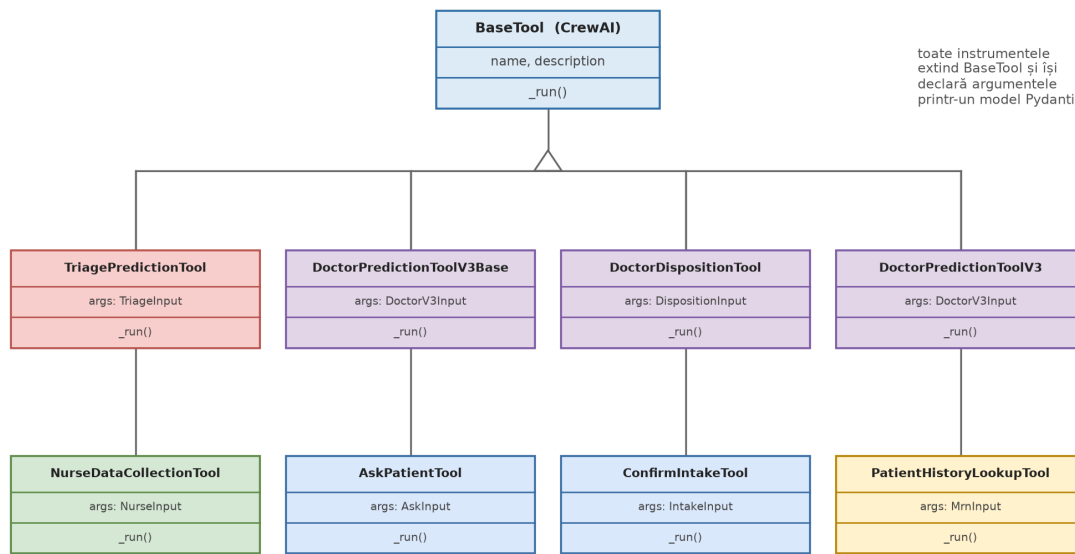


Figura 5.3: Diagrama de clase a instrumentelor agenților: toate extind **BaseTool** și își declară schema de argumente printr-un model Pydantic.

5.5. Agentul doctor

Agentul **doctor_agent** este singurul care intervine de mai multe ori în flux. El execută trei sarcini distincte: o evaluare inițială imediat după triaj, o rafinare a deciziei de internare după ce asistentul colectează datele suplimentare, și o reevaluare finală a diagnosticului și departamentului. Această împărțire în trei nu este o redundanță, ci principiul central de proiectare al lucrării, formulat ca obiectivul O4 din secțiunea 2.2: se separă explicit predicția făcută înainte de colectarea datelor de cea făcută după, pentru a măsura direct valoarea clinică a informațiilor adunate de asistent. Cele trei sarcini folosesc trei instrumente diferite, fiecare cu propriul set de modele.

5.5.1. Evaluarea inițială

Sarcina **doctor_assessment_task** citește blocul structurat produs de triaj și, dacă pacientul este prezis a fi internat, apelează instrumentul **DoctorPredictionToolV3Base**.

Schema lui de intrare este **DoctorV3BaseInput** și conține șapte câmpuri: simptomele, scorul durerii, vârsta, sexul, modul de sosire, acuitatea prezisă la triaj și indicatorul de internare. Acuitatea este transmisă mai departe ca o caracteristică, realizează cascada dură descrisă în secțiunea 4.5. Modelul de diagnostic primește douăzeci și trei de caracteristici structurate, două mii de caracteristici TF-IDF și nivelul de acuitate prezis. După ce diagnosticul este estimat, eticheta lui cea mai probabilă este la rândul ei adăugată ca o caracteristică în vectorul transmis către modelul de departament. Și a doua treaptă a cascadei este deci una dură. Instrumentul returnează primele trei categorii de diagnostic și primele trei departamente, fiecare cu probabilitatea asociată, pentru pacienții internați.

Spațiul de etichete al diagnosticului nu este format din coduri ICD individuale, ci din categorii suficient de populate pentru a permite antrenarea, conform discuției din secțiunea 4.1. Gruparea se face în două etape. Codurile ICD sunt mai întâi reduse la categorii de diagnostic, apoi acestea sunt unite în paisprezece clase prin tabelul **DIAGNOSIS_GROUP_MAP**. Unele categorii sunt păstrate ca atare pentru că sunt destul de mari, altele sunt unite. Variantele de sistem nervos, împreună cu bolile de ochi și de ureche, sunt unite într-o singură clasă pentru a corecta o separare artificială apărută ca urmare a coexistenței codificărilor ICD-9 și ICD-10. Categoriile mici, precum sarcina, neoplasmale sau anomalii congenitale, sunt adunate într-o clasă **Other**. Cele paisprezece categorii rezultate sunt prezentate în Tabelul 5.3. Tier-ul v3, care este cel mai recent model și cel folosit la runtime, exclude în plus clasa ambiguă **Symptoms, Signs, Ill-Defined**. Această clasă conține aproximativ o treime din pacienții internați, dar în ea sunt prezente cazuri care nu au un diagnostic precis, sau informația clinică este insuficientă sau ambiguă. Astfel, rămân treisprezece clase bine-definite.

Tabela 5.3: Cele paisprezece categorii de diagnostic obținute prin gruparea codurilor ICD. Tier-ul v3 exclude clasa atrapă, rămânând cu treisprezece.

Categorie grupată	Observație
Circulatory	păstrată
Digestive	păstrată
Respiratory	păstrată
Genitourinary	păstrată
Musculoskeletal	păstrată
Endocrine, Nutritional, Metabolic	păstrată
Mental Disorders	păstrată
Skin and Subcutaneous Tissue	păstrată
Blood and Blood-Forming Organs	păstrată
Infectious and Parasitic	păstrată
Injury and Poisoning	păstrată
Nervous System and Sense Organs	sistem nervos + ochi + ureche (artefact ICD-9/10)
Other	sarcină, neoplasme, anomalii congenitale, coduri rare
Symptoms, Signs, Ill-Defined	clasă atrapă, exclusă în v3

Departamentele urmează aceeași logică de grupare. Tabelul **SERVICE_GROUP_MAP** reduce nouăsprezece servicii spitalicești brute la unsprezece clase. Serviciile chirurgicale de specialitate (vascular, toracic, cardiac, plastic) sunt unite într-o clasă **OTHER_SURG**, obstetrica și ginecologia într-o clasă **OB_GYN**, iar specialitățile mici (urologie, psihiatrie,

ORL, oftalmologie, stomatologie) într-o clasă **OTHER**. Cele unsprezece departamente sunt date în Tabelul 5.4, împreună cu denumirile lor clare din **DEPARTMENT_NAMES**.

Tabela 5.4: Cele unsprezece departamente obținute prin gruparea serviciilor spitalicești.

Cod	Denumire
MED	General Medicine
CMED	Cardiac Medicine
NMED	Neuro Medicine
SURG	General Surgery
OMED	Oncology Medicine
ORTHO	Orthopedics
NSURG	Neurosurgery
TRAUM	Trauma
OTHER_-SURG	Other Surgery (vascular, toracic, cardiac, plastic)
OB_GYN	Obstetrics / Gynecology
OTHER	Other Specialty (urologie, psihiatrie, ORL, oftalmologie, stomatologie)

Cele două modele de diagnostic și departament ale evaluării inițiale sunt produse de pipeline-ul `train_doctor_v3.py`. Acesta filtrează setul de date la pacienții internați, exclude clasa ambiguă pe care am menționat-o anterior și nu mai aplică limitarea la o sub-eșantionare de o sută de mii de rânduri, antrenând astfel pe întregul set filtrat. O particularitate de implementare care merită menționată este că modelele de diagnostic și departament reutilizează artefactele de cascadă ale triajului v1 (vectorizatorul TF-IDF și modelele de acuitate și dispoziție din **TRIAGE_V1_DIR**), pentru că pe acestea au fost antrenate. Păstrarea aceleiași surse de cascadă la inferență menține similaritatea dintre antrenare și inferență cerută în secțiunea 4.7, chiar dacă agentul de triaj din runtime folosește modelele v3.

5.5.2. Rafinarea dispoziției

După ce asistentul colectează datele suplimentare, dar înainte de reevaluarea diagnosticului, sarcina `doctor_disposition_task` apelează instrumentul `DoctorDispositionTool`, iar rolul lui realizează obiectivul O5. Astfel, rezultă o decizie de internare sau externare mai precisă și calibrată, care poate înlocui verdictul de filtrare produs în etapa de triaj. Schema de intrare `DoctorDispositionInput` are douăzeci de câmpuri, adunând toate semnalele disponibile în acest moment: ieșirile triajului, semnele vitale instantanee și traiectoria lor opțională de mai multe citiri, ritmul cardiac, medicația, antecedentele și blocurile opționale preluate din fișa electronică.

Acest model de dispoziție se deosebește de modelul de dispoziție al triajului prin două decizii de proiectare. Prima este legată de datele de antrenare. Spre deosebire de modelele de diagnostic și departament, antrenate doar pe pacienții internați, modelul de dispoziție este antrenat pe întregul set de prezentări (internate și externate), pentru că decizia de internare are sens pentru orice pacient. A doua ține de cascadă. În locul cascadei dure, în care se transmite o singură etichetă, acest model folosește cascada moale descrisă în secțiunea 4.5. Instrumentul reia modelul de acuitate al triajului v3 ca să obțină întreaga distribuție softmax pe cele cinci niveluri. La acestea adaugă probabilitatea de internare estimată de triaj. Rezultă șase coloane suplimentare (`triage_acuity_proba_1` până la 5 și `triage_disposition_proba_admit`), care înlocuiesc transmiterea unei

singure etichete de acuitate. Rezultă un vector de caracteristici de aproximativ 2128 de coloane, prin concatenarea blocului triajului v3, a celor șase coloane de cascadă moale, a celor unsprezece coloane de medicație și a celor patruzeci și una de coloane de semne vitale longitudinale.

Modelul salvat este un clasificator binar calibrat izotonic (`CalibratedClassifierCV`), astfel încât metoda `predict_proba` returnează probabilități calibrate, în sensul discutat în secțiunea 4.6.5. O probabilitate de internare de 0,82 înseamnă că aproximativ 82% dintre pacienții cu acest scor au fost într-adevăr internați. Pragul de decizie este fixat la 0,40, nu la valoarea implicită de 0,50, în constanta `DECISION_THRESHOLD`. Dacă pragul este coborât și mai mult, rata de sub-triaj se reduce și mai mult, adică mai puține internări ratate. În schimb, ar crește rata de supra-triaj, conform asimetriei menționate în secțiunea 4.1. Pentru că probabilitățile sunt calibrate, pragul are o semnificație clinică clară și poate fi deplasat de-a lungul curbei ROC fără a reantrena modelul. Rezultatul instrumentului raportează probabilitatea calibrată de internare, decizia binară și dacă aceasta confirmă sau contrazice verdictul de triaj. Raportul conține și direcția contrazicerii (de exemplu, decizia de externare s-a transformat în internare). Toate acestea sunt ilustrate în Listingul 5.5. Reevaluarea finală se realizează pe baza acestei decizii rafinate, nu pe cea de triaj. Modelul este produs de pipeline-ul `train_doctor_disposition.py`.

```
{
  "disposition_prediction": {
    "is_admitted": true,
    "decision": "ADMIT",
    "p_admit": 0.83,
    "p_discharge": 0.17,
    "decision_threshold": 0.40
  },
  "triage_v3_baseline": {
    "triage_admit_input": false,
    "doctor_override_triage": true,
    "override_direction": "discharge -> admit"
  }
}
```

Listing 5.5: Exemplu ilustrativ de rezultat al instrumentului de rafinare a dispoziției (prescurtat).

5.5.3. Reevaluarea și recuperarea codului ICD exact

Sarcina `doctor_reassessment_task` reia evaluarea de diagnostic și departament, de data aceasta cu datele colectate de asistent, și se declanșează numai dacă decizia de dispoziție rafinată este de internare. Sarcina apelează instrumentul `DoctorPredictionToolV3`, a cărui schemă `DoctorV3Input` adaugă, peste câmpurile evaluării inițiale, semnele vitale, ritmul cardiac, medicația, antecedentele și traiectoria opțională de semne vitale. Vectorul de caracteristici cuprinde blocul de caracteristici al triajului, douăzeci de caracteristici de semne vitale instantanee (valorile, indicatorii de absență, indicatorii clinici și presiunea arterială medie), unsprezece coloane de medicație, patruzeci și una de coloane de semne vitale longitudinale și ritm, și nouăsprezece coloane de antecedente. Rezultă astfel aproximativ 2116 de coloane în total.

Spre deosebire de evaluarea inițială, care transmite modelului de departament doar eticheta de diagnostic cea mai probabilă, reevaluarea folosește o cascadă moale și aici. Întreaga distribuție softmax pe cele treisprezece categorii de diagnostic este transmisă

către modelul de departament prin coloanele construite de `build_diag_cascade_cols`, permițându-i să distingă cazurile sigure de cele incerte. Mecanismul concret al celor două tipuri de cascadă, dură și moale, este arătat în Listingul 5.6. Cascada dură transmite doar clasa finală (un singur indice), în timp ce cascada moale transmite întreaga distribuție de probabilități.

```
# Cascada dura (evaluarea initiala): doar eticheta de diagnostic
features_dept["predicted_diagnosis"] = diag_pred_idx

# Cascada moale (dispozitie): toata distributia de acuitate + P(internare)
for k in range(5):
    triage_features[f"triage_acuity_proba_{k+1}"] = float(acuity_proba[k])
triage_features["triage_disposition_proba_admit"] = triage_dispo_proba_admit

# Cascada moale (reevaluare): toata distributia de diagnostic
for k, col in enumerate(diag_cascade_cols):
    features_dept[col] = float(diag_proba[k])
```

Listing 5.6: Cascada dură (un singur indice) față de cascada moale (distribuția completă), extras din instrumentele doctorului.

Instrumentul returnează diagnosticul și departamentul reevaluate, fiecare cu primele trei variante, precum și o comparație explicită cu evaluarea inițială. Această comparație este măsurarea directă a valorii aduse de datele asistentului, cerută de obiectivul O4.

Predicția de diagnostic se oprește la nivelul categoriei. Codul ICD exact din interiorul categoriei este recuperat printr-o etapă suplimentară de ordonare, formulată ca o problemă de ranking în secțiunea 4.6.6. Funcția `_resolve_exact_diagnoses` încarcă, dacă există, indexul construit în prealabil din directorul `DOCTOR_V3_ICD_RESOLVER_DIR` și ordonează codurile candidate din primele cinci categorii prezise după scorul de combinație dintre similaritatea TF-IDF, profilul fiziologic, antecedentele condiționate și prevalența codului. Etapa alege cea mai bogată variantă de scor pe care o permit datele pacientului. Varianta cu antecedente condiționate e folosită atunci când pacientul are istoric, varianta cu semne vitale când acestea există, iar în rest e folosită varianta de bază cu text și prevalență. Recuperarea aceasta are rol strict consultativ și nu modifică predicția de categorie sau de departament, așa cum se vede în Listingul 5.7. Dacă indexul nu a fost construit, blocul este pur și simplu absent, fără a opri restul predicției. Modelele reevaluării sunt produse de `train_nurse_v3.py`. Indexul de recuperare a codului ICD e produs de `train_icd_resolver.py`, care construiește prototipurile pe categorii și calibrează ponderile combinației offline.

```
"exact_diagnoses": {
  "granularity": "ICD 3-char rollup",
  "method": "TF-IDF cosine + vitals + PMH + prevalence (gated)",
  "flat_ranking": [
    { "icd_3char": "I50", "title": "Heart failure", "combined_score": 0.71 },
    { "icd_3char": "I21", "title": "Acute MI", "combined_score": 0.55 }
  ]
}
```

Listing 5.7: Exemplu ilustrativ de bloc consultativ de coduri ICD exacte (prescurtat).

Cele trei intervenții ale doctorului, instrumentele și datele lor sunt rezumate în Tabelul 5.5.

Pentru a urmări cum se schimbă predicțiile pe parcursul celor trei evaluări, se poate relua pacientul din exemplul de intrare prezentat în secțiunea 5.3, adică bărbatul

Tabela 5.5: Cele trei evaluări ale agentului doctor, cu instrumentul, datele disponibile și ieșirea fiecăreia.

Evaluare	Instrument	Date	Ieșire principală
Inițială	DoctorPredictionToolV3Base	triaj	diagnostic + departament (top-3)
Dispoziție	DoctorDispositionTool	triaj + asistent	internare/externare calibrată
Reevaluare	DoctorPredictionToolV3	triaj + asistent	diagnostic + departament reevaluate + cod ICD exact

de 68 de ani adus cu ambulanța, cu durere în piept și dificultate de respirație, cu hipertensiune și diabet în antecedente. La prima evaluare, doctorul dispune doar de datele de la triaj. Instrumentul `DoctorPredictionToolV3Base` produce o primă listă de categorii de diagnostic și de departamente, de exemplu categoria `Circulatory` și departamentul `CMED`, fără să cunoască încă semnele vitale sau medicația pacientului.

Asistentul colectează apoi semnele vitale, inclusiv o a doua măsurătoare care arată o frecvență cardiacă în creștere (de exemplu, la prima citire 80 de bătăi pe minut, iar la a doua citire 90 de bătăi pe minut) și o saturație în scădere (de exemplu, la prima citire saturația 95, iar la a doua citire saturația 90), împreună cu medicația și antecedentele. Instrumentul de dispoziție folosește aceste date pentru a produce o probabilitate de internare calibrată, de exemplu 0,83, ceea ce confirmă decizia de internare. Pentru un pacient aflat la limită, aceeași etapă ar putea schimba verdictul de triaj, pentru că decizia se ia acum pe pragul de 0,40 și pe traiectoria reală a semnelor vitale, nu pe o singură măsurătoare. Reevaluarea reia în final diagnosticul și departamentul cu setul complet de date. Tendința de agravare a saturației și prezența medicației cardiace pot modifica distribuția de probabilități față de evaluarea inițială, iar instrumentul raportează explicit această diferență. Etapa consultativă de recuperare a codului ICD sugerează codul exact cel mai probabil în interiorul categoriei prezise. Comparația dintre prima și ultima evaluare se face pe același pacient și în aceleași condiții și reprezintă exact măsurarea valorii adăugate de datele colectate de asistent, cerută de obiectivul O4.

5.6. Agentul asistent

Agentul `nurse_agent` colectează interactiv datele care nu sunt disponibile la sosire, prin sarcina `nurse_data_collection_task` și instrumentul `NurseDataCollectionTool`. Spre deosebire de celelalte instrumente, acesta nu încapsulează un model, ci poartă un dialog cu pacientul, folosind aceeași funcție `make_ask` ca agentul de parsare. La terminal întrebările sunt citite de la tastatură, iar în interfața web sunt trimise în browser. Instrumentul nu este conștient de calea aleasă.

Pentru a se colecta o serie de citiri se folosește funcția `_collect_reading_round`, care cere pe rând temperatura, frecvența cardiacă, frecvența respiratorie, saturația de oxigen, tensiunea arterială (interpretată din forme precum „120/80”) și ritmul cardiac, plus un moment de timp opțional al citirii. Oricare câmp poate fi omis. Orice valoare pe care pacientul nu o știe este marcată ca lipsă, fără ca instrumentul să o estimeze. Aceasta este punerea în practică a modului în care sunt tratate datele parțiale discutate în secțiunea 4.2. O valoare necunoscută rămâne necunoscută, iar modelul o tratează ca atare printr-un indicator de absență.

Instrumentul poate colecta unul sau mai multe seturi de citiri în ordine cronologică. Primul set este măsurătoarea de la sosire. Seturile suplimentare permit construirea unei traiectorii reale a semnelor vitale. Funcția `build_nurse_payload` construiește rezultatul standard al datelor: un instantaneu (`vital_signs`), traiectoria pe fiecare semn vital (`vital_trajectory`, ordonată după momentul citirii dacă toate seturile conțin un moment al citirii, altfel după ordinea în care au fost introduse), ritmul, medicația, antecedentele și numărul de internări anterioare. Această funcție este sursa unică a formei rezultatului, apelată identic de varianta din terminal și de cea din interfața web, astfel încât cele două produc o structură identică. Un exemplu de rezultat este dat în Listingul 5.8.

```
{
  "vital_signs": { "temperature": 98.6, "heartrate": 112, "resprate": 22,
                  "o2sat": 92, "sbp": 158, "dbp": 94 },
  "vital_trajectory": { "heartrate": [88, 105, 112], "o2sat": [98, 95, 92] },
  "rhythm": "sinus tachycardia",
  "medications_raw": "metoprolol, metformin",
  "prior_history": "hypertension, diabetes",
  "n_prior_admissions": 2
}
```

Listing 5.8: Exemplu ilustrativ de rezultat al agentului asistent, cu o traiectorie de mai multe citiri (prescurtat).

Traectoria de semne vitale este transformată în caracteristici de modulul `vital_trajectory.py`, prin funcția `build_longitudinal_block`, care produce blocul de patruzeci și una de coloane folosit de instrumentul de dispoziție și de cel de reevaluare. Pentru fiecare dintre cele șase semne vitale se calculează minimul, maximul, ultima valoare și diferența dintre prima și ultima citire (douăzeci și patru de coloane). La acestea se adaugă șapte numărători de citiri anormale (febră, tahicardie și celelalte), indicatoarele one-hot ale ritmului cardiac împreună cu un indicator de ritm neregulat și un indicator care arată dacă a existat o traiectorie reală sau doar o singură măsurătoare. Atunci când există o singură citire, blocul se reduce la citiri de tip instantaneu, adică o singură valoare pentru fiecare. Minimul, maximul și ultima valoare egale și diferența vor fi zero, în acest caz. Această cale degradată este corectă, dar nu poate să surprindă tendința, motiv pentru care colectarea unei a doua sau mai multor citiri îmbunătățește predicția de internare. Aceeași logică de agregare este folosită la antrenare, pe citirile din `vitalsign.csv`, și la inferență, prin același modul. În acest fel, reprezentarea nu poate să difere între cele două situații.

Medicația raportată de pacient este clasificată prin vocabularul partajat din modulul `med_vocab.py`, în nouă categorii (cardiace, antidiabetice, psihiatrice, respiratorii, opioide, anticoagulante, gastrointestinale, tiroidiene și anticonvulsivante). Acestea sunt reprezentate prin unsprezece coloane, împreună cu numărul de medicamente și un indicator de medicație necunoscută. Potrivirea se face atât pe numele medicamentului, generic sau comercial, prin `flags_from_name`, cât și pe cuvinte-cheie de clasă, prin `flags_from_text`. Sunt tratate în mod explicit negațiile pentru a evita interpretările eronate. Vocabularul este partajat între antrenare, unde citește coloanele din `medrecon.csv`, și inferență, unde interpretează lista raportată de pacient. Astfel, reprezentarea indicatorilor de medicație rămâne aliniată între cele două etape, în același spațiu de caracteristici. Parsarea traiectoriei transmise mai departe către instrumentele doctorului se face în modulul `vital_trajectory_io.py`, prin funcțiile `parse_vital_trajectory` și

parse_rhythm_readings.

5.7. Pipeline-ul de antrenare și modelul de date

Modelele folosite de instrumente sunt produse de pipeline-urile din subpachetul `training`, fiecare expus ca o subcomandă în `pyproject.toml` și apelat din `main.py`. Există câte un pipeline pentru fiecare tier de model: triajul în trei variante (`train_triage_v1`, `train_triage_v2`, `train_triage_v3`), modelele inițiale și de reevaluare ale doctorului (`train_doctor`, `train_nurse`, `train_doctor_v3`, `train_nurse_v3`), modelul de dispoziție (`train_doctor_disposition`) și indexul de recuperare a codului ICD (`train_icd_resolver`).

Principiul comun al tuturor pipeline-urilor este funcția `build_features`. Aceeași funcție este folosită atât la antrenare, cât și la inferență. La antrenare este folosită cu argumentul `fit=True`, care estimează vectorizatorul TF-IDF și medianele. La inferență, este folosită cu `fit=False`, apelată de instrumente cu artefactele deja salvate. Această reutilizare este mecanismul concret prin care se asigură simetria dintre antrenare și inferență cerută în secțiunea 4.7. Logica de construire a caracteristicilor nu este rescrisă în două locuri, ci apelată din același modul, ca în Listingul 5.9.

```
# La antrenare (fit=True): estimeaza TF-IDF, harta de severitate, medianele
X_train, tfidf, severity_map, vital_medians = build_features(
    df_train, tfidf=None, severity_map=None,
    vital_medians=None, fit=True,
)

# La inferenta (fit=False): refoloseste aceleasi artefacte salvate
X, _, _ = build_features(
    df_row, tfidf=tfidf, severity_map=severity_map,
    vital_medians=vital_medians, fit=False,
)
```

Listing 5.9: Aceeași funcție `build_features` la antrenare (`fit=True`) și la inferență (`fit=False`).

Tot în pipeline-uri sunt aplicate ponderarea atenuată a claselor, oprirea timpurie pe kappa ponderat quadratic pentru acuitate și calibrarea izotonică pentru dispoziție și departament, descrise în secțiunile 4.6.3, 4.6.4 și 4.6.5. Artefactele rezultate sunt serializate cu `jolib` și salvate în directoare versionate sub `artifacts/triage` și `artifacts/doctor`, iar căile acestora sunt exportate din `paths.py`. Conținutul directoarelor modelelor active este arătat în Listingul 5.10. Fiecare director păstrează modelele lui, vectorizatorul și metadatele. Directorul de reevaluare adaugă modelul de dispoziție și subdirectorul de recuperare a codului ICD.

```
artifacts/
  triage/v3/
    acuity_model.joblib
    disposition_model.joblib
    tfidf_vectorizer.joblib
    severity_map.joblib
    vital_medians.joblib
    model_metadata.json
  doctor/v3_base/
    diagnosis_model.joblib
    department_model.joblib
    metadata.json
```



```

doctor/v3/
  diagnosis_model.joblib
  department_model.joblib
  disposition_model.joblib
  metadata.json
  icd_resolver/

```

Listing 5.10: Organizarea pe disc a artefactelor modelelor active.

Dimensiunile vectorilor de caracteristici ai celor patru modele active sunt rezumate în Tabelul 5.6. Diferențele dintre ele reflectă datele suplimentare pe care le primește fiecare etapă a fluxului.

Tabela 5.6: Dimensiunea vectorului de caracteristici pentru cele patru modele active.

Model	Caracteristici
Triaj v3 (acuitate, dispoziție)	2070
Doctor v3 inițial (diagnostic, departament)	2025
Doctor dispoziție v3	2128
Doctor v3 reevaluare (diagnostic, departament)	2116

Etichetele și caracteristicile provin din baza de date MIMIC-IV-ED. Tabelul `triage.csv` aduce simptomele, scorul durerii, semnele vitale de la triaj și nivelul de acuitate ESI, care este eticheta modelului de acuitate. Tabelul `edstays.csv` aduce sexul, modul de sosire și dispoziția finală, care este eticheta modelului de internare, împreună cu momentele de sosire și de plecare și identificatorii care leagă prezentarea de restul tabelor. Categoria de diagnostic provine din `diagnosis.csv`, prin gruparea codurilor ICD, iar departamentul din `services.csv`. Vârsta se calculează din `patients.csv`. Semnele vitale longitudinale și ritmul cardiac provin din `vitalsign.csv`, medicația de acasă din `medrecon.csv`, iar antecedentele din codurile ICD ale prezentărilor anterioare (`diagnoses_icd.csv`) și din secțiunea de antecedente a rezumatelor de externare anterioare (`discharge.csv`). Fluxul de la tabele la blocurile de caracteristici și la etichete este prezentat în Figura 5.4.

Modelele active sunt antrenate pe seturi de dimensiuni diferite, în funcție de populația pe care o vizează. Modelele de triaj și modelul de dispoziție folosesc întregul set de aproximativ 418.000 de prezentări curățate, împărțit 80/20 într-un set de antrenare de aproximativ 334.000 de prezentări și un set de test de aproximativ 83.000. Modelele de diagnostic și departament ale doctorului sunt antrenate numai pe pacienții internați și cu clasa ambiguă exclusă, ceea ce lasă aproximativ 102.000 de prezentări, împărțite în același raport, cu un set de test de aproximativ 20.000. Împărțirea folosește o sămânță fixă, astfel încât același set de test este reținut la fiecare reantrenare. Rezultatele măsurate pe aceste seturi de test reținute sunt prezentate în capitolul următor.

O cerință principală legată de construirea caracteristicilor este absența scurgerilor de informație, discutată teoretic în secțiunea 4.7. Antecedentele și medicația preluate din înregistrări sunt filtrate strict temporal, folosind numai prezentările anterioare momentului de sosire al cazului curent. Rezumatele de externare sunt restrânse la internările anterioare, astfel încât rezumatul prezentării curente nu poate să se scurgă în caracteristici. Semnele vitale longitudinale folosite la antrenare sunt limitate la o fereastră scurtă de la sosire. Indexul pe care îl interoghează instrumentul de căutare în fișa electronică la inferență este construit offline de funcția `build_pmh_index` și salvat tot ca artefact,

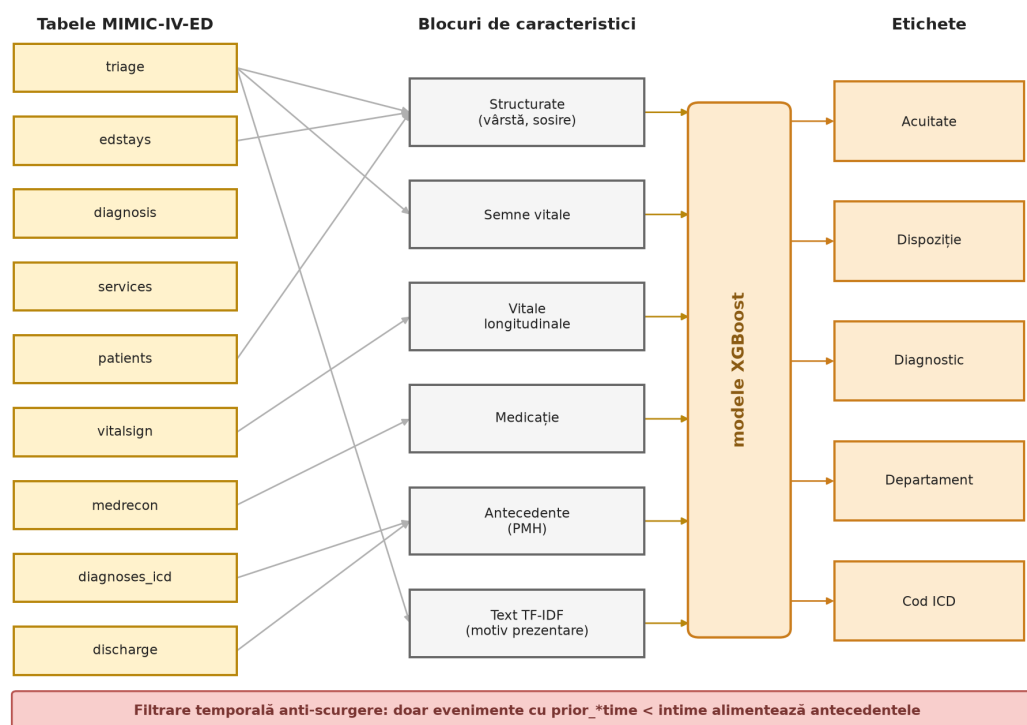


Figura 5.4: Modelul de date: tabelele MIMIC-IV-ED, blocurile de caracteristici derivate și etichetele, cu filtrarea temporală care previne scurgerile de informație.

păstrând aceeași filtrare temporală. Astfel, performanța măsurată reflectă onest ce poate face sistemul în practică, fără a beneficia de informație care nu ar fi fost disponibilă la momentul deciziei.

5.8. Interfața web și runtime-ul conversațional

Sistemul poate fi rulat din linia de comandă, dar dispune și de o interfață web conversațională, care realizează obiectivul O7. Particularitatea acestei interfețe este că nu reimplementează logica de predicție, ci pune în practică exact același echipaj de agenți ca varianta din terminal, prin același apel `ProiectLicenta().crew().kickoff(...)`. Singura schimbare este la nivelul transportului. În terminal, interacțiunea se realizează prin tastatură și consolă, iar în varianta web aceasta are loc prin browser. Prin urmare, predicțiile produse în interfața web sunt identice cu cele din linia de comandă.

Partea de server este construită cu FastAPI, în modulul `webapp/backend/live.py`. Fiecare sesiune primește un obiect `SessionChannel`, care funcționează ca o legătură sigură pentru comunicarea între echipaj și browser. Echipajul rulează într-un fir de execuție de fundal, iar canalul de sesiune este legat la acest fir prin variabila de context introdusă în secțiunea 5.2. Atunci când un instrument interactiv pune o întrebare, metoda `ask` a canalului o trimite către browser și blochează firul echipajului până când răspunsul sosește printr-o cerere POST. În acest fel, aceleași instrumente interactive folosite în terminal funcționează și în interfața web, fără modificări, pentru că primesc canalul prin câmpul lor `channel`. Mecanismul de blocare și deblocare a firului echipajului este arătat în Listingul 5.11. Întrebarea este emisă către browser, iar firul așteaptă pe o coadă până când endpoint-ul POST depune răspunsul.

```

class SessionChannel:
    def ask(self, kind, prompt, meta=None):
        self.emit({"type": "question", "kind": kind,
                  "prompt": prompt, "meta": meta or {}})
        answer = self._answer.get() # blocheaza firul echipajului
        if answer is _ABORT:
            raise RuntimeError("Session cancelled")
        return answer

    def answer(self, text): # apelata din endpoint-ul POST
        self._answer.put_nowait(text) # deblocheaza firul echipajului
    
```

Listing 5.11: Blocarea firului echipajului până la sosirea răspunsului din browser, în `SessionChannel`.

Activitatea agenților, a sarcinilor și a instrumentelor este transmisă către browser prin ascultători înregistrați pe magistrala de evenimente a CrewAI. Funcția `register_listeners` înregistrează o singură dată gestionari pentru evenimentele de început și de sfârșit de sarcină, de execuție a agentului și de folosire a instrumentului. Fiecare gestionar emite un eveniment către canalul sesiunii active. Raționamentul și ieșirile fiecărei sarcini sunt capturate prin funcțiile `step_callback` și `task_callback`. Evenimentele de la server către browser sunt transmise printr-un flux *Server-Sent Events* (SSE), expus la ruta `/api/live/stream`. Răspunsurile pacientului sunt trimise prin cereri POST la `/api/live/answer`. Un exemplu de evenimente din acest flux este dat în Listingul 5.12.

```

data: {"seq": 7, "type": "tool_started", "tool": "triage_prediction_tool",
      "agent": "Emergency Department Triage Specialist"}

data: {"seq": 8, "type": "question", "kind": "number",
      "prompt": "Heart rate? (bpm, e.g., 80)", "meta": {"reading": 1}}
    
```

Listing 5.12: Exemplu ilustrativ de evenimente din fluxul SSE (prescurtat).

Tipurile de evenimente transmise prin fluxul SSE către browser sunt rezumate în Tabelul 5.7. Ele acoperă întregul ciclu de execuție, de la pornirea echipajului până la rezultatul final, și sunt suficiente pentru ca browserul să reconstruiască întreaga conversație fără a interoga serverul.

Tabela 5.7: Tipurile de evenimente transmise prin fluxul SSE către browser.

Tip eveniment	Semnificație
<code>started</code>	echipajul a pornit
<code>task_started</code>	o sarcină a început
<code>agent_started</code>	un agent a început execuția
<code>tool_started</code>	un instrument a fost apelat, cu argumentele lui
<code>question</code>	un instrument cere un răspuns (tip, text, context)
<code>reasoning</code>	un pas de raționament al agentului
<code>tool_finished</code>	un instrument a terminat, cu rezultatul lui
<code>agent_message</code>	un agent a produs un mesaj
<code>task_completed</code>	o sarcină s-a încheiat, cu ieșirea ei
<code>final</code>	rezultatul final al echipajului
<code>error</code>	o eroare în execuție
<code>done</code>	sesiunea s-a încheiat, fluxul se închide

Ciclul de viață al unei sesiuni este gestionat prin patru rute. Ruta `/api/live/s-`

tart primește descrierea pacientului, creează un identificator unic de sesiune, pornește firul de execuție al echipajului și întoarce identificatorul. Ruta `/api/live/answer` transmite un răspuns către firul blocat, ruta `/api/live/cancel` întrerupe o întrebare aflată în așteptare, iar ruta de flux livrează evenimentele. Fluxul SSE trimite periodic un mesaj de menținere a conexiunii dacă nu apare niciun eveniment, iar la primirea evenimentului **done** închide conexiunea și eliberează sesiunea. Magistrala de evenimente a CrewAI este globală, dar pot exista mai multe sesiuni în paralel. Ținând cont de acest aspect, atribuirea unui eveniment către sesiunea corectă se face printr-o variabilă de context. Aceasta este legată la începutul firului fiecărui echipaj și propagată automat către ascultători, astfel încât fiecare eveniment ajunge numai la canalul sesiunii care l-a generat.

Partea de client este o aplicație React, construită cu Vite și stilizată cu Tailwind, în **webapp/frontend**. Componenta principală **App** consumă fluxul SSE și construiește conversația pe măsură ce evenimentele sosesc: un panou cu cei patru agenți și cele șase sarcini, cu evidențierea celei active, mesajele agenților și ale pacientului, indicatoarele instrumentelor în execuție cu argumentele și rezultatele lor, și un panou pliabil cu raționamentul fiecărei sarcini. Întrebările agenților sunt afișate cu un widget pentru tipul lor (text, număr, tensiune arterială, ritm, da/nu, întreg sau formular de intrare), folosind indiciul de tip transmis în eveniment. Rezultatele structurate ale instrumentelor sunt prezentate prin carduri specializate. De exemplu, există un indicator vizual pentru nivelul ESI, o reprezentare a deciziei de internare sau externare, bare pentru primele trei diagnostice și un panou de comparație între evaluarea inițială și cea de după colectarea datelor. Punctul de control al intrării, descris în secțiunea 5.3, apare aici ca un formular editabil înaintea triajului.

Pe lângă modul de execuție conversațional, este păstrată și o cale neinteractivă, care este structurată pe etape și implementată în modulul **pipeline.py**. Ea apelează direct instrumentele și expune câte un endpoint pentru fiecare etapă. Aceasta este folosită de testul de paritate **test_parity.py**, care verifică faptul că rezultatele căii directe sunt identice cu cele produse de echipaj, susținând cerința de reproductibilitate din obiectivul O7. Structura de ansamblu a interfeței web, cu legătura dintre browser, serverul FastAPI și firul de execuție al echipajului, este prezentată în Figura 5.5. În capitolul de manual de instalare și utilizare este descrisă utilizarea interfeței din perspectiva utilizatorului, însoțită de capturi de ecran.

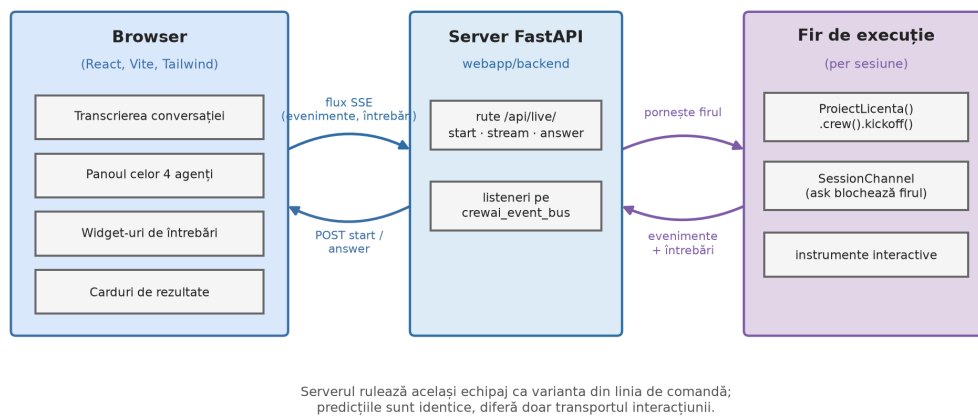


Figura 5.5: Structura interfeței web: browserul comunică cu serverul FastAPI prin SSE și POST, iar serverul rulează același echipaj de agenți ca varianta din linia de comandă.

Capitolul 6. Testare și validare

Capitolul de față arată cum se comportă sistemul descris în Capitolul 5 atunci când este pus în fața datelor de test. Fiecare model din lanțul de agenți a fost evaluat separat, pe seturi de test ținute deoparte de la antrenare, iar apoi întregul lanț a fost rulat cap la cap pe cazuri exprimate în limbaj natural, ca să se măsoare cât pierde sistemul atunci când primește un text liber în locul unui vector de caracteristici deja structurat. Metricile folosite (acuratețe pe primele k clase, kappa ponderat pentru variabila ordinală de acuitate, eroare de calibrare, rate de sub- și supra-triaj) au fost definite teoretic în Capitolul 4. Aici sunt aplicate și interpretate, nu redefinite. Toate cifrele provin dintr-o singură rulare de audit a modelelor v3 aflate în producție, astfel încât rezultatele să fie reproductibile și urmărite cu ușurință, având la bază un artefact concret.

6.1. Metodologia de evaluare

Datele provin din *MIMIC-IV-ED* [4], o bază publică ce conține aproximativ 425.000 de prezentări reale la urgență. După eliminarea intrărilor fără nivel de acuitate sau fără motiv al prezentării, rămân 418.083 de cazuri utilizabile. Fiecare model își construiește propriul set de test printr-o împărțire 80/20 stratificată, cu `random_state=42`, astfel încât aceeași rulare să poată fi reprodusă identic. Stratificarea se face pe ținta relevantă pentru model: pe acuitate la triaj, pe grupul de diagnostic la modelele doctorului și pe variabila de internare la modelul de dispoziție. Dimensiunile seturilor sunt rezumate în Tabelul 6.1.

Tabela 6.1: Dimensiunile seturilor de antrenare și de test pentru fiecare model evaluat. Modelele v1 și v2 ale doctorului folosesc un eșantion plafonat la 100.000 de internări și un spațiu cu 14 clase. Modelele v3 folosesc întreaga populație de internări, din care este exclusă clasa reziduală.

Model	Antrenare	Test	Stratificare
Triaj v3 (acuitate + dispoziție)	334.466	83.617	acuitate
Doctor v3 (diagnostic + departament)	81.680	20.420	grup de diagnostic
Doctor v3 dispoziție (calibrat)	334.466	83.617	internare
Doctor v1/v2 (14 clase, referință)	≈80.000	≈20.000	grup de diagnostic

Modelul de dispoziție al doctorului rezervă în plus 10% din porțiunea de antrenare (aproximativ 33.447 de cazuri) pentru calibrarea izotonică a probabilităților, discutată în secțiunea 4.6.5, astfel încât calibrarea să nu fie măsurată pe date văzute la potrivirea arborilor.

Înainte de a analiza rezultatele, este important să clarificăm un aspect metodologic, pentru a evita interpretarea greșită a cifrelor prezentate în continuare. Atât evaluarea tabelară, cât și cea cap la cap rulează lanțul de predicție în cascadă pe ieșirile care au fost prezise de modelele din etapele anterioare, nu pe valorile reale. Modelul de dispoziție al triajului primește acuitatea prezisă de modelul de acuitate, iar modelul de departament al doctorului primește diagnosticul prezis de modelul de diagnostic. Nu există, așadar, o

etapă în care unui model să i se ofere răspunsul corect al etapei precedente. De exemplu, acuratețea de 60,66% a departamentului ține cont deja de erorile modelului de diagnostic. Singurul loc unde o metrică „presupune răspunsul anterior corect” este raportată explicit este determinarea codului ICD, unde metrica oracol (secțiunea 6.3.5) este condiționată de categoria de diagnostic reală. Acest lucru este făcut tocmai pentru a separa calitatea rezolverului de calitatea cascadei.

Evaluarea se desfășoară pe două planuri. Primul este evaluarea tabelară. Aici, fiecare model primește vectorul de caracteristici construit direct din câmpurile structurate ale bazei de date, pe zeci de mii de cazuri de test. Al doilea este evaluarea cap la cap pe 250 de cazuri sintetice exprimate în limbaj natural (secțiunea 6.4), care trec prin întregul lanț de agenți, inclusiv prin parserul LLM. La acestea se adaugă comparația celor două backenduri de limbaj (secțiunea 6.5) și un audit al alegerii hiperparametrilor (secțiunea 6.6).

6.2. Evaluarea agentului de triaj

6.2.1. Predicția acuității (ESI 1–5)

Pe cele 83.617 de cazuri de test, modelul de acuitate atinge o acuratețe exactă de 67,55% și clasifică 98,18% dintre pacienți la maxim un nivel distanță de eticheta reală. Deoarece ESI este o scală ordinală, distanța contează mai mult decât potrivirea exactă. Kappa ponderat pătratic este 0,6449, eroarea medie absolută 0,3437, iar aria sub curbă macro (unu-contra-restul) 0,8856. Calibrarea este bună, cu o eroare de calibrare așteptată de 0,0266. Distribuția erorilor pe cele cinci niveluri este prezentată în Figura 6.1. Precizia, sensibilitatea și scorul F1 pe fiecare clasă sunt raportate în Figura 6.2.

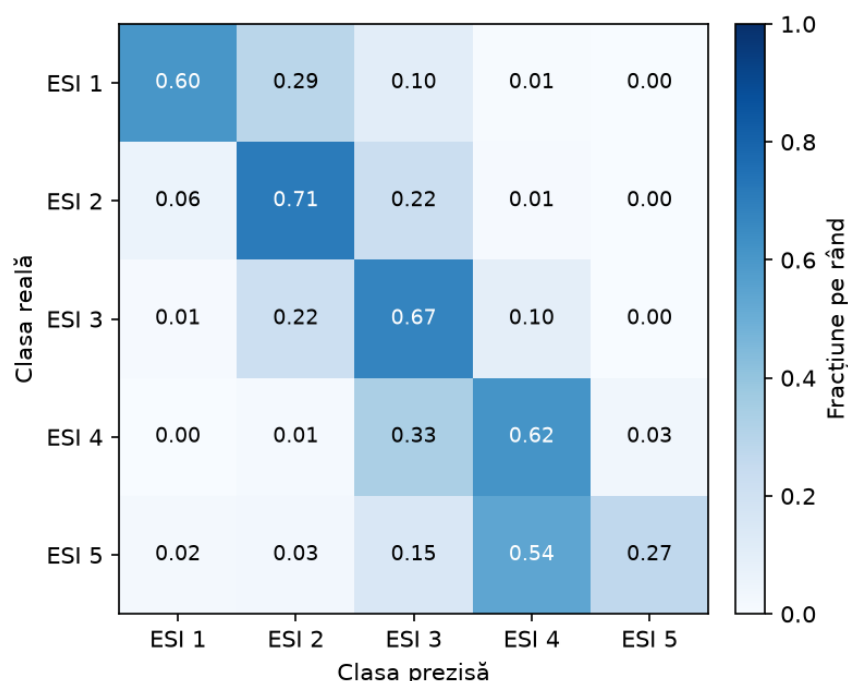


Figura 6.1: Matricea de confuzie normalizată pe rânduri pentru modelul de acuitate. Masa se concentrează pe diagonală și pe vecinii imediați, ceea ce explică acuratețea de 98,18% la toleranță de un nivel.

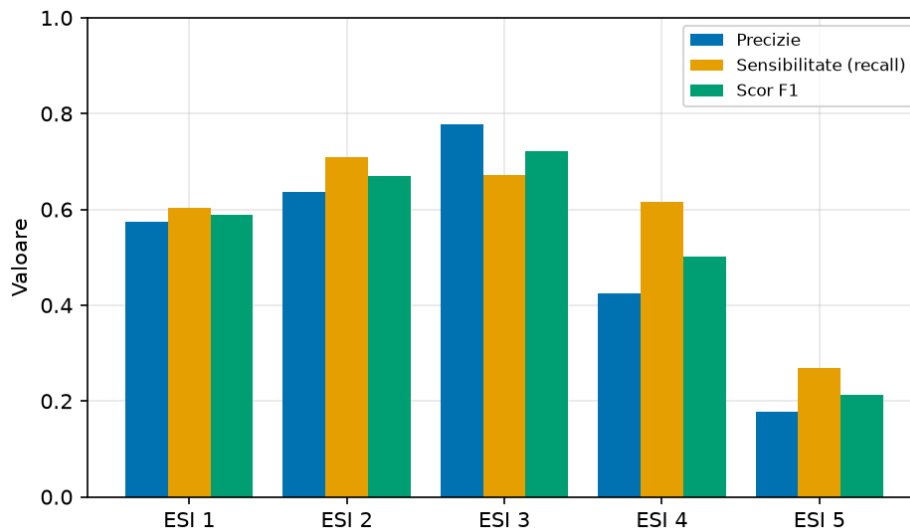


Figura 6.2: Precizie, sensibilitate și scor F1 pe fiecare nivel de acuitate. Clasele frecvente (ESI 2 și ESI 3) sunt separate bine; ESI 5, cu doar 220 de cazuri în test, rămâne clasa cea mai greu de recunoscut.

Rezultatele obținute pentru fiecare clasă reflectă distribuția reală din UPU. Nivelurile ESI 2 și ESI 3 acoperă împreună mai mult de trei sferturi din cazuri și au sensibilități de 70,83% și 67,23%. Clasa ESI 5, cu doar 220 de cazuri în test, rămâne cea mai greu de recunoscut (sensibilitate 26,82%, scor F1 0,214). Clasa extremă ESI 1 este recunoscută în 60,30% din cazuri. Din perspectiva siguranței clinice contează asimetria erorilor. Rata de sub-triaj (a atribui unui pacient un nivel mai puțin urgent decât cel real) este 15,56%, iar cea de supra-triaj 16,90%. Pentru pacienții cei mai gravi, sub-triajul critic (cazuri ESI 1 clasate ca ESI 3 sau mai jos) este 21,48% din cazurile ESI 1, un rezultat pe care iterarea păstrată a modelului l-a îmbunătățit intenționat față de variantele anterioare.

Mascarea semnelor vitale, care a fost motivată în secțiunea 5.4, produce o diferență vizibilă în funcție de modul de sosire. Pentru pacienții aduși cu ambulanța sau elicopterul, unde semnele vitale de la fața locului sunt disponibile, fiind colectate în timpul transportului, acuratețea exactă crește la 69,34%, cu un sub-triaj de doar 9,89%. Pentru pacienții care se prezintă pe cont propriu, unde vitalele lor sunt mascate ca lipsă, acuratețea este 66,63%, cu un sub-triaj mai ridicat, de 19,01%. Diferența confirmă că modelul beneficiază de semnele vitale atunci când le are, exact cum era de așteptat.

6.2.2. Predicția dispoziției (internare / externare)

Modelul binar de dispoziție al triajului estimează probabilitatea de internare pornind de la aceleași caracteristici, folosind în plus acuitatea prezisă anterior. Pe setul de test, aria sub curba ROC este 0,8644, cu o eroare de calibrare de 0,0746 la prevalența reală de internare de 36,71%. Pragul deplasează echilibrul dintre cele două tipuri de eroare, așa cum arată Tabelul 6.2. La pragul de operare 0,50, acuratețea este 77,98%, cu sub-triaj și supra-triaj aproape egale (22,37% și 21,82%). Coborârea pragului la 0,40 crește sensibilitatea la 85,10% și taie sub-triajul la 14,90%, consecința fiind o rată de supra-triaj mai mare. Această predicție este doar o primă estimare. Rafinarea ei se face de către modelul dedicat al doctorului, evaluat în secțiunea 6.3.3.

Tabela 6.2: Modelul de dispoziție al triajului la câteva praguri de decizie. Coborârea pragului reduce sub-triajul (pacienți care ar trebui internați și sunt trimiși acasă) cu prețul creșterii supra-triajului.

Prag	Acuratețe	Sensibilitate	Specificitate	Sub-triaj	Supra-triaj
0,15	59,67%	97,18%	37,91%	2,82%	62,09%
0,20	64,33%	95,63%	46,18%	4,37%	53,82%
0,40	75,35%	85,10%	69,70%	14,90%	30,30%
0,50	77,98%	77,63%	78,18%	22,37%	21,82%
0,75	77,96%	51,52%	93,30%	48,48%	6,70%

6.2.3. Evoluția pe versiuni a triajului

Modelele v1, v2 și v3 ale triajului au fost evaluate pe același set de test și pe același spațiu de clase (cinci niveluri de acuitate, două clase de dispoziție), așa că se pot compara direct. Tabelul 6.3 urmărește această evoluție. Trecerea de la v1 la v2 a adus vectorizarea TF-IDF a motivului prezentării și semnele vitale pentru pacienții aduși cu ambulanța, ridicând acuratețea exactă de la 66,7% la 68,0%. La prima vedere, iterația păstrată a lui v3 pare o regresie, deoarece acuratețea exactă scade cu 0,46 puncte procentuale, la 67,55%. Cifra ascunde însă un compromis asumat. Ponderarea ordinală a claselor extreme și antrenarea mai lungă au coborât rata de sub-triaj de la 17,67% la 15,56% și au recuperat sensibilitatea pe ESI 5, de la 21,8% la 26,82%. Într-un context clinic, ratarea unui pacient grav este eroarea costisitoare sau chiar fatală. Din acest motiv, schimbul de 0,46 puncte de acuratețe brută pentru două puncte de sub-triaj și cinci puncte de sensibilitate pe clasa cea mai vulnerabilă este justificat.

Tabela 6.3: Evoluția triajului pe versiuni, pe același set de test de 83.617 cazuri. Valorile lipsă nu au fost raportate pentru versiunea respectivă.

Metrică	v1	v2	v3
Acuitate, acuratețe exactă	66,7%	68,0%	67,55%
Acuitate, la un nivel distanță	97,7%	98,0%	98,18%
Acuitate, kappa ponderat pătratic	—	0,638	0,645
Acuitate, rată de sub-triaj	—	17,67%	15,56%
Acuitate, sensibilitate ESI 5	—	21,8%	26,82%
Dispoziție, acuratețe	75,9%	76,2%	77,98%
Dispoziție, arie sub curba ROC	0,84	0,84	0,864

6.3. Evaluarea agentului doctor

Doctorul rulează de trei ori în parcursul lanțului, așa cum s-a arătat în Tabelul 5.5. O evaluare inițială pe baza datelor de triaj, o rafinare a dispoziției și o reevaluare după ce asistentul colectează semnele vitale, medicația și antecedentele. Împărțirea între evaluarea inițială și cea de după asistent este o decizie de proiectare, nu o etapă tehnică accidentală. Ea permite măsurarea directă valorii pe care o adaugă datele colectate de asistent. Modelele inițiale sunt notate v3_base, iar cele de după asistent v3.

6.3.1. Diagnosticul (13 categorii)

Diagnosticul este o problemă cu 13 categorii, obținute prin gruparea codurilor ICD așa cum s-a descris în Tabelul 5.3. Modelul inițial atinge o acuratețe top-1 de 60,09% și top-5 de 90,56%; după colectarea datelor de către asistent, acuratețea top-1 urcă la 64,07%, iar top-5 la 92,77%. Câștigul de 3,99 puncte procentuale pe top-1 vine din două surse: datele propriu-zise ale asistentului (semne vitale, medicație, ritm și traiectorii vitale longitudinale) și cele 19 caracteristici de antecedente medicale. Îmbunătățirea nu este doar de acuratețe brută: scorul F1 macro crește de la 0,516 la 0,553, iar aria sub curbă macro de la 0,889 la 0,912. Aceste lucruri ne arată că progresul se distribuie și pe clasele mai rare, nu doar pe cele frecvente. Ambele modele sunt bine calibrate, cu eroarea de calibrare scăzând de la 0,0191 la 0,0122. Rezultatele corespunzătoare celor două capete de predicție ale doctorului sunt sintetizate în Tabelul 6.4 și ilustrate grafic în Figura 6.3.

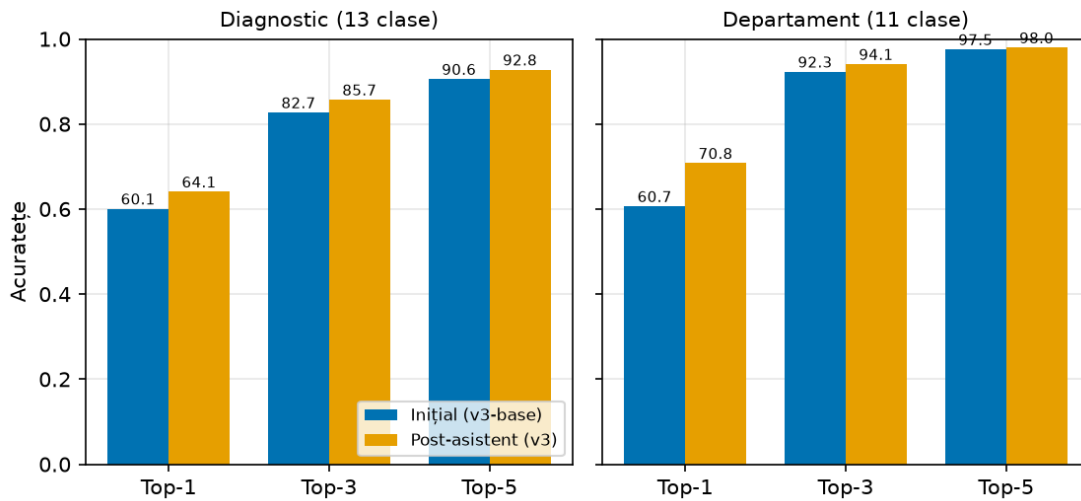


Figura 6.3: Acuratețea top-1/top-3/top-5 a doctorului la diagnostic și departament, înainte (v3_base) și după (v3) colectarea datelor de către asistent. Câștigul este vizibil mai ales pe top-1 la departament.

Tabela 6.4: Sinteza metricilor doctorului pe cele două capete de predicție, inițial (v3_base) și după asistent (v3), pe cele 20.420 de cazuri de test.

Metrică	Diagnostic (13 clase)		Departament (11 clase)	
	v3_base	v3	v3_base	v3
Acuratețe top-1	60,09%	64,07%	60,66%	70,79%
Acuratețe top-3	82,71%	85,72%	92,28%	94,08%
Acuratețe top-5	90,56%	92,77%	97,55%	97,97%
Acuratețe echilibrată	52,25%	54,25%	49,58%	45,19%
Scor F1 macro	0,516	0,553	0,445	0,490
Kappa Cohen	0,549	0,591	0,422	0,495
Eroare de calibrare	0,019	0,012	0,020	0,019

6.3.2. Departamentul (11 servicii)

Determinarea departamentului la care trebuie trimiși pacienții arată cel mai mare câștig de pe urma colectării datelor de către asistent, dar și un compromis care trebuie declarat onest. Acuratețea top-1 crește de la 60,66% la 70,79%, un plus de 10,13 puncte procentuale. Aria sub curbă macro crește de la 0,893 la 0,922. Cu toate acestea, acuratețea echilibrată *scade* de la 49,58% la 45,19%. Explicația este în Tabelul 6.5. Aproape tot câștigul se concentrează pe clasa majoritară, Medicina Generală, a cărei sensibilitate urcă de la 65,41% la 87,26% pe cele 12.045 de cazuri (59% din setul de test). În contrast, mai multe departamente minoritare regresează. Chirurgia Generală scade de la 56,05% la 36,48%, categoria „Alte specialități” de la 23,90% la 8,18%, iar Neurochirurgia de la 41,86% la 34,69%. Informațiile furnizate de către agentul asistent fac modelul mai încrezător în predicția dominantă, ceea ce ajută acuratețea globală, dar reduce capacitatea de a recunoaște clasele rare. Rutarea către departamente chirurgicale depinde de natura leziunii, nu de semnele vitale, iar adăugarea semnelor vitale poate duce la confuzii între departamentele chirurgicale.

Tabela 6.5: Sensibilitatea pe departament, înainte și după colectarea datelor de către asistent. Câștigul este concentrat pe clasa majoritară (Medicina Generală); mai multe clase minoritare regresează, ceea ce explică scăderea acurateței echilibrate.

Departament	Cazuri	v3_base	v3	Δ
Medicină Generală	12.045	65,41%	87,26%	+21,8
Medicină Cardiacă	1.683	66,19%	61,68%	-4,5
Chirurgie Generală	1.472	56,05%	36,48%	-19,6
Medicină Neurologică	1.226	69,49%	70,23%	+0,7
Ortopedie	1.045	65,36%	58,56%	-6,8
Medicină Oncologică	902	14,86%	16,19%	+1,3
Neurochirurgie	614	41,86%	34,69%	-7,2
Chirurgie diversă	540	36,48%	29,81%	-6,7
Traumatologie	509	59,53%	54,03%	-5,5
Obstetrică / Ginecologie	225	46,22%	40,00%	-6,2
Alte specialități	159	23,90%	8,18%	-15,7

6.3.3. Rafinarea dispoziției de către doctor

Modelul dedicat de dispoziție al doctorului, calibrat izotonic (secțiunea 5.5.2), este net superior primei estimări din triaj. Aria sub curba ROC este 0,9138, față de 0,8644 la triaj, iar calibrarea este aproape perfectă: eroarea de calibrare așteptată scade de la 0,0746 la 0,0036. Figura 6.4 compară cele două curbe de fiabilitate una lângă alta. Curba doctorului urmează diagonală, în timp ce cea a triajului subestimează sistematic probabilitatea de internare. Pragul de decizie folosit în producție este 0,40, nu 0,50, iar Tabelul 6.6 arată de ce. La 0,50 acuratețea este ușor mai mare (83,96% față de 83,33%), dar sub-triajul urcă la 23,51%. Pragul de 0,40 coboară sub-triajul la 18,14% și are un cost de mai puțin de un punct de acuratețe. Acest lucru îl face alegerea potrivită într-un context în care externarea greșită a unui pacient care avea nevoie de internare este eroarea mai gravă. Evoluția performanței în funcție de pragul ales este prezentată în Figura 6.5.

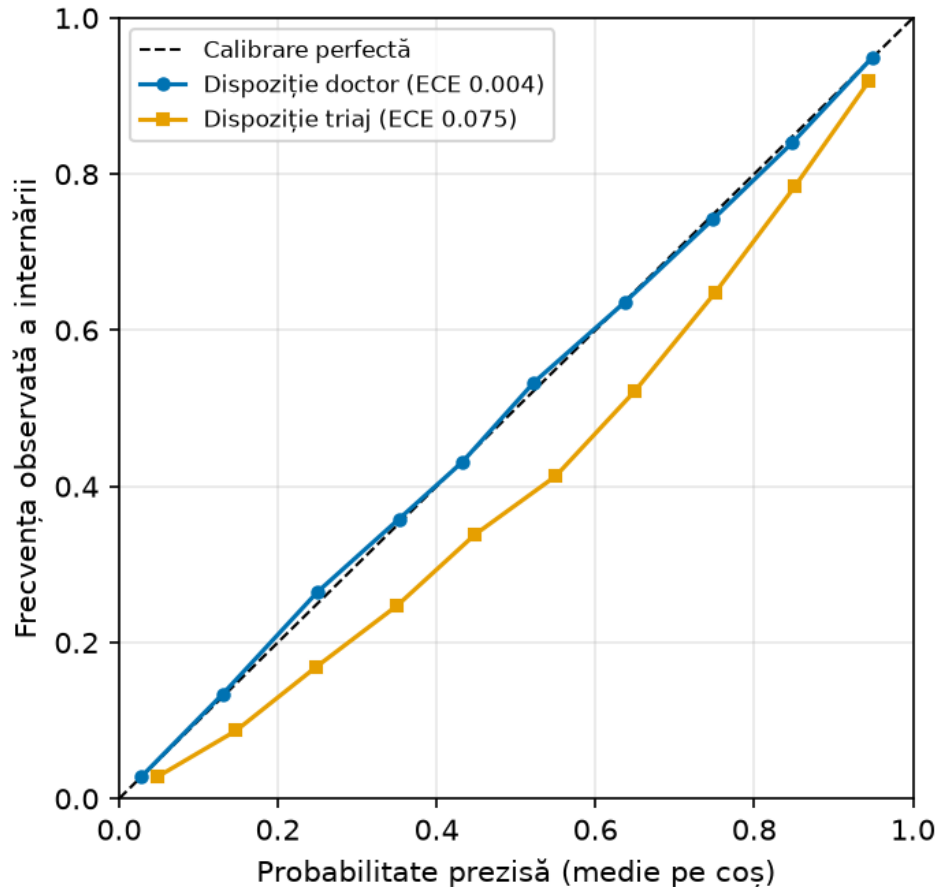


Figura 6.4: Curbele de fiabilitate pentru dispoziția doctorului (calibrată izotonic) și pentru estimarea din triaj. Curba doctorului urmează aproape exact diagonala calibrării perfecte.

6.3.4. Robustețea dispoziției pe subgrupuri clinice

O medie bună pe întregul set de test poate ascunde comportamente diferite pe categorii de pacienți, așa că modelul de dispoziție a fost evaluat separat pe câteva subgrupuri cu relevanță clinică. Tabelul 6.7 compară pentru fiecare subgrup acuratețea estimării din etapa de triaj cu cea a modelului doctorului. Ambele sunt comparate la pragul de producție 0,40. Modelul doctorului este mai bun pe fiecare subgrup, iar câștigul este cel mai mare tocmai la pacienții complicați, cum ar fi vârstnici (plus 7,70 puncte procentuale) și pacienți cu internări anterioare (plus 7,41 puncte), unde decizia de internare este și cea mai delicată. Chiar și pe subgrupul mic cu ritm cardiac non-sinusal, unde estimarea din triaj era deja bună, modelul câștigă în plus încă 3,32 puncte. Aria sub curbă se menține peste 0,89 pentru toate subgrupurile. Faptul că îmbunătățirea apare pe fiecare categorie, nu doar în medie, argumentează faptul că modelul este unul solid.

Analiza importanței caracteristicilor pentru același model explică de unde vine cea mai mare parte a semnalului. Tabelul 6.8 arată câștigul total al arborilor XGBoost, grupat pe familii de caracteristici. Textul motivului prezentării, vectorizat TF-IDF, domină categoric, cu 84,6% din câștig. Semnele vitale longitudinale, caracteristicile structurate (vârstă, mod de sosire) și antecedentele medicale adună împreună restul, fiecare cu contribuții de câteva procente. Rezultatul confirmă observația din secțiunea 6.4.

Tabela 6.6: Modelul de dispoziție al doctorului la câteva praguri. Pragul de producție 0,40 reduce sub-triajul față de 0,50, cu un cost minim de acuratețe.

Prag	Acuratețe	Sensibilitate	Specificitate	Sub-triaj	Supra-triaj
0,15	75,01%	94,62%	63,62%	5,38%	36,38%
0,20	78,15%	92,38%	69,89%	7,62%	30,11%
0,40	83,33%	81,86%	84,19%	18,14%	15,81%
0,50	83,96%	76,49%	88,30%	23,51%	11,70%

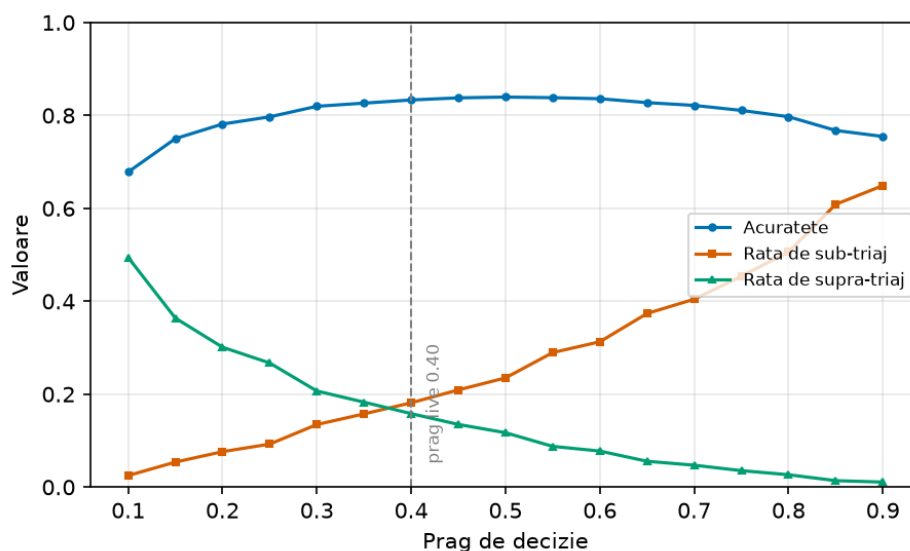


Figura 6.5: Acuratețea și ratele de sub-triaj și supra-triaj ale dispoziției doctorului în funcție de prag. Cele două rate de eroare se intersectează în jurul pragului de producție 0,40.

Pentru că modelul se bazează în principal pe limbajul natural al motivului prezentării, datele colectate de asistent aduc un aport real, dar limitat. Reformularea textului de către parser rămâne cea mai costisitoare parte a întregului lanț.

6.3.5. Recuperarea codului ICD exact

Etapa a doua, descrisă în secțiunea 5.5.3, caută codul ICD exact în interiorul categoriei prezise, ordonând codurile candidate după scorul de amestec definit în secțiunea 4.6.6 (similaritate de text, profil fiziologic, antecedente condiționate și prevalență). Înainte de rezultate, câțiva termeni de evaluare au nevoie de o definiție, pentru că nu apar în capitolele anterioare.

Evaluarea este realizată la nivel de *rollup*, ceea ce înseamnă că fiecare cod ICD complet (de exemplu, *I50.9*) este redus la primele trei caractere (*I50*), care reprezintă categoria clinică principală și este considerată cea mai relevantă din punct de vedere medical. Codul ICD complet este raportat doar ca nivel suplimentar de detaliu.

Rezultatele obținute în varianta *oracol* pornesc de la categoria de diagnostic reală, ceea ce permite estimarea performanței maxime pe care o poate atinge rezolverul în condiții ideale. În schimb, evaluarea **cap la cap** utilizează categoria prezisă de modelul de diagnostic și reflectă performanța reală a întregului lanț de procesare.

De asemenea, este raportată și metrica de **uniune**, care ia în considerare candidații proveniți din primele cinci categorii de diagnostic prezise de model, nu doar din categoria

Tabela 6.7: Acuratețea dispoziției pe subgrupuri clinice, la pragul de producție 0,40: estimarea din triaj față de modelul doctorului. Modelul câștigă la fiecare subgrup, cel mai mult la pacienții în vârstă și la cei cu internări anterioare.

Subgrup	Cazuri	Triaj	Doctor	Δ	AUC doctor
Vârstnici (≥ 65 ani)	26.349	71,91%	79,61%	+7,70	0,886
Polipragmazie (≥ 5 medica- mente)	41.856	72,79%	79,59%	+6,80	0,888
Semne vitale anormale (≥ 1 semnal)	18.702	78,71%	83,35%	+4,64	0,920
Vizitator repetat (UPU ante- rior)	43.923	77,13%	83,52%	+6,39	0,919
Internare anterioară	43.137	74,16%	81,57%	+7,41	0,906
Ritm non-sinusal	1.445	79,17%	82,49%	+3,32	0,893

Tabela 6.8: Importanța caracteristicilor pentru modelul de dispoziție al doctorului, ca procent din câștigul total al arborilor, grupată pe familii. Textul motivului prezentării domină, ceea ce explică de ce aportul datelor de asistent este real, dar limitat.

Familie de caracteristici	Pondere din câștig
Text TF-IDF (motivul prezentării)	84,6%
Semne vitale longitudinale	4,6%
Caracteristici structurate (vârstă, mod de sosire)	4,1%
Antecedente medicale	1,9%
Cascadă soft (probabilități din amonte)	1,9%
Semne vitale instantanee	1,7%
Medicație	1,2%

aflată pe primul loc. Această abordare oferă o imagine mai completă asupra capacității sistemului de a include diagnosticul corect printre variantele cele mai probabile.

În evaluarea **strictă**, o predicție este considerată corectă doar dacă exact codul ICD real se regăsește printre primele k variante propuse de sistem. Totuși, în practică, două coduri care diferă doar printr-o cifră pot descrie afecțiuni foarte apropiate și pot fi considerate acceptabile din punct de vedere clinic. Din acest motiv, pe lângă evaluarea strictă, sunt calculate și metrici **gradate**, care oferă un credit parțial atunci când codul prezis este apropiat de cel corect.

Gradul de apropiere este determinat prin compararea **titlurilor** asociate celor două coduri ICD, și nu prin compararea textului pacientului cu descrierea unui cod. Pentru această evaluare sunt utilizate cele trei metode prezentate în Tabelul 6.9. Compararea titlu cu titlu are avantajul că o potrivire exactă primește întotdeauna scorul maxim, iar codurile similare sunt evaluate proporțional cu gradul lor de asemănare.

Este important de făcut distincția între aceste metode de evaluare și scorul TF-IDF utilizat de rezolverul descris în Secțiunea 4.6.6. În etapa de rezolvare, TF-IDF compară descrierea pacientului cu prototipurile asociate fiecărui cod pentru a selecta diagnosticul cel mai probabil. În schimb, în etapa de evaluare, metricile gradate nu aleg un cod, ci măsoară cât de apropiată este predicția sistemului de diagnosticul corect.

Tabelul 6.10 prezintă rezultatele obținute la nivelul de granularitate **rollup**, utilizând varianta de scor implementată în versiunea de producție. Aceasta se bazează pe combinația de caracteristici folosită de rezolver, la care sunt adăugate și informațiile pri-

Tabela 6.9: Cele trei motoare de similaritate gradată folosite la evaluarea codului ICD. Toate compară titlul codului prezis cu titlul codului real, astfel încât o potrivire exactă primește scorul maxim, iar un cod apropiat clinic primește credit parțial.

Motor	Ce măsoară
TF-IDF de titlu	Cosinusul dintre vectorii TF-IDF ai celor două titluri de cod, la nivel de cuvinte (asemănare lexicală).
Gemini de titlu	Cosinusul dintre reprezentările semantice ale titlurilor, produse de modelul de embedding gemini-embedding-001 (asemănare de sens, nu doar de cuvinte).
Arbore ICD	Creditul dat după poziția celor două coduri în ierarhia ICD: cu cât prefixul comun este mai lung, cu atât codurile sunt mai apropiate.

vind semnele vitale ale pacientului, motiv pentru care este denumită **amestec + vitale**.

Identificarea codului ICD exact rămâne o sarcină dificilă. În evaluarea de tip oracol, recall-ul strict pentru prima poziție este 33,12%, iar recall-ul pentru primele cinci poziții prezise 69,13%. Faptul că rezolverul reglat ridică recall-ul strict de la 18,34% (varianta bazată doar pe prevalență) la 33,12% indică faptul că utilizarea similarității față de prototipuri contribuie în mod semnificativ la îmbunătățirea performanței. Din perspectiva utilității clinice, sunt mai relevante metricile gradate. Acestea acordă punctaj parțial codurilor apropiate, după cum este descris în Secțiunea 4.6.6). Valorile obținute pentru aceste metrici sunt considerabil mai ridicate: similaritatea semantică, calculată cu Gemini (Gemini Embeddings), ajunge la 91,47% pentru primele cinci coduri, iar distanța în arborele ICD la 78,37%. Figura 6.6 face o comparație între metrica strictă și metricile gradate. În iterațiile cap la cap (end-to-end), prin modelul de după asistent, uniunea candidaților atinge 64,88% la granularitatea de tip rollup, iar varianta gradată cu Gemini 90,38%. Câștigul adus de datele colectate de asistent la această etapă este mic (aproximativ un punct procentual). Acest lucru se datorează faptului că recuperarea codului depinde în primul rând de categoria prezisă corect în prima etapă.

Tabela 6.10: Rezolvarea codului ICD exact, granularitate rollup pe trei caractere, varianta amestec plus vitale. Oracol condiționează pe categoria reală; cap la cap condiționează pe categoria prezisă de modelul de după asistent.

Metrică	Oracol	Cap la cap (v3)
Strict, prima poziție	33,12%	–
Strict, primele cinci	69,13%	–
Uniune candidați	–	64,88%
Gradată TF-IDF, primele cinci	72,57%	–
Gradată Gemini, primele cinci	91,47%	90,38%
Gradată arbore ICD, primele cinci	78,37%	–
Recall categorie (etapa 1), primele cinci	–	92,77%

6.3.6. Evoluția pe versiuni a doctorului

Comparația dintre versiunile doctorului trebuie analizată separat, pe două piste diferite, pentru că spațiul de clase s-a schimbat de la o versiune la alta. Modelele v1 și v2 folosesc 14 categorii de diagnostic. Acestea includ o clasă reziduală de simptome nespeci-

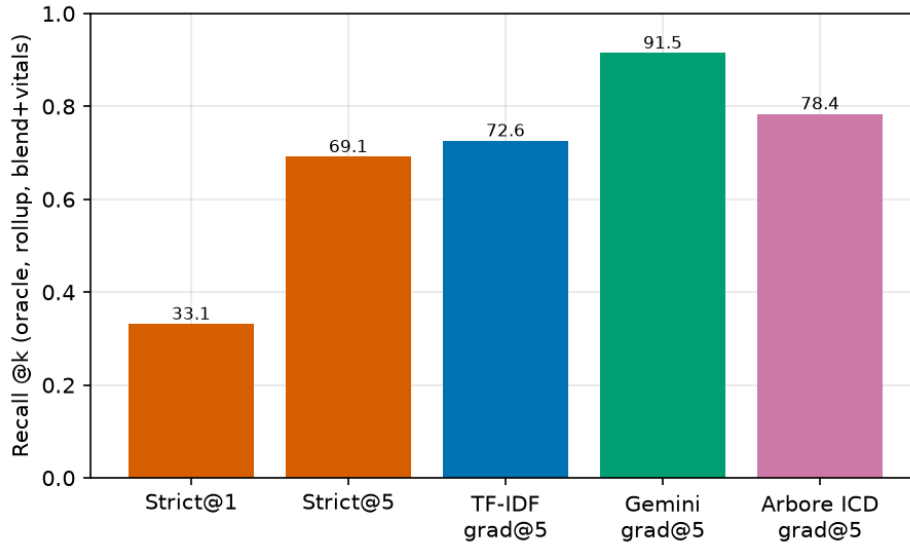


Figura 6.6: Rezolvarea codului ICD (oracol, rollup). Potrivirea strictă a codului este grea, dar metricile gradate, care creditează codurile apropiate clinic, sunt mult mai ridicate, mai ales cea bazată pe similaritatea de titlu.

fică (Symptoms-Signs-Ill Defined). Modelele v3 exclud această clasă reziduală, pentru că aceasta conține aproximativ o treime din internări cu coduri ambigue, care repetă motivul prezentării în loc să indice o cauză exactă. Astfel, modelele v3 rămân cu 13 categorii finale. Cele două piste nu se pot uni, pentru că nu măsoară aceeași sarcină. Tabelul 6.11 le prezintă separat. Pe pista cu 14 clase, trecerea de la v1 la v2 (adăugarea datelor de asistent) a adus 2,3 puncte la diagnostic și 5,9 puncte la departament. Pe pista cu 13 clase, adăugarea aceluiași date plus antecedentele aduce 3,99 puncte la diagnostic și 10,13 puncte la departament. Diferența dintre piste vine în primul rând din excluderea clasei reziduale și din antrenarea pe întreaga populație de internări, nu doar din caracteristicile de asistent.

Tabela 6.11: Evoluția doctorului pe versiuni, pe două piste care nu sunt direct comparabile: v1/v2 pe 14 clase (cu clasă reziduală), v3 pe 13 clase (fără clasă reziduală). Fiecare pistă compară modelul inițial cu cel de după asistent.

Acuratețe top-1	14 clase		13 clase	
	v1	v2	v3_base	v3
Diagnostic	50,2%	52,4%	60,1%	64,1%
Departament	59,1%	65,0%	60,7%	70,8%

6.4. Validarea cap la cap pe cazuri în limbaj natural

Evaluarea tabelară testează modelele pe vectori de caracteristici deja structurați. Atunci când este folosit în lumea reală, sistemul primește un text în limbaj natural, care descrie pacientul. Parserul LLM trebuie să extragă din acest text câmpurile pe care modelele le primesc ca intrări. Pentru a măsura acest cost, au fost generate 250 de cazuri sintetice în limbaj natural (dintre care 163 de internări). Fiecare dintre aceste cazuri este bazat pe o prezentare reală din baza de date, iar ele au fost trecute prin fluxul complet. Aceleași cazuri au fost evaluate în patru moduri, ceea ce a permis analizarea separată a

surselor de eroare (pierderile existente), rezumată în Tabelul 6.12.

Interpretarea rezultatelor pornește de la faptul că toate cele patru moduri utilizează aceeași cascadă de procesare, diferența dintre ele fiind doar sursa caracteristicilor de intrare. Rezultatele obținute folosind modul cu vector de caracteristici filtrat și modul cu apel direct al instrumentelor pe câmpurile tabelare exacte sunt apropiate (de exemplu, 47,2% și 49,1% la diagnostic top-1). Acest aspect arată că poarta de dispoziție și degradarea caracteristicilor în timpul execuției programului (la runtime) nu costă mai nimic. Prin urmare, aproape toată scăderea până la modul cu parser LLM este costul parserului. Pentru Gemini Flash, diagnosticul top-1 pierde 6,7 puncte față de apelul direct (apelarea instrumentelor în mod direct, cu argumentele extrase direct din datele tabulare), iar departamentul top-1 pierde 11,7 puncte. Alegerea departamentului de internare este cea mai sensibilă la reformularea în limbaj natural, pentru că depinde de termeni specifici din motivul prezentării. Parserul nu poate reconstrui mereu cu exactitate acești termeni. Diferențele observate nu sunt cauzate de o evaluare mai favorabilă a cascadei pe datele tabelare, ci de modul în care informațiile sunt furnizate ca date de intrare.

Tabela 6.12: Descompunerea performanței cap la cap pe cele 250 de cazuri sintetice (diagnostic, departament și ICD pe cele 163 de internări). Vectorul filtrat și apelul direct sunt apropiate, deci restul până la modul cu parser este costul parserului LLM.

Metrică	Vector filtrat	Apel direct	Parser Flash	Parser MedGemma
Acuitate, acuratețe exactă	68,4%	68,0%	66,0%	60,4%
Dispoziție, acuratețe la 0,40	82,0%	81,6%	79,2%	78,4%
Diagnostic top-1	47,2%	49,1%	42,3%	42,3%
Departament top-1	58,9%	59,5%	47,9%	49,7%
ICD, uniune candidați	53,4%	55,2%	46,6%	46,0%

Fidelitatea parserului a fost măsurată și direct, independent de modele. Din cele 250 de cazuri, vârsta a fost extrasă corect în toate, genul în 232 (92,8%), iar modul de transport în 242 (96,8%). Similaritatea Jaccard între motivul prezentării generat și cel extras este 0,598, ceea ce reflectă faptul că parserul rescrie formularea, nu o copiază. Această pierdere de fidelitate a textului se propagă în vectorizarea TF-IDF și explică o parte din costul observat la diagnostic și departament.

6.5. Compararea backendurilor de limbaj

Sistemul poate comuta între două modele de limbaj pentru partea de înțelegere a textului: Gemini Flash 2.5, accesat prin API, și MedGemma, un model medical autogăzduit. Comparăția din secțiunea 6.4 a fost repetată identic cu fiecare backend, pe aceleași 250 de cazuri, iar rezultatul este în Tabelul 6.13 și în Figura 6.7. Flash conduce clar la acuitate (66,0% față de 60,4% acuratețe exactă, cu un kappa ponderat mai bun) și la fidelitatea extragerii textului (Jaccard 0,598 față de 0,529). La dispoziție, Flash are un mic avans (79,2% față de 78,4%). Diagnosticul este la egalitate (42,3% top-1 pentru ambele), iar la departament MedGemma are un avantaj marginal (49,7% față de 47,9% top-1). În principiu, pentru configurația curentă, Flash este alegerea mai bună. MedGemma rămâne totuși o alternativă bună atunci când autogăzduirea este luată în calcul din motive de confidențialitate a datelor sau a costurilor (fiind auto-găzduit, nu necesită apeluri costisitoare la API-uri externe).

Tabela 6.13: Gemini Flash 2.5 vs. MedGemma pe cele 250 de cazuri, în modul cu parser. Flash conduce la acuitate, dispoziție și fidelitatea textului; MedGemma are un avans marginal la departament.

Metrică	Flash 2.5	MedGemma
Acuitate, acuratețe exactă	66,0%	60,4%
Acuitate, kappa ponderat pătratic	0,615	0,539
Dispoziție, acuratețe la 0,40	79,2%	78,4%
Dispoziție, arie sub curba ROC	0,888	0,873
Diagnostic top-1	42,3%	42,3%
Departament top-1	47,9%	49,7%
ICD, strict prima poziție	17,8%	17,2%
Fidelitate simptome (Jaccard)	0,598	0,529

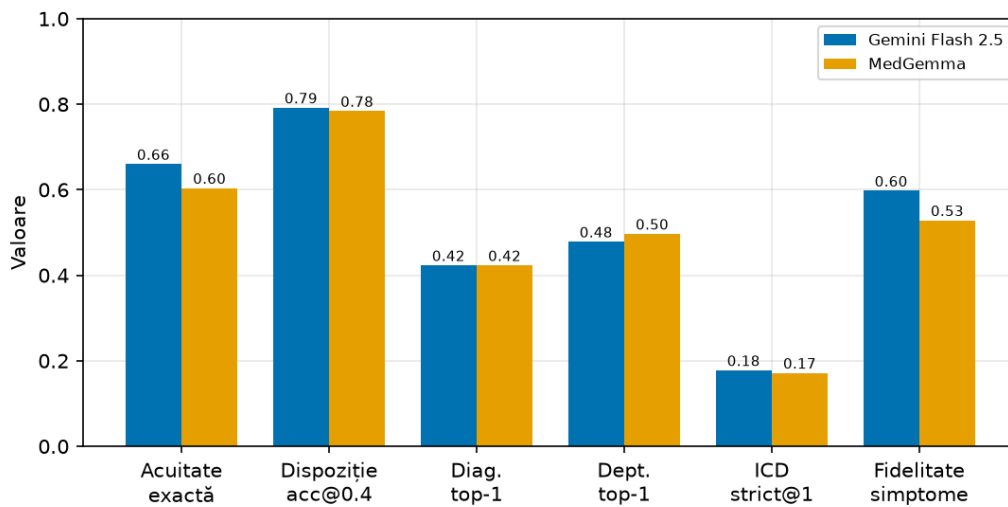


Figura 6.7: Comparația celor două backenduri de limbaj pe metricile comune. Diferențele cele mai mari sunt la acuitate și la fidelitatea extragerii simptomelor, ambele în favoarea Flash.

6.6. Auditul configurației prin optimizarea hiperparametrilor

Hiperparametrii modelelor XGBoost au fost aleși manual în timpul dezvoltării, pe baza testelor și a rezultatelor obținute. Ca să verific dacă aceste alegeri sunt apropiate de cele optime, am rulat o căutare sistematică cu biblioteca Optuna [32], structurată în două grupuri: un grup pentru pârgiile de siguranță clinică (ponderarea claselor extreme, rata de învățare, numărul de arbori) și un grup pentru parametrii de regularizare. Pentru modelele cu obiective ce pot intra în conflict, cum este acuitatea, unde se vrea simultan acuratețe mare și sub-triaj mic, am folosit o căutare multi-obiectiv de tip NSGA-II (Non-dominated Sorting Genetic Algorithm) [33]. Aceasta nu oferă o singură soluție optimă, ci generează un set de variante optime (frontiera Pareto), fiecare reprezentând un compromis diferit între cele două obiective. Trebuie menționat faptul că această analiză a avut rol strict de verificare și validare a alegerilor făcute inițial. Modelele utilizate în aplicația finală au rămas neschimbate.

Concluzia este că valorile alese manual sunt aproape de valorile optime. Tabelul 6.14 rezumă diferențele identificate în urma procesului de optimizare. Pentru acui-

tate, configurația aleasă inițial a fost reselectată pe frontiera Pareto și are cea mai bună sensibilitate ESI 5 dintre toate punctele frontierei. Niciun alt punct nu o domină. Pentru dispoziția triajului, configurația inițială are chiar cea mai bună arie sub curbă din toate încercările. La doctor, capul de departament este un schimb neutru (o pierdere de 0,04 puncte de acuratețe pentru un plus de 0,70 puncte de F1 macro), iar cel de dispoziție o îmbunătățire reală dar mică (0,42 puncte de acuratețe și 1,15 puncte mai puțin sub-triaj). Acestea au fost considerate sub pragul care ar justifica o reantrenare și o implementare a unei noi versiuni în aplicație.

Tabela 6.14: Rezultatele auditului de hiperparametri, exprimate ca diferență față de configurația din producție. Toate rezultatele sunt sub pragul de redistribuire; modelele live nu au fost modificate.

Cap de model	Cea mai bună configurație găsită
Triaj, acuitate	incumbentul reselectat pe frontieră; cea mai bună sensibilitate ESI 5
Triaj, dispoziție	incumbentul are cea mai bună arie sub curbă (0,864)
Doctor, departament	−0,04 p.p. acuratețe, +0,70 p.p. F1 macro (neutru)
Doctor, dispoziție	+0,42 p.p. acuratețe, −1,15 p.p. sub-triaj (sub prag)

6.7. Discuție și limitări

Rezultatele obținute evidențiază atât punctele forte ale sistemului, cât și limitele sale, pe care le recunosc și le prezint în mod transparent. Triajul clasifică acuitatea corect în peste 98% din cazuri la toleranță de un nivel și este orientat spre siguranță, cu un sub-triaj menținut sub 16% și recuperat intenționat pentru clasa cea mai vulnerabilă. Modelul de dispoziție al doctorului este foarte bine calibrat (eroare de calibrare 0,0036) și discriminează internările cu o arie sub curbă de 0,914. Datele colectate de asistent aduc o îmbunătățire semnificativă la diagnostic și la determinarea departamentului de internare a pacientului. Acest lucru justifică decizia de a separa evaluarea inițială de cea de după consultul asistentului.

Limitările sistemului sunt la fel de evidente precum punctele sale forte. Câștigul performanței de la departament este concentrat pe clasa majoritară și vine cu o scădere a acurateței echilibrate. Pentru clasele minoritare, mai ales cele chirurgicale, datele asistentului nu ajută sau chiar încurcă. Identificarea exactă a codului ICD rămâne o provocare. În varianta „oracol”, modelul reușește să prezică exact codul corect pe prima poziție în aproximativ 33% din cazuri. Totuși, dacă sunt utilizate metrici care țin cont de apropierea dintre codurile ICD și sunt mai relevante din punct de vedere clinic, performanța este considerabil mai bună. O altă limitare provine chiar din structura datelor. Aproximativ o treime dintre cazurile internate sunt etichetate cu un cod ICD care descrie doar motivul prezentării pacientului și nu diagnosticul final. Acest aspect introduce o limitare implicită a procesului de învățare și care nu poate fi eliminată doar prin adăugarea unor caracteristici noi. Eliminarea acestei categorii de cazuri în versiunea v3 reprezintă o încercare de a reduce impactul acestei probleme. În ceea ce privește performanța aplicației, transformarea datelor din date structurate în text liber implică un cost suplimentar de procesare. Analiza efectuată arată că acest cost este generat aproape în totalitate de

parserul bazat pe modele lingvistice de mari dimensiuni (LLM), și nu de mecanismul de inferență în cascadă. Din acest motiv, optimizarea parserului reprezintă direcția cu cel mai mare potențial de îmbunătățire, aspect discutat în capitolul dedicat concluziilor și direcțiilor viitoare de dezvoltare.

Capitolul 7. Manual de instalare și utilizare

Acest capitol descrie cum se instalează și cum se folosește sistemul, din perspectiva unui utilizator care vrea să îl ruleze, nu să îl modifice. Detaliile interne ale agenților și ale modelelor au fost tratate în Capitolul 5. Aici ne interesează doar pașii practici. Sistemul poate fi folosit în două moduri, ambele rulând exact același lanț de agenți: din linia de comandă, printr-o conversație în terminal, și prin interfața web conversațională descrisă în secțiunea 5.8, care realizează obiectivul O7. Instalarea este aceeași pentru ambele moduri, cu un singur pas în plus pentru interfața web.

7.1. Cerințe software și hardware

Sistemul este scris în Python și folosește gestionarul de pachete și de mediu `uv`, care instalează versiunea corectă a interpretorului și toate dependențele pornind de la fișierul `pyproject.toml`. Partea de interfață web are în plus un server FastAPI și o aplicație client React, aceasta din urmă construită cu Node.js. Cerințele minime sunt rezumate în Tabelul 7.1.

Tabela 7.1: Cerințele software și hardware pentru instalarea și rularea sistemului. Componentele marcate ca opționale sunt necesare doar pentru interfața web, respectiv pentru reantrenarea modelelor.

Componentă	Cerință
Sistem de operare	Windows, Linux sau macOS
Python	versiunea 3.13, instalată automat de <code>uv</code>
Gestionar de pachete	<code>uv</code> (obligatoriu)
Node.js și <code>npm</code>	necesare doar pentru interfața web (opțional)
Cheie de acces LLM	o cheie <code>GEMINI_API_KEY</code> pentru backendul Gemini Flash, sau un punct de acces MedGemma autogăzduit
Modele antrenate	fișierele <code>.joblib</code> din <code>artifacts/</code> , necesare pentru inferență (obținute prin antrenare sau furnizate separat)
Procesor	orice procesor modern este suficient pentru inferență
Placă grafică	opțională, folositoare doar la reantrenarea modelelor
Memorie	aproximativ 8 GB pentru inferență; reantrenarea, care încarcă baza MIMIC-IV-ED, cere mai mult

Este important de precizat că rularea propriu-zisă a sistemului, adică etapa de inferență, nu are nevoie de baza de date MIMIC-IV-ED și nici de o placă grafică. Fișierele CSV brute și placa grafică sunt necesare doar dacă utilizatorul vrea să reantreneze modelele de la zero, folosind comenzile `uv run train_*` descrise în secțiunea 7.2.4. Pentru utilizarea obișnuită a aplicației este suficient ca modelele deja antrenate să fie disponibile în directorul `artifacts/`, iar utilizatorul să dețină o cheie de acces validă pentru modelul de limbaj.

Dintre componentele care sunt necesare pentru rularea interfeței web, doar Node.js se instalează manual, pentru că nu este un pachet Python și nu poate fi instalat de către `uv`. El se descarcă de pe pagina oficială nodejs.org, de preferință într-o versiune cu suport pe termen lung (LTS), iar împreună cu el vine și gestionarul de pachete `npm`, folosit mai departe pentru construirea aplicației client. Celelalte două componente nu necesită o instalare separată. FastAPI este o dependență Python, instalată automat de comanda `uv sync --extra web` împreună cu serverul `uvicorn`. React este o dependență JavaScript instalată de `npm install`, alături de Vite și Tailwind, în directorul `webapp/frontend`. Prin urmare, singura cerință suplimentară pe care utilizatorul o pregătește manual pentru interfața web este Node.js, restul cerințelor fiind gestionate de `uv` și `npm`.

7.2. Instalarea aplicației

7.2.1. Obținerea codului și a dependențelor

După ce codul sursă () este descărcat, întregul mediu se pregătește printr-o singură comandă rulată din rădăcina proiectului (`Suport-Clinic-pentru-Decizii-In-Departamentul-De-Urgenta`). Comanda `uv sync` creează un mediu virtual izolat, descarcă versiunea de Python cerută și instalează toate dependențele fixate în fișierul de configurare. Dacă se dorește și interfața web, se folosește varianta cu `extra web`, care adaugă serverul FastAPI și bibliotecile aferente.

```
# dependentele de baza (linia de comanda)
uv sync

# dependentele suplimentare pentru interfața web
uv sync --extra web
```

Listing 7.1: Instalarea dependențelor Python. A doua comandă este necesară doar pentru interfața web.

7.2.2. Configurarea backendului de limbaj

Sistemul folosește un model de limbaj doar pentru partea de înțelegere a textului pacientului. Alegerea backendului și cheia (Gemini API Key) se dau printr-un fișier `.env` aflat în rădăcina proiectului `licenta/proiect_licenta`. Varianta implicită este Gemini Flash 2.5, care are nevoie de o cheie `GEMINI_API_KEY`. Comutarea pe modelul autogăzduit MedGemma se face schimbând variabila `LLM_BACKEND` și indicând adresa punctului de acces, așa cum s-a arătat în secțiunea 6.5. Un fișier minimal de configurare este prezentat în Listingul 7.2.

```
MODEL=gemini/gemini-2.5-flash
GEMINI_API_KEY=<cheia_personala>
LLM_BACKEND=flash
```

Listing 7.2: Conținutul minimal al fișierului `.env` pentru backendul implicit Gemini Flash.

7.2.3. Pregătirea interfeței web

Pentru interfața web mai este nevoie de un singur pas, instalarea dependențelor aplicației client. Acestea se descarcă cu `npm` din directorul `webapp/frontend`, o singură dată după clonarea proiectului.

```
cd webapp/frontend
```

```
npm install
```

Listing 7.3: Instalarea dependențelor aplicației client, necesară o singură dată.

După acești pași instalarea este completă. Sistemul poate fi pornit fie din linia de comandă, fie prin interfața web, așa cum se descrie în continuare.

7.2.4. Reantrenarea modelelor (opțional)

Fișierele cu modele antrenate din directorul **artifacts/** nu sunt incluse, așa că un utilizator care deschide proiectul fie le primește separat, fie le regenerează prin antrenare. Acest pas este necesar doar în al doilea caz și nu face parte din utilizarea obișnuită a aplicației.

Reantrenarea are câteva cerințe suplimentare față de simpla inferență. Este nevoie de baza de date MIMIC-IV-ED, care nu este accesibilă publicului larg. Ea se obține de pe platforma PhysioNet, după acreditare și semnarea acordului de utilizare a datelor. Pe lângă tabelele de urgență, caracteristicile de antecedente folosesc și modulul de note clinice (fișierul **discharge.csv**), din care se extrag diagnosticele anterioare ale pacientului. Toate aceste fișiere se pun în directorul **data/**. Antrenarea necesită putere de calcul (8-16 GB RAM, recomandat placă video) și poate dura de la câteva zeci de minute până la una sau două ore pentru fiecare model, în funcție de puterea de calcul. O placă grafică nu este obligatorie, dar scurtează foarte mult acest timp de antrenare al modelelor.

Modelele folosite de sistemul care rulează efectiv, adică stiva v3, se antrenează în ordinea din Listingul 7.4. Fiecare comandă generează artefactele proprii în subdirectorul corespunzător din **artifacts/**.

```
uv run train_triage_v3 # triaj: acuitate + dispozitie
uv run train_doctor_v3 # doctor v3 base (evaluarea initiala)
uv run train_nurse_v3 # doctor v3 dupa asistent (reevaluare)
uv run train_doctor_disposition # model dedicat de dispozitie, calibrat
uv run train_icd_resolver # rezolverul de cod ICD exact (etapa a doua)
```

Listing 7.4: Reantrenarea modelelor active (stiva v3), în ordinea dependențelor.

Dacă utilizatorul dorește, pot fi antrenate și modelele de referință v1 și v2 opționale, care nu sunt folosite de aplicația care rulează, ci doar pentru comparația între versiuni din Capitolul 6. Acestea se obțin cu **uv run train_models** și **uv run train_triage_v2** pentru triaj, respectiv **uv run train_doctor** și **uv run train_nurse** pentru doctor. După ce directorul **artifacts/** este populat, sistemul se folosește exact ca în secțiunile 7.3 și 7.4.

7.3. Utilizarea din linia de comandă

Cel mai simplu mod de rulare este direct în terminal. Comanda **uv run run-crew** pornește întregul echipaj de agenți și începe o conversație interactivă. Utilizatorul descrie starea pacientului în limbaj natural, iar agenții preiau cazul pas cu pas. Atunci când un agent are nevoie de o informație suplimentară, cum ar fi un semn vital sau o confirmare, pune o întrebare în terminal și așteaptă răspunsul, prin mecanismul descris în secțiunea 5.8. La final, sistemul afișează un raport structurat care conține nivelul de acuitate, decizia de internare sau externare și, pentru pacienții pentru care decizia de dispoziție este de internare, diagnosticul, departamentul și codurile ICD candidate.

Un exemplu de interacțiune cu sistemul este prezentat în Listingul 7.5. Ieșirea poate fi inclusă în lucrare sub formă de text, fără a fi necesară o captură de ecran, pentru

că aplicația afișează rezultatele direct în terminal. Valorile prezentate au caracter orientativ și vor doar să ilustreze modul de utilizare al aplicației, fără a reprezenta rezultate specifice obținute în urma unei evaluări experimentale.

```
$ uv run run_crew

Descrieti starea pacientului:
> Barbat de 68 de ani cu durere in piept apasatoare de o ora si
  dificultate in respiratie.

[Confirmare intake] Varsta: 68 | Gen: M | Sosire: pe cont propriu
Sunt corecte campurile extrase? (da / corecteaza) > da

--> Triaj: acuitate ESI 2 (posibil ESI 1); recomandare de internare.

[Asistent]: Care este tensiunea arteriala? > 150/95
[Asistent]: Aveti afectiuni cronice cunoscute? > hipertensiune

--> Reevaluare doctor:
  Diagnostic (top 3): Boli circulatorii, Boli respiratorii, ...
  Departament (top 3): Medicina Cardiaca, Medicina Generala, ...
  Coduri ICD candidate: I20, I21, ...
```

Listing 7.5: Fragment ilustrativ dintr-o conversație în terminal (prescurtat).

Modul de rulare din linia de comandă este util pentru testare rapidă și pentru rularea seturilor de evaluare. Pentru o utilizare de zi cu zi, interfața web oferă o experiență mai clară, cu aceleași rezultate.

7.4. Utilizarea interfeței web

Interfața web prezintă același flux ca varianta din terminal, dar sub forma unei conversații clinice în browser, cu widget-uri de introducere a datelor și cu rezultatele afișate sub formă de carduri vizuale. Predicțiile sunt identice cu cele din terminal, pentru că serverul rulează exact același echipaj de agenți. Doar modul de prezentare diferă în acest mod, așa cum s-a explicat în secțiunea 5.8.

7.4.1. Pornirea interfeței

Interfața web are nevoie de două procese care rulează în paralel, ambele pornite din rădăcina proiectului. Primul este serverul FastAPI. Acesta găzduiește echipajul de agenți și citește configurarea din fișierul `.env`. Cel de-al doilea pornește serverul de dezvoltare al aplicației client, care afișează interfața utilizator și transmite cererile către serverul FastAPI. După pornirea ambelor, interfața este disponibilă în browser la adresa afișată de serverul client, în mod implicit `http://localhost:5173`.

```
# terminal 1 -- serverul FastAPI (backend)
uv run uvicorn webapp.backend.main:app --port 8000 --reload

# terminal 2 -- serverul client (frontend)
cd webapp/frontend
npm run dev
```

Listing 7.6: Pornirea celor două procese ale interfeței web, în două terminale separate.

7.4.2. Parcurgerea unei consultații

La deschiderea interfeței, primul lucru pe care îl vede utilizatorul este un câmp în care descrie starea pacientului în limbaj natural, la fel ca în terminal. După ce trimite descrierea, agentul de parsare extrage câmpurile structurate (vârstă, gen, mod de sosire, motivul prezentării) și le afișează într-un formular editabil, astfel încât orice eroare de extragere să poată fi corectată înainte de triaj. Acest pas de confirmare este ilustrat în Figura 7.1.

După confirmare, conversația pornește. Interfața afișează în partea laterală un panou cu cei patru agenți și cele șase sarcini, evidențiind sarcina activă în fiecare moment. În corpul paginii se construiește transcrierea conversației pe măsură ce agenții lucrează. Instrumentele care rulează apar ca indicatoare „în curs / gata”, iar întrebările agenților sunt afișate cu widget-ul potrivit în funcție de tipul de răspuns așteptat, cum ar fi spre exemplu un câmp numeric pentru un semn vital sau o alegere da/nu. Această vedere de ansamblu este prezentată în Figura 7.2.

Rezultatele fiecărei etape sunt afișate ca și carduri vizuale, nu ca text brut. Predicția de triaj apare ca un indicator de acuitate pe scala ESI, împreună cu decizia de internare sau externare cu probabilitatea asociată, ca în Figura 7.3. Evaluarea doctorului afișează primele trei categorii de diagnostic și primele trei departamente, fiecare cu probabilitatea corespunzătoare, iar pentru pacienții internați și lista de coduri ICD candidate, ca în Figura 7.4. Pentru utilizatorul care vrea să înțeleagă cum s-a ajuns la o predicție, fiecare sarcină are un panou pliant care arată raționamentul agentului.

Folosind acest manual, o persoană fără cunoștințe despre implementarea internă poate instala sistemul și poate parcurge o consultație completă, de la descrierea pacientului până la raportul final, în oricare dintre cele două moduri de rulare.

ED

Emergency Department Decision Support
Live multi-agent runtime · MIMIC-IV models · research prototype

Describe the patient's symptoms

The intake specialist will read this, ask any follow-up questions, and the care team (triage, physician, nurse) will work through the case live.

I am a 65 year old male, I woke up with a sharp pain in my chest and I feel dizzy. I also can't feel my right hand.

Examples: 1 2 3

Begin assessment

ED

Emergency Department Decision Support
Live multi-agent runtime · MIMIC-IV models · research prototype

New patient

WORKFLOW

1 Intake parsing
Intake Specialist

2 Triage
Triage Specialist

3 Initial assessment
Physician

4 Nurse data
Nurse

5 Disposition
Physician

6 Reassessment
Physician

INTAKE PARSING

✓ Asking the patient done

✓ Asking the patient done

✓ Asking the patient done

✓ Asking the patient done

✓ Asking the patient done

Confirming intake running...

I am a 65 year old male, I woke up with a sharp pain in my chest and I feel dizzy. I also can't feel my right hand.

17909008

5

walk in

I've had heart failure in the past, I'm also struggling with diabetes

2 or three times

Intake is asking

Please review the information I extracted and correct anything that is wrong.

CHIEF COMPLAINTS

chest pain, dizziness, right hand paresthesia

PAIN (0-10, -1 UNKNOWN)

5

AGE

65

GENDER

male

ARRIVAL TRANSPORT

walk-in

MRN (SUBJECT_ID, -1 IF NEW)

17909008

PRIOR ADMISSIONS (-1 UNKNOWN)

3

PAST MEDICAL HISTORY

heart failure, diabetes

Confirm and continue to triage

Figura 7.1: Pagina de introducere a cazului. Descrierea în limbaj natural este transformată de agentul de parsare într-un set de câmpuri structurate, afișate într-un formular pe care utilizatorul îl poate corecta înainte de triaj.

1 Intake parsing •
Intake Specialist

2 Triage
Triage Specialist

3 Initial assessment
Physician

4 Nurse data
Nurse

5 Disposition
Physician

6 Reassessment
Physician

INTAKE PARSING

Asking the patient running...

Agent is asking

Hello! I'm here to help gather some information for your triage. Have you been treated at this hospital before? If so, do you have your medical record number (MRN)?

17909008

Send

17909008

Asking the patient running...

Agent is asking

Thank you. On a scale of 0 to 10, with 0 being no pain and 10 being the worst pain imaginable, what would you rate your chest pain?

5

Send

Figura 7.2: Conversația în desfășurare. Panoul lateral arată agentul și sarcina active, iar în transcriere apar mesajele agenților și întrebările adresate utilizatorului, fiecare cu widget-ul de introducere potrivit.



Figura 7.3: Cardul de rezultat al triajului: nivelul de acuitate pe scala ESI și decizia de internare sau externare, cu probabilitatea estimată de model.

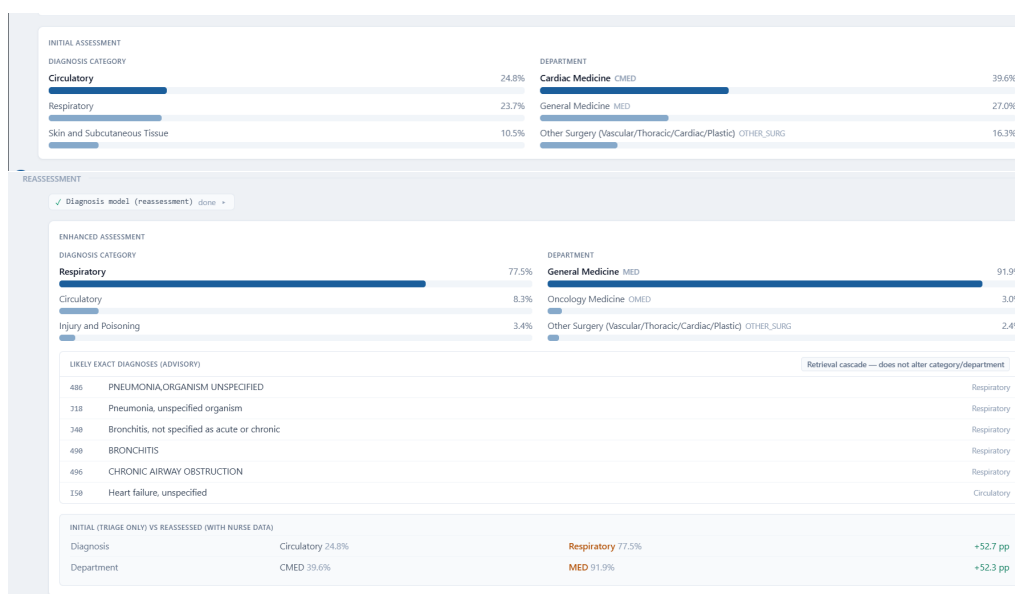


Figura 7.4: Cardul de rezultat al doctorului: primele trei categorii de diagnostic și primele trei departamente, fiecare cu probabilitatea sa, plus codurile ICD candidate pentru pacienții internați.

Capitolul 8. Concluzii

Lucrarea a pornit de la observația formulată în Capitolul 1, potrivit căreia parcursul unui pacient prin Unitatea de Primiri Urgențe este un lanț de decizii legate între ele, dar tratate de obicei separat în literatură, iar cele două direcții tehnice care abordează problema, învățarea automată pe date tabulare și sistemele cu modele lingvistice mari, au fiecare un punct slab clar. Soluția propusă a fost o arhitectură cu mai mulți agenți care combină cele două abordări astfel încât fiecare să compenseze slăbiciunea celeilalte, cu o limită strictă impusă rolului modelului de limbaj. Obiectivele stabilite în Capitolul 2 au fost atinse. Rezultatele din Capitolul 6 confirmă că sistemul modelează întregul flux clinic folosind predicții auditabile.

Contribuția principală este modelarea completă a lanțului de decizii din UPU ca o cascadă de cinci predicții înlanțuite, acuitate, internare, diagnostic, departament și cod ICD, în locul unei singure sarcini tabulare izolate. Fiecare predicție folosește ieșirea prezisă de etapa anterioară, nu răspunsul corect, așa că evaluarea reflectă comportamentul real al sistemului pus cap la cap. A doua contribuție este separarea strictă a rolurilor: modelul de limbaj este folosit exclusiv pentru a înțelege textul natural al pacientului și a-l transforma într-o reprezentare structurată. Toate deciziile clinice sunt astfel preluate de modele supervizate de tip XGBoost, antrenate pe prezentări reale din MIMIC-IV-ED. Această împărțire face decizia clinică să poată fi verificată și reprodusă, răspunzând cerinței de auditabilitate. A treia contribuție, și cea mai apropiată de întrebarea de cercetare, este proiectarea evaluării medicului în două momente, înainte și după ce asistentul colectează semnele vitale, medicația și antecedentele. Această separare permite măsurarea directă a valorii clinice aduse de datele suplimentare, o comparație care lipsește din majoritatea lucrărilor din domeniu. La acestea se adaugă modelul dedicat de dispoziție, calibrat izotonic, etapa a doua de determinare a codului ICD exact în interiorul categoriei prezise, backendul de limbaj comutabil între Gemini Flash și MedGemma autogăzduit și interfața web conversațională care rulează același echipaj de agenți ca varianta din linia de comandă.

Rezultatele obținute susțin aceste contribuții cu cifre concrete. Triajul clasifică acuitatea corect în peste 98% din cazuri la toleranță de un nivel și este orientat spre siguranță, cu o rată de sub-triaj menținută sub 16% și recuperată intenționat pe clasa cea mai vulnerabilă. Această alegere e justificată de faptul că este mult mai costisitor să ratăm un pacient grav, reprezentând o eroare cu cel mai mare cost clinic [2]. Modelul care prezice dispoziția de internare sau externare, prin intermediul agentului Doctor, este foarte bine calibrat, cu o eroare de calibrare de 0,0036 și o arie sub curba ROC de 0,914, cu mult peste prima estimare din triaj. Câștigul acesta se menține pentru fiecare subgrup clinic pe care l-am analizat. Cel mai important, comparația dintre evaluarea inițială și cea de după asistent arată un plus de 3,99 puncte procentuale la predicția de diagnostic și de 10,13 puncte la cea de departament, ceea ce răspunde direct întrebării de cercetare: datele suplimentare colectate de asistent aduc o îmbunătățire reală și măsurabilă a predicțiilor.

Aceleași rezultate trebuie însă privite critic, iar limitele sistemului sunt la fel de evidente ca punctele lui forte. Câștigul mare de la determinarea departamentului este concentrat pe clasa majoritară, adică Medicina Generală, și vine însoțit de o scădere

a acurateții echilibrate, de la 49,58% la 45,19%. Mai multe departamente minoritare, în special cele chirurgicale, regresează după adăugarea datelor de asistent, pentru că rutarea către ele depinde de natura leziunii, nu de semnele vitale. Determinarea codului ICD exact rămâne cea mai grea parte, în varianta oracol, modelul nimerește exact codul pe prima poziție în aproximativ 33% din cazuri, deși metricile gradate, care creditează codurile apropiate clinic, sunt considerabil mai ridicate, până la 91% cu similaritatea semantică pe primele cinci coduri. O limitare vine chiar din structura datelor. După cum am menționat anterior, aproximativ o treime dintre internări sunt etichetate cu un cod care descrie doar motivul prezentării, nu diagnosticul final. Aceste etichete sunt prea ambigue pentru a putea fi folosite eficient. Excluderea acestei clase în versiunea v3 a redus problema, dar nu a putut să o elimine. Analiza importanței caracteristicilor arată că descrierea motivului prezentării, transformată prin metoda TF-IDF, are cea mai mare influență asupra modelului de dispoziție, având o contribuție de 84,6% la performanța acestuia. Asta explică de ce aportul datelor colectate de asistent este real dar limitat, și de ce reformularea textului de către parser rămâne partea cea mai costisitoare a întregului lanț.

Limitările identificate evidențiază și principalele direcții în care sistemul poate fi dezvoltat în continuare. Cea mai promițătoare dintre acestea vizează parserul de limbaj, deoarece evaluarea a arătat că cea mai mare parte a degradării performanței la trecerea de la date tabulare la descrieri în limbaj natural este cauzată de reformulările realizate de modelul de limbaj, și nu de mecanismul de inferență în cascadă. Reducerea acestui cost se poate obține prin folosirea unor termeni clinici în promptul de extragere, printr-un dicționar de sinonime aplicat peste ieșirea parserului sau prin reprezentări de text robuste la reformulare, toate acestea fără a modifica modelele de predicție. O a doua direcție privește modelul de dispoziție, care ar putea fi reantrenat cu o funcție de cost asimetrică, ce penalizează mai tare sub-triajul, împreună cu o ajustare mai fină a pragului de decizie. O a treia direcție se referă la clasele minoritare de la determinarea departamentului. Aici, o tratare dedicată ar putea recupera scăderea de acuratețe echilibrată fără a sacrifica performanța globală. Rămân deschise ca și potențiale direcții de extindere și unificarea codurilor ICD-9 și ICD-10 și extinderea etapei de determinare a codului ICD astfel încât aceasta să poată fi realizată încă din faza evaluării inițiale a pacientului. Pe termen mai lung, sistemul poate fi integrat într-o infrastructură de spital reală, cu acces la dosarul electronic al pacientului, direcție care a fost menționată ca fază separată în planul proiectului.

În ansamblu, lucrarea demonstrează că un sistem de sprijin al deciziilor clinice poate acoperi întregul flux al Unității de Primiri Urgențe folosind un model de limbaj strict pentru înțelegerea textului și modele supervizate auditabile pentru fiecare decizie clinică. Predicțiile rămân orientative și sub controlul personalului medical. Faptul că fiecare dintre ele poate fi urmărită până la datele care au produs-o face din sistem un instrument de sprijin verificabil, nu o cutie neagră care decide în locul doctorului.

Bibliografie

- [1] N. Gilboy, P. Tanabe, D. Travers, and A. M. Rosenau, “Emergency severity index (ESI): A triage tool for emergency department care, version 4,” Agency for Healthcare Research and Quality (AHRQ), Publication No. 12-0014, 2012.
- [2] S. Tyler, M. Olis, N. Aust, L. Patel, L. Simon, C. Triantafyllidis, V. Patel, D. W. Lee, B. Ginsberg, H. Ahmad, and R. J. Jacobs, “Use of artificial intelligence in triage in hospital emergency departments: A scoping review,” *Cureus*, vol. 16, no. 5, p. e59906, 2024. [Online]. Available: <https://doi.org/10.7759/cureus.59906>
- [3] F. Gaber, M. Shaik, F. Allega, A. J. Bilecz, F. Busch, K. Goon, V. Franke, and A. Akalin, “Evaluating large language model workflows in clinical decision support for triage and referral and diagnosis,” *npj Digital Medicine*, vol. 8, no. 1, p. 263, 2025. [Online]. Available: <https://doi.org/10.1038/s41746-025-01684-1>
- [4] A. Johnson, L. Bulgarelli, T. Pollard, S. Horng, L. A. Celi, and R. Mark, “MIMIC-IV-ED (version 2.2),” PhysioNet, 2023. [Online]. Available: <https://doi.org/10.13026/5ntk-km72>
- [5] F. Xie, J. Zhou, J. W. Lee, M. Tan, S. Li, L. Rajnthern, M. L. Chee, B. Chakraborty, A.-K. I. Wong, A. Dagan, M. E. H. Ong, F. Gao, and N. Liu, “Benchmarking emergency department prediction models with machine learning and public electronic health records,” *Scientific Data*, vol. 9, no. 1, p. 658, 2022. [Online]. Available: <https://doi.org/10.1038/s41597-022-01782-9>
- [6] Y. Liu, Z. I. Carrero, X. Jiang, D. Ferber, G. Wölflein, L. Zhang, S. Jayabalan, T. Lenz, Z. Hui, and J. N. Kather, “Benchmarking large language model-based agent systems for clinical decision tasks,” *npj Digital Medicine*, vol. 9, no. 1, p. 259, 2026. [Online]. Available: <https://doi.org/10.1038/s41746-026-02443-6>
- [7] Z. Chen, Z. Peng, X. Liang, C. Wang, P. Liang, L. Zeng, M. Ju, and Y. Yuan, “Map: Evaluation and multi-agent enhancement of large language models for inpatient pathways,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.13205>
- [8] Y. Oliveira, I. Rios, P. Araújo, A. Macambira, M. Guimarães, L. Sales, M. Rosa Júnior, A. Nicola, M. Nakayama, H. Paschoalick, F. Nascimento, C. Castillo-Salgado, V. M. Ferreira, and H. Carvalho, “Artificial intelligence in triage of COVID-19 patients,” *Frontiers in Artificial Intelligence*, vol. 7, p. 1495074, 2024. [Online]. Available: <https://doi.org/10.3389/frai.2024.1495074>
- [9] Y. Raita, T. Goto, M. K. Faridi, D. F. M. Brown, C. A. Camargo, and K. Hasegawa, “Emergency department triage prediction of clinical outcomes using machine learning models,” *Critical Care*, vol. 23, no. 1, p. 64, 2019. [Online]. Available: <https://doi.org/10.1186/s13054-019-2351-7>

-
- [10] L.-H. Yao, K.-C. Leung, C.-L. Tsai, C.-H. Huang, and L.-C. Fu, “A novel deep learning-based system for triage in the emergency department using electronic medical records: Retrospective cohort study,” *Journal of Medical Internet Research*, vol. 23, no. 12, p. e27008, 2021. [Online]. Available: <https://doi.org/10.2196/27008>
 - [11] A. Vântu, A. Vasilescu, and A. Băicoianu, “Medical emergency department triage data processing using a machine-learning solution,” *Heliyon*, vol. 9, no. 8, p. e18402, 2023. [Online]. Available: <https://doi.org/10.1016/j.heliyon.2023.e18402>
 - [12] T. J. Pollard, A. E. W. Johnson, J. D. Raffa, L. A. Celi, R. G. Mark, and O. Badawi, “The eICU collaborative research database, a freely available multi-center database for critical care research,” *Scientific Data*, vol. 5, p. 180178, 2018. [Online]. Available: <https://doi.org/10.1038/sdata.2018.178>
 - [13] A. Z. Ahmed Abdalhalim, S. N. Nureldaim Ahmed, A. M. Dawoud Ezzelarab, M. Mustafa, M. G. A. Albasheer, R. E. A. Ahmed, and M. B. G. E. Elsayed, “Clinical impact of artificial intelligence-based triage systems in emergency departments: A systematic review,” *Cureus*, vol. 17, no. 6, p. e85667, 2025. [Online]. Available: <https://doi.org/10.7759/cureus.85667>
 - [14] E. Lansiaux, R. Azzouz, E. Chazard, A. Vromant, and E. Wiel, “Artificial intelligence models for predicting triage in emergency departments: Seven-month retrospective comparative study of natural language processing, large language model, and joint embedding predictive architectures,” *JMIR Medical Informatics*, p. e83318, 2026. [Online]. Available: <https://doi.org/10.2196/83318>
 - [15] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, “Large language model based multi-agents: A survey of progress and challenges,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.01680>
 - [16] B. Yan, Z. Zhou, L. Zhang, L. Zhang, Z. Zhou, D. Miao, Z. Li, C. Li, and X. Zhang, “Beyond self-talk: A communication-centric survey of llm-based multi-agent systems,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.14321>
 - [17] H. Yang, H. Chen, H. Guo, Y. Chen, C.-S. Lin, S. Hu, J. Hu, X. Wu, and X. Wang, “Llm-medqa: Enhancing medical question answering through case studies in large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.05464>
 - [18] D. Jin, E. Pan, N. Oufattole, W.-H. Weng, H. Fang, and P. Szolovits, “What disease does this patient have? a large-scale open domain question answering dataset from medical exams,” *Applied Sciences*, vol. 11, no. 14, p. 6421, 2021. [Online]. Available: <https://doi.org/10.3390/app11146421>
 - [19] M. Lu, B. Ho, D. Ren, and X. Wang, “Triageagent: Towards better multi-agents collaborations for large language model-based clinical triage,” in *Findings of the Association for Computational Linguistics: EMNLP 2024*. Association for Computational Linguistics, 2024, aCL Anthology 2024.findings-emnlp.329. [Online]. Available: <https://aclanthology.org/2024.findings-emnlp.329/>
 - [20] J. M. Ruiz Mejia and D. B. Rawat, “Medscrubcrew: A medical multi-agent framework for automating appointment scheduling based on patient-provider profile

- resource matching,” *Healthcare*, vol. 13, no. 14, p. 1649, 2025. [Online]. Available: <https://doi.org/10.3390/healthcare13141649>
- [21] D. Srirag, Q. D. Nguyen, A. Joshi, P. Narasimhan, and S. Kanhere, “Triagesim: A conversational emergency triage simulation framework from structured electronic health records,” 2026. [Online]. Available: <https://arxiv.org/abs/2603.10035>
- [22] N. Mehandru, N. Golchini, T. Zack, D. Bamman, and M. Molina, “Er-reason: A benchmark dataset for llm-based clinical reasoning in the emergency room,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.22919>
- [23] K. Wu, E. Wu, R. Thapa, K. Wei, A. Zhang, A. Suresh, J. J. Tao, M. W. Sun, A. Lozano, and J. Zou, “Medcasereasoning: Evaluating and learning diagnostic reasoning from clinical case reports,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.11733>
- [24] R. Hou, S. Zhu, Z. Huai, G. Yu, Y. Fan, C. Wang, and T. Ruan, “Clinicalmc: A benchmark for multi-course clinical decision-making with large language models,” 2026. [Online]. Available: <https://arxiv.org/abs/2606.03157>
- [25] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001. [Online]. Available: <https://doi.org/10.1214/aos/1013203451>
- [26] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, San Francisco, CA, USA, 2016, pp. 785–794. [Online]. Available: <https://doi.org/10.1145/2939672.2939785>
- [27] K. Spärck Jones, “A statistical interpretation of term specificity and its application in retrieval,” *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972. [Online]. Available: <https://doi.org/10.1108/eb026526>
- [28] J. Cohen, “Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit,” *Psychological Bulletin*, vol. 70, no. 4, pp. 213–220, 1968. [Online]. Available: <https://doi.org/10.1037/h0026256>
- [29] A. Niculescu-Mizil and R. Caruana, “Predicting good probabilities with supervised learning,” in *Proceedings of the 22nd International Conference on Machine Learning (ICML '05)*, 2005, pp. 625–632. [Online]. Available: <https://doi.org/10.1145/1102351.1102430>
- [30] crewAI Inc., “CrewAI: Framework for orchestrating role-playing, autonomous AI agents,” <https://github.com/crewAIInc/crewAI>, 2025, accesat: 30 iunie 2026.
- [31] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://www.jmlr.org/papers/v12/pedregosa11a.html>

- [32] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '19)*, Anchorage, AK, USA, 2019, pp. 2623–2631. [Online]. Available: <https://doi.org/10.1145/3292500.3330701>
- [33] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, “A fast and elitist multiobjective genetic algorithm: NSGA-II,” *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002. [Online]. Available: <https://doi.org/10.1109/4235.996017>

Anexa A. Secțiuni relevante din cod

Această anexă adună câteva fragmente de cod reprezentative pentru mecanismele descrise în Capitolul 5, altele decât listingurile deja incluse în corpul capitolului. Fragmentele sunt prescurtate pentru lizibilitate, păstrând logica esențială. Codul complet se află în proiectul propriu-zis, în directorul `src/proiect_licenta/`.

A.1. Algoritmul de ordonare a codurilor ICD

Etapa a doua de determinare a codului ICD, discutată în secțiunea 6.3.5, ordonează codurile candidate dintr-o categorie după un scor care combină similaritatea de text, profilul fiziologic, antecedentele și prevalența. Termenul de antecedente este aplicat doar pacienților cu istoric, restul folosind varianta cu trei termeni. Funcția din Listingul A.1 implementează acest amestec, din modulul `icd_resolution.py`.

```
def gated_score(text_cos, vital_sim, pmh_sim, prevalence,
                weights_pmh, weights_nopmh, has_pmh):
    """Amesteca termenii, aplicand termenul PMH doar pacientilor cu istoric.
    Fiecare termen este normalizat min-max in interiorul setului de candidati.
    Pacientii cu istoric folosesc ponderile de 4 termeni (text, vitale, PMH,
    prevalenta); ceilalti folosesc varianta de 3 termeni."""
    text_cos = np.asarray(text_cos, dtype=np.float32)
    Tn = _minmax(text_cos) # similaritate de text
    Vn = _minmax(vital_sim) if vital_sim is not None else 0.0 # fiziologie
    Pn = _minmax(prevalence) # prevalenta

    def _combine(w):
        return w["text"] * Tn + w.get("vit", 0.0) * Vn + w["prev"] * Pn

    score_no = _combine(weights_nopmh)
    if pmh_sim is None:
        return score_no
    Mn = _minmax(pmh_sim) # similaritate PMH
    score_pmh = _combine(weights_pmh) + weights_pmh.get("pmh", 0.0) * Mn
    return score_pmh if has_pmh else score_no
```

Listing A.1: Amestecul cu termen de antecedente condiționat, din `icd_resolution.py` (prescurtat, cazul unei singure interogări).

A.2. Construirea antecedentelor fără scurgeri de informație

Caracteristicile de antecedente medicale provin din prezentările anterioare ale pacientului. Corectitudinea lor depinde de un filtru strict care păstrează numai evenimentele petrecute înainte de momentul curent, astfel încât modelul să nu vadă niciodată informație din viitor. Listingul A.2 arată acest filtru și asamblarea celor 19 caracteristici, din modulul `pmh_features.py`.

```
def assemble_pmh_for_stay(subject_id, intime, complaint_norm, index):
    """Deriva cele 19 caracteristici de antecedente pentru o prezentare,
    aplicand filtrul strict prior_*time < intime. Pacientii fara prezentari
    anterioare primesc tiparul all-zero cu no_history=1."""
```

```

intime = pd.to_datetime(intime)
sid = int(subject_id)

# Doar internarile si vizitele la UPU strict anterioare momentului curent.
prior_adm = [(at, hid) for at, hid in index["adm_by_subject"].get(sid, [])
              if at < intime]
prior_ed = [(it, _, cc) for it, _, cc in index["ed_by_subject"].get(sid, [])
            if it < intime]

# Garantie explicita: nimic la sau dupa intime nu poate trece filtrul.
assert all(at < intime for at, _ in prior_adm), "scurgere PMH"
assert all(it < intime for it, _, _ in prior_ed), "scurgere PMH"

# Steaguri de diagnostic din notele si codurile prezentarilor anterioare.
flags = set()
for _, hid in prior_adm:
    flags |= index["note_pmh_by_hadm"].get(hid, set())
    flags |= index["icd_pmh_by_hadm"].get(hid, set())

rec = {"pmh_cat": (1 if cat in flags else 0) for cat in PMH_CATEGORIES}
rec["n_prior_admissions"] = len(prior_adm)
rec["n_prior_ed_visits"] = len(prior_ed)
rec["no_history"] = int(len(prior_adm) == 0 and len(prior_ed) == 0)
# ... plus days_since_last_admission / _ed si same_complaint_as_prior
return rec

```

Listing A.2: Asamblarea antecedentelor cu filtrul anti-scurgere `prior_*time < intime`, din `pmh_features.py` (prescurtat).

A.3. Reprezentarea în timp a semnelor vitale

Modelele doctorului folosesc un bloc de semne vitale longitudinale, construit din mai multe citiri colectate de asistent. Când un semn vital nu are citiri de traiectorie, blocul revine la valoarea instantanee, exact cum a fost tratată situația la antrenare, ceea ce menține reprezentarea din timpul rulării apropiată de distribuția de antrenare. Listingul A.3 arată nucleul construcției, din modulul `vital_trajectory.py`.

```

def build_longitudinal_block(snapshot, readings=None, rhythm="",
                             rhythm_readings=None):
    """Construiește blocul longitudinal din mai multe citiri. Cand un semn vital
    nu are citiri de traiectorie, revine la valoarea instantanee (snapshot)."""
    cleaned = _clean_readings(readings)
    block = {}
    any_reading = any(len(cleaned[v]) > 0 for v in _VITAL_COLS_LONG)

    # min / max / ultima valoare / delta pentru fiecare semn vital
    for vital in _VITAL_COLS_LONG:
        vals = cleaned[vital]
        if vals:
            block[f"{vital}_min"] = float(min(vals))
            block[f"{vital}_max"] = float(max(vals))
            block[f"{vital}_last"] = float(vals[-1])
            block[f"{vital}_delta"] = float(vals[-1] - vals[0])
        else: # fara traiectorie -> snapshot
            snap = float(snapshot[vital])
            block[f"{vital}_min"] = block[f"{vital}_max"] = snap
            block[f"{vital}_last"] = snap

```

```

        block[f"{vital}_delta"] = 0.0

    # 1 daca a existat cel putin o citire reala; 0 = doar snapshot
    block["has_longitudinal_vitals"] = 1 if any_reading else 0
    # ... plus contoarele de citiri anormale si one-hot-ul de ritm cardiac
    return {col: block[col] for col in LONG_VITAL_FEATURE_COLS}

```

Listing A.3: Construirea blocului longitudinal de semne vitale din mai multe citiri, din `vital_trajectory.py` (prescurtat).

A.4. Mecanismul de comunicare cu utilizatorul

Instrumentele interactive nu știu dacă rulează în terminal sau în interfața web. Ele obțin o funcție `ask` de la fabrica din Listingul A.4 și o apelează pentru fiecare întrebare. Când există un canal de sesiune, întrebarea este trimisă în browser. Altfel se folosește consola. Acest mecanism din modulul `interaction.py` este cel care face ca aceleași instrumente să funcționeze identic în ambele moduri de rulare descrise în Capitolul 7.

```

def make_ask(channel=None, *, default_role="Nurse"):
    """Intoarce un apelabil 'ask' legat de 'channel' (sau la stdin cand e None).
    Instrumentele apeleaza ask(prompt, kind=...) pentru fiecare intrebare;
    comportamentul din terminal ramane neschimbat."""

    def ask(prompt, kind="text", *, role=None, meta=None):
        ch = channel if channel is not None else _current_channel.get()
        if ch is not None: # interfata web
            m = dict(meta or {})
            m.setdefault("role", role or default_role)
            return ch.ask(kind, prompt, m)
        # Varianta din terminal -- se pastreaza formatarea CLI.
        print(f"\n [{role or default_role}]: {prompt}")
        return input(" [You]: ")

    return ask

```

Listing A.4: Fabrica `make_ask` care comută transportul întrebărilor între browser și terminal, din `interaction.py`.

Anexa B. Lucrări publicate (dacă există)

În cadrul cercetării aferente acestei lucrări de licență a fost elaborat și trimis următorul articol științific:

R. R. Szomiu, D. R. Ember și A. Groza, „Towards glass-box triage: an interpretable multi-agent clinical decision support system for emergency department”, în *Proc. IEEE International Conference on Intelligent Computer Communication and Processing (ICCP)*, Cluj-Napoca, România, 2025.

Versiunea integrală a articolului, în forma publicată, este reprodusă în paginile următoare.

Towards glass-box triage: an interpretable multi-agent clinical decision support system for emergency department

Roxana Ramona Szomiu, Darius Robert Ember, Adrian Groza

Artificial Intelligence Research Institute AIRi@UTCN

Department of Computer Science

Technical University of Cluj-Napoca

Cluj-Napoca, Romania

Email: Roxana.Szomiu@cs.utcluj.ro, Ember.Gh.Darius@student.utcluj.ro, Adrian.Groza@cs.utcluj.ro

Abstract—Emergency Departments (EDs) are high-pressure, resource-constrained environments in which accurate and rapid patient triage is critical to clinical safety. The patient’s initial assessment determines the urgency of care required and shapes their subsequent clinical pathway - including resource allocation, specialist routing, and final disposition. While machine learning (ML) have demonstrated strong predictive capabilities for clinical outcomes, their adoption in real-world healthcare settings remains limited by a lack of interpretability: such models typically operate as black boxes, returning probabilities without transparent clinical reasoning.

To address this gap, we introduce a multi-agent, pipeline-based clinical decision support system for ED triage. Rather than relying on a single model, the architecture delegates distinct clinical tasks to specialized agents, using Large Language Models (LLMs) exclusively to parse patient narratives and manage the interaction with patient, while all clinical judgments (acuity scoring, diagnosis, and hospital disposition) are handled by calibrated, supervised XGBoost classifiers. Trained and validated on over 400,000 real-world encounters from the MIMIC-IV-ED dataset, the system also distinguishes between an initial assessment and a refined assessment based on supplementary nurse-collected data, quantifying the contribution of additional clinical information to the triage decision.

This paper presents the system’s architecture and evaluates its clinical safety with a focus on under-triage mitigation, showing that this decoupled design preserves full decision traceability without sacrificing predictive performance.

Index Terms—clinical triage, machine learning, large language models, multi-agent system, specialized agents, emergency department, evidence-based nursing, clinical decision support, interpretability

I. INTRODUCTION

Emergency departments (EDs) across Europe and beyond are experiencing growing pressure from increasing demand and limited resources. Across 26 OECD ¹ countries, EDs recorded an average of 31 visits per 100 population in 2023, with approximately 20% of these visits resulting in hospital admission. This demand differs across countries: annual ED utilization ranges from fewer than 15 visits per 100 population in some health systems to more than 65 visits per 100

population in countries such as Portugal and Spain [1]. At the population level, the European Society for Emergency Medicine (EUSEM) estimates that approximately 25% of Europeans visit an ED each year - equivalent to 100–300 visits per 1,000 inhabitants annually depending on the country - and that 70–80% of all hospital admissions originate from the ED [2]. Together, these statistics demonstrate that the ED is a central gateway for acute care triage rather than a peripheral entry point to the healthcare system.

Triage - the process of prioritizing patients based on the urgency of their condition at arrival - is a key component of emergency care. Accurate triage is essential: under-triage can delay care for critically ill patients, while over-triage can waste limited healthcare resources.

Over recent decades, EDs have experienced increasing patient volumes, especially among more complex and critically ill cases, while emergency care resources have remained limited. This imbalance has contributed to ongoing overcrowding and longer waits before patients receive triage and treatment. Since, triage is the first step in emergency care, delays at this stage can affect the entire patient pathway.

These challenges highlight the need for improved triage strategies that optimize resource use and support clinical decision-making. Based on these limitations, AI-based systems have emerged as a promising approach to enhance ED triage. Recent progress in ML and LLMs has the potential to improve clinical decision support, reduce variability in triage assessments, and enable more efficient patient prioritization under high-demand conditions.

Although traditional predictive models often achieve high accuracy, they typically operate as black boxes, providing little insight into the reasoning behind their predictions. This lack of interpretability limits the use of clinical applications in the real-world, as medical experts must be able to understand and justify decisions that directly affect patient care.

To address this limitation, we propose a modular, pipeline-based framework that mirrors the sequential clinical triage workflow. Using specialized LLM-based agents, the system separates natural language understanding from structured ML-

¹Organisation for Economic Co-operation and Development

based clinical prediction, enabling a more transparent and interpretable decision-making process.

Our system exposes the reasoning process at each stage of the clinical workflow, allowing clinicians to inspect, verify, and interpret intermediate decisions. By combining modular reasoning with transparent decision support, the system improves accountability, increases clinician trust, and supports—rather than replaces—human judgment in emergency triage.

II. RELATED WORK

In recent years, the use of LLMs has increased significantly across various domains. In parallel, the healthcare systems have evolved from single-task predictive models to complex, collaborative multi-agent systems (MAS). Recent literature further highlights the potential of LLMs to support and simulate aspects of clinical reasoning through automated and interactive decision-making frameworks. Research on automated support for medical triage has evolved along two largely parallel tracks: (1) classical machine learning (ML) and deep learning (DL) models trained on structured/unstructured EHR data, and (2) more recent evaluations of LLMs applied zero-shot or few-shot to triage decision-making.

Supervised ML and DL models trained on initial triage data have consistently outperformed traditional rule-based systems such as the Emergency Severity Index (ESI) protocol [3]. Raita et al. [4] showed that gradient-boosted and deep neural network models predicted hospitalization and critical-care outcomes more accurately than an ESI-based logistic regression baseline while reducing over-triage. Yao et al. [5] further demonstrated that combining structured vital signs with unstructured electronic health record (EHR) data improves outcome prediction beyond ESI alone. Extending this work to clinical practice, Vântu et al. [6] showed that automated ML pipelines applied to raw triage data can support scalable and consistent decision-making in hospital settings. A systematic review by Porto [7] concluded that ML and NLP approaches generally improve triage accuracy and consistency compared with conventional methods, while emphasizing the need for stronger external validation and broader clinical implementation.

While classical ML/DL models rely on task-specific training, recent research has explored zero-shot LLMs (e.g., GPT-4 model) as flexible triage decision-support systems that require no retraining. Studies [8], [9], [10] found that LLMs can achieve competitive, sometimes physician-comparable, triage performance through language-based reasoning, but they become less reliable for complex cases and remain unsuitable for autonomous decision-making. These findings are further supported by Cui et al. [11] in a systematic review and meta-analysis that reported high specificity (97%) but only moderate sensitivity (61%) to identify patients with the highest-acuity, suggesting strong ability to rule out lower-risk cases, but limitations in detecting the most urgent patients, while Lee et al. [12] found that LLM performance differed across demographic groups. Overall, current evidence supports zero-shot LLMs as

clinician decision-support tools rather than autonomous triage systems.

Recent studies have explored the use of multi-agent LLM architectures for clinical tasks beyond single-model prediction, revealing that the benefits of agentic collaboration are task-dependent, with improvements over baseline LLMs observed in specific settings.

TriageAgent [13] specifically targets clinical triage through a dynamic multi-agent framework in which LLM-based agents are engaged in multi-round discussion, verify each other’s reasoning, incorporate self-confidence and early-stopping mechanisms, and ground their decisions using retrieval-augmented generation over the ESI handbook. The framework achieves higher diagnostic and triage accuracy than state-of-the-art single-LLM baselines across three clinical triage benchmarks, suggesting that structured multi-agent collaboration reduces isolated hallucinations and improves triage performance.

The authors [14] introduced the MAP framework that combines specialized triage, diagnosis, and treatment agents under a supervising agent, achieving a 25.1% improvement in diagnostic accuracy over HuatuoGPT2-13B and outperforming clinicians on the IPDS benchmark of 51,274 MIMIC-IV cases. Together, *TriageAgent* and MAP demonstrate the potential of multi-agent LLM collaboration to surpass single-model baselines in clinical prediction.

Chen et al. [15] conducted the first large-scale systematic review and meta-analysis comparing the independent and collaborative performance of LLMs and clinicians in diagnosis and triage. Across 50 studies evaluating 25 LLMs, they found that LLMs achieved triage accuracy comparable to clinicians, while clinician-LLM collaboration consistently improved diagnostic performance over clinicians alone. The results support the integration of LLMs as decision-support tools in clinical triage rather than autonomous replacements for clinicians.

Several publicly available datasets are widely used in the literature for ED triage and downstream clinical prediction addressing complementary aspects, including MIMIC-IV-ED [16] and eICU-CRD [17]. MIMIC-IV-ED supports emergency department triage by capturing patient encounters from arrival through disposition, whereas eICU-CRD focuses on post-triage critical care and is primarily used for outcome prediction and cross-institutional validation.

III. METHODOLOGY

The proposed methodology follows the multi-agent system paradigm by breaking the main task into specialized sub-tasks, each assigned to an agent with a specific responsibility. Instead of relying on a single end-to-end neural network to map a raw patient case directly to a diagnosis, the architecture distributes the processing across multiple agents.

Our system sequentially predicts five clinical outcomes: (i) the Emergency Severity Index (ESI) triage level, (ii) the admission or discharge decision, (iii) the diagnosis for admitted patients, (iv) the appropriate hospital department and (v) the corresponding diagnostic ICD (International Classification of Diseases) code. This separation of responsibilities enables a

predictable transformation of data-from unstructured natural language to structured clinical features, and finally to deterministic clinical predictions - while enhancing modularity and explainability.

A. Dataset: MIMIC-IV-ED

To ground the proposed system in real-world clinical practice, we trained and evaluated it using the MIMIC-IV Emergency Department (MIMIC-IV-ED) dataset [16], a large, publicly available, database containing approximately 425,000 ED encounters collected between 2011 - 2019. We selected this dataset because it is well suited for modeling sequential clinical workflow in ED triage. It includes triage assessments, longitudinal vital signs, medication reconciliation records, discharge diagnoses, and ED stay information, providing a comprehensive representation of the patient care pathway.

The dataset is organized into multiple tables capturing different stages of the ED visit: triage (chief complaint, pain score, and initial vital signs), vitalsign (timestamped vital sign measurements), medrecon (pre-admission medications), diagnosis (final ICD-9/ICD-10 diagnoses), and edstays (arrival and discharge times, length of stay, and patient disposition).

The timestamped structure enables realistic modeling of sequential clinical workflows by restricting each predictive model to information available at the corresponding stage of care. For example, the first agent of the pipeline (Triage Agent) is trained solely on initial triage data, whereas next agents incorporate longitudinal vital signs and medication histories collected later during the ED stay. This temporal separation prevents data leakage and ensures clinically valid model development.

B. System architecture

The architecture of our multi-agent system is illustrated in Figure 1. Our solution consists of four specialized agents executing six sequential tasks, coordinated through a deterministic sequential workflow, with each agent responsible for a specific stage of the clinical process. Information is passed sequentially, following the natural progression of the patient’s care pathway:

- 1) **NLP Parser Agent:** is a LLM-based agent that extracts structured features from free text.
- 2) **Triage Agent (ML):** predicts initial ESI-score for acuity and disposition based strictly on intake data using XGBoost.
- 3) **Assistant (Nurse) Agent:** simulates interactive clinical data collection, gathering vitals and information about medications.
- 4) **Doctor Agent (ML Cascade):** refines predictions and establishes a final diagnosis using all available longitudinal data ².

The proposed system is built on the CrewAI orchestration framework, integrating a set of specialized LLM-based agents with dedicated XGBoost predictive models. The architecture

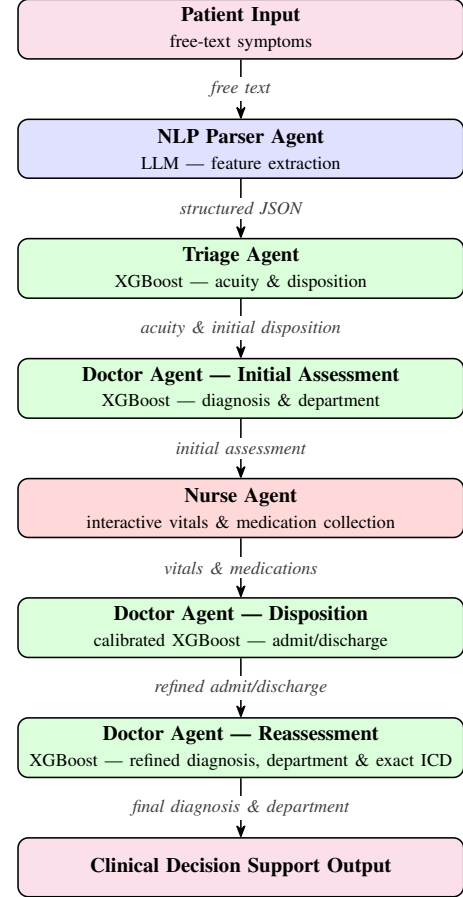


Fig. 1. The sequential pipeline architecture of the clinical prototype. Colour encodes the agent type: LLM-based natural-language processing (blue), supervised machine-learning prediction (green), and interactive data collection (red).

follows a hybrid design principle that explicitly separates two concerns: (i) natural language understanding, handled by the LLM layer, and (ii) clinical prediction, handled by gradient-boosted tree models trained on structured and text-derived features. The LLM does not generate clinical judgments directly; it converts free-text inputs into a structured format, invokes prediction tools that wrap the trained models, and reports their outputs. This design separates language understanding and orchestration from clinical prediction, which remains the responsibility of validated and auditable statistical models.

Information is exchanged between tasks through a lightweight harness layer rather than direct agent-to-agent communication. The harness enforces a structured interaction contract: an upstream agent must produce outputs in a pre-defined format, and downstream agents receive exactly this validated structure [18]. In our system, inter-agent communication relies on two mechanisms: a *context channel* that forwards textual outputs to preserve the clinical narrative, and a *structured-data contract* that appends machine-readable blocks containing key variables (e.g., ESI level, disposition, age, and gender). This fixed interface enables deterministic

²health information collected from the same person over an extended period.

parsing of prediction outputs while keeping agents decoupled from internal prompts, reasoning traces, and intermediate representations.

Each specialized agent processes the output of the previous stage, transforming free-text input into structured clinical predictions. The four agents are invoked across six stages and the Doctor agent runs three times (initial assessment, disposition, and reassessment).

1) *Natural Language Parsing Agent*: The system processes a patient case through a cascading, multi-stage pipeline. Similar to an ED, the workflow begins when a patient arrives and provides a free-text description of their symptoms to the intake module. This LLM-based agent (Gemini 2.5 Flash, MedGemma) is the entry point of the system. The agent transforms unstructured patient-reported text into a structured intake object with a fixed schema containing key demographic, clinical, and arrival information, including: *chief complaints, pain score, history, prior admissions, transport mode, and available pre-hospital vital signs*.

Missing required fields are resolved through interactive clarification using *AskPatientTool*, rather than imputation, and the parsed intake is verified by a human before initiating triage prediction. This step is crucial because the downstream XGBoost machine learning models require a consistent feature matrix to perform accurately. By isolating the LLM at this specific stage, we completely eliminate the risk of the language model "hallucinating" a diagnosis; its sole, verifiable purpose is data normalization and extraction.

2) *Triage Agent*: Once the patient data is structured into a clean JSON format, it is passed to the Triage Agent. This agent predicts the ESI acuity and admission probability via *TriagePredictionTool*, which wraps two pre-trained XGBoost models. Its input schema uses explicit missingness conventions (e.g., unknown vitals encoded as -1) so the model can represent absence of information rather than have the LLM generated values.

Since the input consists mainly of short chief complaints and demographic features, textual information is transformed using TF-IDF to generate a sparse feature representation that captures the importance of clinically relevant terms. XGBoost was selected for its strong performance on tabular data, efficient handling of sparse inputs, and fast inference. To address clinical class imbalance and reduce harmful under-triage errors, the training objective incorporates class weighting and additional penalties for misclassifying high-severity patients into lower-risk categories, prioritizing patient safety over unweighted accuracy.

3) *Nurse Agent*: Following the initial triage, a patient in a real ED would interact with a triage nurse to have their vital signs measured and their medical history recorded. The system simulates this critical data-gathering phase via the Nurse Agent.

Unlike the other agents, this agent does not rely on a predictive model. Instead, it acquires clinical information through an interactive dialogue, collecting vital signs, timestamped

measurements, medication use, and medical history, while preserving missing values.

A preprocessing pipeline transforms these inputs into the final feature representation, including vital-sign trends, abnormality indicators, medication categories, and history features, using the same processing steps during training and inference. Raw numerical inputs are constrained to biologically plausible ranges (e.g., heart rate: 20–250 bpm) to prevent erroneous values from corrupting the feature vector and affecting downstream predictions.

To ensure robust downstream predictions, the module applies automated data preprocessing. Physiological values are constrained within biologically plausible ranges to prevent erroneous inputs from affecting the model.

When the LLM parses patient responses, any information that is unknown or not provided by the patient is simply marked as missing (e.g., using -1, 'None', or 'NaN') rather than being estimated. To remain consistent with the way the models were trained, the Doctor Disposition, Nurse, and Triage tools then handle these missing values using median imputation based on the training set, together with missingness indicators that allow the models to capture the information carried by the absence of a measurement. In addition, free-text medication entries are grouped into broad clinical categories, such as cardiovascular, anticoagulant, and respiratory medications, providing useful information about the patient's medical history without requiring direct access to electronic health records (EHRs).

At runtime, the nurse agent extracts the required clinical variables by asking the patient targeted questions using an LLM. Information already available in the patient record (e.g., past medical history, current medications, and other structured data) is retrieved through dedicated tools that provide access to the corresponding dataset fields. The agent is prompted to collect all features required by the downstream prediction model; when a required variable is unavailable from the dataset, it is requested directly from the patient. However, patients are not required to answer every question, and missing responses (e.g., declining to provide information about current medications) are handled as unavailable inputs.

4) *Doctor Agent*: The Doctor Agent represents the final decision-making component of the pipeline. It integrates the complete patient representation - including the NLP-parsed intake information, the predicted ESI level from the Triage agent, and the validated clinical data collected by the Nurse agent - to perform a multi-stage diagnostic inference process.

The Doctor agent is based on three different XGBoost trained models for each type of task. The first is a *diagnosis model*, which predicts the patient's primary diagnostic category. The second is a *department model*, responsible for recommending the appropriate hospital department. The third is a *disposition model*, which estimates the probability of hospital admission based on the complete clinical picture. To ensure that the predicted probabilities accurately reflect observed admission rates, the disposition model is calibrated

using isotonic regression, producing well-calibrated risk estimates suitable for clinical decision support.

The diagnostic workflow is explicitly divided into: stage 1 (initial assessment) and stage 2 (reassessment). During the stage 1, only the information available at triage is considered to generate an initial prediction of the patient’s primary diagnostic category (e.g., cardiovascular, respiratory, or gastrointestinal) together with the most appropriate hospital department for patient routing. This stage provides an early diagnostic hypothesis before incorporating additional clinical information obtained during the subsequent reassessment. Stage 2 functions as an advisory module that refines this prediction by estimating the most likely ICD code within the Stage 1 category, without altering the predicted diagnostic category itself.

IV. EXPERIMENTS AND RESULTS

A. Evaluation protocol

The evaluation is conducted at two levels. First, a quantitative evaluation assesses each model using feature vectors derived directly from the structured fields of the dataset across multiple test cases. Secondly, while the tabular evaluation uses structured feature vectors, real-world deployment requires an LLM to extract these features from free-text patient descriptions. To assess the impact of this parsing step, we performed an end-to-end evaluation on 250 synthetic natural-language cases derived from real emergency department presentations.

By comparing four input configurations that share the same prediction models but differ only in how the patient’s complaint reaches them, we isolate the error introduced by the LLM parser from the error of the predictive models themselves. This breakdown is reported in Section IV-D and Table IV.

Specifically, the tabular sets used for testing contain 83,617 encounters to predict acuity and disposition and 20,420 admitted encounters to predict the diagnosis, department and ICD code (80/20 stratified split). The 250 generated cases are split into 163 admitted and 87 discharged patients. The *filtered feature vector* and *direct invocation* both use the exact tabular data and never invoke the LLM, whereas the *Gemini Flash* and *MedGemma* based parser use the information extracted by the LLM from unstructured input. Consequently, the isolated impact of the parsing stage can be quantified as the performance difference between direct invocation and the corresponding parser-based configuration.

B. Tabular results on the held-out test sets

Table I summarizes the performance on the large test sets. For acuity prediction (ESI 1–5), on the 83,617 test cases, the model achieved 67.55% exact accuracy, with 98.18% of predictions within one ESI level of the ground truth. Given the ordinal nature of the ESI scale, the model also achieved a quadratically weighted Cohen’s K of 0.64 (using *cohen_kappa_score* from scikit).

For diagnosis and department, the table shows the Doctor’s initial assessment (triage data only) and its reassessment

after the vital signs, medication and past medical history are collected by the Nurse agent, emphasizing the value of the additional data collected.

Target (classes)	<i>n</i>	Headline metrics
ESI acuity (5)	83,617	exact 67.6%, within-1 98.2%, QWK 0.64
Disposition (2, calibrated)	83,617	acc 83.3%, AUC 0.914
Diagnosis (13), initial	20,420	top-1 60.1%, top-3 82.7%
Diagnosis (13), post-nurse	20,420	top-1 64.1% , top-3 85.7%
Department (11), initial	20,420	top-1 60.7%, top-3 92.3%
Department (11), post-nurse	20,420	top-1 70.8% , top-3 94.1%
Exact ICD (3-char, Stage-2)	20,420	union 64.9%, graded 0.90

TABLE I

TABULAR PERFORMANCE OF EACH PREDICTION STAGE ON TEST SETS.
QWK: QUADRATIC-WEIGHTED COHEN’S κ

The collected data brings a significant improvement in diagnosis in Top-1 from 60.1% to 64.1% and a large Top-1 gain in department from 60.7% to 70.8%. We have to mention that the department gain comes from a higher concentration around the majority class, meaning *General Medicine* department recall rises from 0.65 to 0.87 while some minority departments decrease, so the balanced accuracy drops 4.4pp, from 49.6% to 45.2%, as Top-1 rises, from 60.7% to 70.8%.

For diagnosis, the improvement is consistent across all classes. For exact ICD code prediction, however, strict top-5 recall does not fully capture the model’s clinical value. This metric is calculated only within the correct (oracle) ICD category. The Stage-2 model ranks all possible exact codes in that category, and a prediction is counted as correct only if the true 3-character ICD code appears in the top five suggestions. This makes it a very strict, all-or-nothing measure. A prediction that is clinically very close to the correct code, such as a neighboring code in the same ICD branch, receives the same score of zero as a completely unrelated prediction.

To account for these borderline cases, we also use three graded evaluation measures: cosine similarity between the ICD titles, cosine similarity between the title embeddings, and distance within the ICD hierarchy. For each case, these measures assign the highest similarity score between the true code and any of the top five predicted codes. Because an exact match always receives the maximum score, the graded metrics are always equal to or higher than the strict recall score. The difference between them represents the credit given to predictions that are clinically similar, even if they are not exact matches.

For example, within the correct ICD category, the strict top-5 recall is 0.69. However, the graded top-5 score based on title-embedding similarity increases to 0.91. This indicates that most of the apparent errors are actually predictions of codes that are very close to the correct one.

We performed hyperparameter optimization using the Optuna framework. Optuna’s multi-objective NSGA-II algorithm

was used to optimize the triage models, while separate single-objective optimization procedures were carried out for the doctor models. The results showed that the current manually selected configuration is already close to optimal, and no other configuration was found to perform better overall.

C. Probability calibration and operating points

A clinical decision-support system should not only prioritize patients according to their admission risk but also produce reliable admission probabilities, so that each operating threshold corresponds to the real admission rate. Table II compares the raw triage screening probability with the isotonic-calibrated Doctor disposition model, on the same 83,617-row test set. Calibration greatly lowers the expected calibration error (ECE), roughly twenty times (from 0.0746 to 0.0036), making the reliability curve essentially diagonal, while also improving the model’s ability to distinguish between patients who will and will not be admitted.

The table reports the four evaluation metrics used: AUC (Area Under the Curve), which measures ranking quality (higher values indicate better discrimination); Brier score, which quantifies the mean squared error of predicted probabilities; and ECE/MCE (Expected and Mean Calibration Error), which measure the average and worst-case differences between predicted probabilities and observed admission rates, respectively (lower values indicate better calibration).

TABLE II
DISPOSITION CALIBRATION QUALITY: RAW TRIAGE SCREEN VS. ISOTONIC-CALIBRATED DOCTOR MODEL.

Disposition model	AUC	Brier	ECE	MCE
Triage screen (uncalibrated)	0.864	0.150	0.0746	0.137
Doctor (isotonic-calibrated)	0.914	0.113	0.0036	0.015

Because the calibrated probabilities are trustworthy, the operating threshold becomes an intentional configurable value for clinical safety, rather than a tuning artifact. Table III shows how the lower threshold values lead to worse overall accuracy, but also a much lower under-triage rate (missed admissions). The deployed threshold of 0.40 sacrifices only 0.6pp of accuracy compared to 0.50 while lowering the under-triage rate from 23.5% to 18.1% (sensitivity 0.765 $\rightarrow$ 0.819). The threshold can be lowered even further to reach even stricter safety targets (e.g. 13.5% under-triage at 0.30). This directly addresses the mortality risk associated with under-triage, highlighted in the introduction. The department model is also isotonically calibrated (ECE 0.019), which accounts for most of its post-nurse top-1 gain, though concentrated on the majority class, as we noted before.

The table reports Doctor disposition operating points at the deployed probability threshold of 0.40. Under-triage and over-triage represent missed admissions (1–Sens) and unnecessary admissions (1–Spec), respectively. BA (Balanced Accuracy) is defined as the mean of sensitivity and specificity.

TABLE III
DOCTOR DISPOSITION OPERATING POINTS (CALIBRATED MODEL)

Threshold	Accuracy	Sensitivity	Specificity	Under	Over	BA
0.15	0.750	0.946	0.636	0.054	0.364	0.791
0.20	0.782	0.924	0.699	0.076	0.301	0.811
0.30	0.820	0.865	0.793	0.135	0.207	0.829
0.40	0.833	0.819	0.842	0.181	0.158	0.830
0.50	0.840	0.765	0.883	0.235	0.117	0.824

D. End-to-end evaluation and the cost of the NLP parser

Table IV reports the four configurations on the 250 generated cases. The two LLM-free configurations (filtered feature vector and direct invocation) are almost identical, so the prediction pipeline itself incurs negligible loss. Almost the entire end-to-end degradation comes from the LLM parser. Measured as the drop from direct invocation to the Flash parser, the isolated parser cost is -2.0 pp for acuity and -2.4 pp for disposition, but -6.8 pp for diagnosis, -8.6 pp for the ICD union, and -11.7 pp for department. Acuity and disposition remain relatively robust to parsing errors, whereas tasks that depend heavily on interpreting the patient’s complaints are more sensitive. This is likely because the parser must translate lay descriptions into the abbreviated clinical terminology used in medical records (e.g. converting “brain bleed” into “subdural haemorrhage”), which is a lossy process. We note that the lower end-to-end performance numbers reflect this extraction cost together with the much smaller sample size. Therefore, they are not a direct comparison against the full tabular test sets.

The two LLM-free configurations show comparable performance, indicating that the remaining degradation observed in the parser-based configurations can be attributed to the NLP parser.

TABLE IV
END-TO-END BREAKDOWN ON THE 250 SYNTHETIC CASES (DIAGNOSIS, DEPARTMENT, AND ICD ON THE 163 ADMITTED PATIENTS).

Metric	Feat. vec.	Direct	Flash	MedG.
Acuity, exact accuracy	68.4%	68.0%	66.0%	60.4%
Disposition, acc. @0.40	82.0%	81.6%	79.2%	78.4%
Top-1 diagnosis	47.2%	49.1%	42.3%	42.3%
Top-1 department	58.9%	59.5%	47.9%	49.7%
ICD, candidate union	53.4%	55.2%	46.6%	46.0%

E. Language-model backend comparison

Table V presents the results for the two switchable parser back-ends, the hosted Gemini 2.5 Flash and the self-hosted MedGemma-4B, in the same parser configuration (on 250 cases). Flash more accurately translates patients’ lay descriptions into the abbreviated clinical terminology used in medical records, as reflected by its higher complaint-token Jaccard score. This leads to decisively better ESI acuity and disposition, marginally better exact-ICD, and a nearly unbiased acuity error (mean signed error $+0.008$ vs. MedGemma’s -0.152 , indicating that MedGemma-4B consistently assigns patients to higher acuity levels than necessary). MedGemma wins department top-1 by 1.8pp, and diagnosis is essentially a

tie (0.423/0.626 top-1/top-3). Based on these results, Gemini 2.5 Flash is the deployed backend, but the parser-related performance loss is similar for both backends, indicating that the remaining gap comes from the challenges of parsing free text, rather than from the specific model used.

TABLE V
FLASH VS. MEDGEMMA-4B AS THE NLP PARSE. QWK:
QUADRATIC-WEIGHTED κ ON ACUITY.

Metric	Flash	MedGemma
Complaint-token Jaccard	0.598	0.529
Acuity, exact accuracy	0.660	0.604
Acuity, QWK	0.615	0.539
Disposition, ROC-AUC	0.888	0.873
Department, top-1	0.479	0.497
Exact ICD, union	0.466	0.460

F. An end-to-end running scenario

Fig. 2 presents a real clinical case processed through all six stages of the proposed pipeline, demonstrating its transparent design. Instead of producing only a final decision, each stage yields an interpretable intermediate output, allowing users to trace the reasoning process leading to the final diagnosis. The illustrated case involves an 83-year-old man who is brought to the emergency department by ambulance after a fall, presenting with severe left hip pain.

The parser extracts the structured fields, then Triage agent assigns ESI 3 score and an initial disposition of *admission*. The Doctor agent’s initial assessment classifies the case into the *Injury and Poisoning* category. The Nurse agent collects vital signs and medication history. Then, the calibrated disposition model confirms *admission* with high confidence ($P = 0.98$). The Doctor’s reassessment confirms the diagnosis, sends the patient to the *Orthopedics department*, and returns the exact ICD code *S72 (displaced femur fracture)* as its top candidate, via the advisory Stage-2 resolver. A certified clinician can follow and verify the entire chain of decisions, from “fall” and “hip pain” to a specific fracture code, at each step of the process. For this case, every stage of the pipeline produces the correct output. The predicted ESI level (3), disposition (admitted), specialty (Injury and Poisoning/Orthopedics), and ICD-10 code (S72) all match the ground truth.

```

Patient (free text):
"I'm 83 and I fell, now my left hip is just
killing me, a 10/10. The ambulance brought me."

[1] Parser -> {age:83, sex:M, arrival:ambulance,
  complaint:"fall, hip pain",
  pain:10/10}
[2] Triage -> ESI 3 (true: 3);
  initial disposition: ADMIT
[3] Doctor -> Dx#1: Injury and Poisoning
  initial (top-3: Injury, Musculoskel., Resp.)
[4] Nurse -> vitals + medication history
[5] Doctor -> refined P(admit)=0.98 >= 0.40
  dispo. => ADMIT (confirms triage screen)
[6] Doctor -> Dx: Injury and Poisoning;
  reassess Dept: ORTHO; exact ICD #1: S72
  "displaced femur fracture" (correct)

```

Fig. 2. End-to-end trace for a real case (stay 32839233).

V. DISCUSSIONS

The proposed multi-agent framework demonstrates that combining LLMs for natural language understanding with traditional ML models for clinical prediction offers a viable and safe alternative to end-to-end “black box” AI systems in emergency triage. By following the same step-by-step process used in real-world triage, the system delivers strong predictive performance while preserving the level of transparency and interpretability that clinicians need for decision support.

A key barrier to clinical AI adoption is the lack of transparency in model reasoning. Our architecture explicitly addresses this issue by separating information extraction from the statistical prediction model, making the system’s reasoning transparent and easy to evaluate. By relying on LLMs only to parse patient’s unstructured description into a structured schema, rather than generating clinical judgements, we eliminate the risk of hallucinated diagnoses. The pipeline provides verifiable intermediate outputs at each stage, allowing clinicians to identify and correct errors before they propagate. This transparency ensures that final predictions (ESI, disposition, department, diagnosis, and ICD code) are based on traceable clinical features, improving verification and supporting trust in critical emergency care settings.

A. Patient safety through probability calibration

For a clinical support system, raw accuracy is less important than the rate of critical under-triage, where critically-ill patients are assigned a lower urgency level than they should. Our tests showed that raw predictive outputs are often calibrated poorly. Our system applies isotonic calibration to the Doctor Agent’s disposition model, and as a result, the expected calibration error improved twenty-times, from 0.075 to 0.004.

This calibration allows hospitals to intentionally select thresholds based on clinical safety targets, allowing them to minimize under-triage rates, at the cost of lower raw accuracy. This flexibility directly addresses the risks of mortality which are associated with delayed emergency care, proving that the system can be adapted to the risk tolerance of different hospitals.

B. The bottleneck of natural language parsing

The impact of the parsing stage was not consistent across tasks. Predictions primarily driven by physiological information, such as ESI acuity level and patient disposition, remained relatively robust despite the introduction of the parser. In contrast, tasks that depend heavily on accurately interpreting the patient’s chief complaint - such as department assignment and precise ICD code prediction - experienced substantially greater performance degradation, with department prediction alone dropping by more than 10 percentage points. These findings suggest that translating free-text patient descriptions into concise clinical terminology remains an inherently lossy process.

The backend comparison further supports this finding. The hosted Gemini 2.5 Flash model achieved overall better results compared to the self-hosted MedGemma-4B model, largely

because it was more effective at translating patients' natural language into the standardized clinical terminology used in medical records. This superior text normalization is directly correlated to better ESI acuity assignment and disposition predictions, without the over-triage tendency observed with the smaller MedGemma model.

C. Limitations

Although the results are promising, this study has several limitations. First, the models were trained and evaluated on a single database (MIMIC-IV-ED), which reflects the demographic composition and operational practices of a single healthcare system, which may limit its use in other healthcare systems with different protocols.

Second, the end-to-end evaluation was conducted on a relatively small sample of only 250 generated cases. Although these cases were generated based on real encounters in the ED, they can not fully capture the real-world complexity, ambiguity, and conversational dynamics of live interactions between patients and hospital staff.

Diagnosis prediction is performed using 13 ICD-based categories instead of individual ICD codes. The subsequent exact ICD code prediction is intended only as a supportive step and was not validated against clinical outcomes. Similarly, department assignment uses 11 categories created by merging the original 19 hospital departments. While this simplification improves statistical reliability, it may not provide the level of detail required for applications such as routing patients to specific subspecialties.

VI. CONCLUSION AND FUTURE WORK

This paper presented an interpretable, pipeline-based clinical decision support prototype designed for the emergency department. By separating natural language understanding from deterministic ML, the proposed architecture combines the flexibility of LLMs for information extraction with the robustness, transparency, and reproducibility of tabular predictive models.

The system was trained and tested using MIMIC-IV-ED dataset which reflects the challenges of real-world clinical data, including missing observations, irregular sampling, and noisy free-text chief complaints containing abbreviations and spelling variations. The evaluation demonstrated that the modular design enables accurate prediction across multiple clinical tasks, including triage acuity, diagnostic categorization, department recommendation, and hospital disposition. Safety-oriented training strategies, such as class weighting for critical ESI levels and calibrated admission prediction, improve the reliability of clinically important decisions. Furthermore, adding progressively richer clinical information - from initial triage data to longitudinal vital signs, medication history, and past medical history - substantially enhances downstream diagnostic performance.

As a future work, the architecture can be extended to support multimodal inputs, allowing vision-language models to extract structured findings from diagnostic data such as ECG or EKG.

These features can then be incorporated into the deterministic prediction models to enhance diagnostic performance.

Our framework relies on modeling the sequential transition from an initial triage assessment to a refined assessment after supplementary data collection, requiring datasets that preserve the arrival-to-decision workflow. Among the available public datasets, only MIMIC-IV-ED provides this structure, motivating its exclusive use for training and validation. Future work will investigate the adaptation and external validation of the framework on datasets such as eICU-CRD [19] to assess its generalizability across different clinical settings.

A key future objective will be to transition the prototype from a standalone research system into a clinically integrated decision-support platform, followed by evaluation through simulated clinical workflows involving clinicians. These user studies will measure the system's impact on triage efficiency, diagnostic consistency, and clinician workload, evaluating its usefulness in realistic clinical scenarios. All these efforts aim to advance the proposed modular and interpretable architecture toward real-world deployment in emergency care.

REFERENCES

- [1] Oecd, *Health at a Glance 2025: OECD Indicators*, ser. Health at a Glance. OECD, Nov. 2025. [Online]. Available: https://www.oecd.org/content/dam/oecd/en/publications/reports/2025/11/health-at-a-glance-2025_a894f72e/8f9e3f98-en.pdf
- [2] European Society for Emergency Medicine (EUSEM), "European emergency medicine in numbers," <https://eusem.org>, Jul. 2020, accessed: 2026-07-06. [Online]. Available: https://eusem.org/images/European_EM_in_numbers.pdf
- [3] N. Gilboy, P. Tanabe, D. Travers, and A. M. Rosenau, "Emergency severity index (ESI): A triage tool for emergency department care, version 4," Agency for Healthcare Research and Quality (AHRQ), Publication No. 12-0014, 2012. [Online]. Available: https://sgnorch.fileadmin/user_upload/Dokumente/Downloads/Esi_Handbook.pdf
- [4] Y. Raita, T. Goto, M. K. Faridi, D. F. M. Brown, C. A. Camargo, and K. Hasegawa, "Emergency department triage prediction of clinical outcomes using machine learning models," *Critical Care*, vol. 23, no. 1, p. 64, 2019. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/30795786/>
- [5] L.-H. Yao, K.-C. Leung, C.-L. Tsai, C.-H. Huang, and L.-C. Fu, "A novel deep learning-based system for triage in the emergency department using electronic medical records: Retrospective cohort study," *Journal of Medical Internet Research*, vol. 23, no. 12, p. e27008, 2021. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/34958305/>
- [6] A. Vântu, A. Vasilescu, and A. Băicoianu, "Medical emergency department triage data processing using a machine-learning solution," *Heliyon*, vol. 9, no. 8, p. e18402, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405844023056104>
- [7] B. M. Porto, "Improving triage performance in emergency departments using machine learning and natural language processing: a systematic review," *BMC Emerg. Med.*, vol. 24, no. 1, p. 219, Nov. 2024. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/39558255/>
- [8] L. Masannek, L. Schmidt, A. Seifert, T. Kölsche, N. Huntemann, R. Jansen, M. Mehsin, M. Bernhard, S. G. Meuth, L. Böhm, and M. Pawlitzki, "Triage performance across large language models, ChatGPT, and untrained doctors in emergency medicine: Comparative study," *J. Med. Internet Res.*, vol. 26, p. e53297, Jun. 2024. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/38875696/>
- [9] G. B. Haim, M. Saban, Y. Barash, D. Cirulnik, A. Shaham, B. Z. Eisenman, L. Burshtein, O. Mymon, and E. Klang, "Evaluating large language model-assisted emergency triage: A comparison of acuity assessments by GPT-4 and medical experts," *J. Clin. Nurs.*, Nov. 2024. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1111/jocn.17490>

- [10] K. Le, J. Chen, D. Mai, and K. D. R. Le, "An evaluation on the potential of large language models for use in trauma triage," *Emergency Care and Medicine*, vol. 1, no. 4, pp. 350–367, 2024. [Online]. Available: <https://www.mdpi.com/2813-7914/1/4/35>
- [11] L. Cui, M. Wang, X. Ye, W. Wei, and X. Zhao, "Diagnostic accuracy of large language models for emergency department triage: a systematic review and meta-analysis," *BMC Emergency Medicine*, 06 2026. [Online]. Available: https://www.researchgate.net/publication/407078638_Diagnostic_accuracy_of_large_language_models_for_emergency_department_triage_a_systematic_review_and_meta-analysis
- [12] J. Lee, T. Shang, J. Y. Baik, D. A. Duong-Tran, S. Yang, L. Li, and L. Shen, "Investigating llms in clinical triage: Promising capabilities, persistent intersectional biases," *ArXiv*, vol. abs/2504.16273, 2025. [Online]. Available: <https://api.semanticscholar.org/CorpusID:280919406>
- [13] M. Lu, B. Ho, D. Ren, and X. Wang, "TriageAgent: Towards better multi-agents collaborations for large language model-based clinical triage," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 5747–5764. [Online]. Available: <https://aclanthology.org/2024.findings-emnlp.329/>
- [14] Z. Chen, Z. Peng, X. Liang, C. Wang, P. Liang, L. Zeng, M. Ju, and Y. Yuan, "MAP: evaluation and multi-agent enhancement of large language models for inpatient pathways," *Npj Health Syst.*, vol. 3, no. 1, Jun. 2026.
- [15] M. Chen, Y. Wu, J. Ma, X. Jia, C. Gao, F. Zhao, and Y. Qiao, "Independent and collaborative performance of large language models and healthcare professionals in diagnosis and triage," *NPJ Digit. Med.*, vol. 9, no. 1, Feb. 2026.
- [16] A. Johnson, L. Bulgarelli, T. Pollard, L. A. Celi, R. Mark, and S. Horng, "MIMIC-IV-ED," *PhysioNet*, Jan. 2023, version 2.2. [Online]. Available: <https://doi.org/10.13026/5ntk-km72>
- [17] T. J. Pollard, A. E. W. Johnson, J. D. Raffa, L. A. Celi, R. G. Mark, and O. Badawi, "The eICU collaborative research database, a freely available multi-center database for critical care research," *Sci. Data*, vol. 5, no. 1, p. 180178, Sep. 2018. [Online]. Available: <https://www.nature.com/articles/sdata2018178>
- [18] Y. Zhang, H. Yu, L. Hu, H. Jin, and H. Zhang, "General modular harness for llm agents in multi-turn gaming environments," 2025. [Online]. Available: <https://arxiv.org/abs/2507.11633>
- [19] T. J. Pollard, A. E. W. Johnson, J. D. Raffa, L. A. Celi, R. G. Mark, and O. Badawi, "The eICU collaborative research database, a freely available multi-center database for critical care research," *Sci. Data*, vol. 5, no. 1, p. 180178, Sep. 2018.